

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	10
РАЗДЕЛ 1. ПРОЕКТИРОВАНИЕ ФУНКЦИОНАЛЬНОЙ СТРУКТУРЫ АСУ	
.....	12
Введение.....	12
Описание предметной области и обоснование выбора участка	
автоматизации.....	12
Развитие концепции (как работает система).....	14
Функциональная структура и бизнес-процессы.....	15
Выводы.....	20
РАЗДЕЛ 2. ФУНКЦИОНАЛЬНЫЕ И НЕФУНКЦИОНАЛЬНЫЕ	
ТРЕБОВАНИЯ К АСУ.....	21
Введение.....	21
Функциональные требования.....	21
Нефункциональные требования.....	24
Выводы.....	27
РАЗДЕЛ 3. UML МОДЕЛИРОВАНИЕ: ВАРИАНТЫ ИСПОЛЬЗОВАНИЯ....	29
Введение.....	29
Определение модулей проектируемой АСУ.....	29
Перечень пользователей системы.....	30
Модуль 1: «Интерфейс взаимодействия с системой».....	31
Модуль 2: «Управление контентом и каталогом».....	31
Модуль 3: «Аналитика и администрирование системы».....	32
Внутрисистемные функции.....	33
Диаграммы вариантов использования.....	36
Выводы.....	38
РАЗДЕЛ 4. UML МОДЕЛИРОВАНИЕ: ВЗАИМОДЕЙСТВИЕ И	
СТАТИЧЕСКАЯ СТРУКТУРА.....	40
Введение.....	40

Диаграммы последовательности.....	41
Диаграмма классов.....	42
Выводы.....	43
РАЗДЕЛ 5. ВЫБОР АППАРАТНОЙ И ПРОГРАММНОЙ ПЛАТФОРМЫ.....	44
Введение.....	44
Таблица аппаратного обеспечения АСУ по выбранной тематике и обеспечивающих подсистем.....	44
Таблица программного обеспечения для выполнения всех функций описанных в требованиях и отображенных на UML диаграммах.....	47
Выводы.....	51
РАЗДЕЛ 6. РЕАЛИЗАЦИЯ ПРОГРАММНОГО МОДУЛЯ ДООБУЧЕНИЯ НЕЙРОСЕТИ.....	52
Введение.....	52
Функция, требующая доработки.....	52
Анализ существующих языков программирования и IDE.....	53
Языки программирования.....	53
Интегрированные среды разработки (IDE).....	54
Обоснованный выбор инструментов.....	55
Выбор и описание библиотек.....	56
Программный код.....	57
Выводы.....	60
РАЗДЕЛ 7. ОПИСАНИЕ КЛАССОВ И ПОСТРОЕНИЕ ER-ДИАГРАММЫ...	61
Введение.....	61
Выделение и описание полей (свойств) и методов выбранных классов.....	62
Построение ER-диаграммы на основе классовой диаграммы.....	72
Выводы.....	72
РАЗДЕЛ 8. СОЗДАНИЕ БАЗЫ ДАННЫХ В POSTGRESQL И ЗАПОЛНЕНИЕ ДАННЫМИ.....	73
Введение.....	73

Выбор реляционной СУБД.....	73
PostgreSQL.....	73
MySQL (Community Edition).....	74
SQLite.....	75
Сравнительный анализ.....	75
Преимущества и недостатки рассмотренных СУБД.....	76
Выбор СУБД для создания базы данных проектируемой АСУ.....	78
Создание базы данных в выбранной СУБД и заполнение ее данными.....	78
Выводы.....	91
РАЗДЕЛ 9. ДЕТАЛИЗАЦИЯ ХАРАКТЕРИСТИК ОБОРУДОВАНИЯ.....	92
Введение.....	92
Клиентское оборудование.....	92
Серверное оборудование.....	93
Балансировщик нагрузки.....	93
Серверы приложений (Kubernetes Worker Nodes).....	93
Серверы баз данных (PostgreSQL).....	94
Серверы поискового движка (Elasticsearch).....	94
Серверы для нейросетевых моделей (NLP / PyTorch).....	95
Файловое хранилище (S3-совместимое).....	95
Инфраструктурные и вспомогательные серверы.....	96
Сетевое оборудование.....	96
Выводы.....	96
РАЗДЕЛ 10. СОЗДАНИЕ ВИРТУАЛЬНОГО СТЕНДА.....	97
Введение.....	97
Сравнение сред виртуализации и обоснование выбора.....	97
Критерии сравнения.....	97
Анализ преимуществ и недостатков.....	98
Обоснование выбора среды виртуализации.....	98
Проектирование виртуального стенда.....	99

Архитектура стенда.....	99
Распределение ПО по виртуальным машинам.....	99
Аппаратные ресурсы хоста и распределение.....	100
Результаты создания виртуальной инфраструктуры.....	100
Выводы.....	105
РАЗДЕЛ 11. РАЗРАБОТКА ИНФОРМАЦИОННОГО WEB-ПОРТАЛА ДЛЯ	
ПОЛЬЗОВАТЕЛЕЙ.....	106
Введение.....	106
Описание разработанного портала.....	106
Использованные технологии и соответствие требованиям.....	107
Результаты работы.....	108
Вкладка «О системе».....	108
Вкладка «Поиск контента».....	108
Вкладка «Результаты работы».....	109
Написанный код.....	109
Выводы.....	118
РАЗДЕЛ 12. РАЗРАБОТКА МОДУЛЯ МЕНЕДЖЕРА КОНТЕНТА ДЛЯ	
РАБОТЫ С БД.....	119
Введение.....	119
Обоснование выбора модуля.....	119
Архитектура и технологии модуля.....	120
Функциональные возможности модуля.....	120
Структура базы данных и SQL-запросы.....	120
Описание веб-интерфейса.....	121
Результаты работы модуля.....	122
Выводы.....	124
РАЗДЕЛ 13. СХЕМА СЕТИ И АНАЛИЗ УГРОЗ ИНФОРМАЦИОННОЙ	
БЕЗОПАСНОСТИ.....	125
Введение.....	125

Схема компьютерной сети проектируемой АСУ «Семантический подборщик».....	125
Определение проблем информационной безопасности для построенной сети.....	126
Угрозы безопасности хранения и обработки информации на рабочих станциях.....	126
Угрозы безопасности передачи информации по компьютерной сети....	127
Выводы.....	128
РАЗДЕЛ 14. ВЫБОР СРЕДСТВ ЗАЩИТЫ ИНФОРМАЦИИ НА РАБОЧИХ СТАНЦИЯХ.....	129
Введение.....	129
Анализ угроз безопасности для рабочих станций проектируемой АСУ....	129
Требования к защите рабочих станций.....	130
Сравнительный анализ программных средств защиты.....	131
Выбор и описание программных средств защиты для рабочих станций...	133
Выводы.....	135
РАЗДЕЛ 15. ВЫБОР СРЕДСТВ ЗАЩИТЫ ПЕРЕДАЧИ ИНФОРМАЦИИ ПО СЕТИ.....	136
Введение.....	136
Анализ угроз безопасности передачи информации в сети проектируемой АСУ.....	136
Требования к средствам защиты передачи данных.....	137
Сравнительный анализ существующих средств обеспечения безопасности передачи информации.....	138
Выбор и обоснование комплекса средств защиты.....	139
Описание возможностей выбранных средств с учётом угроз.....	140
TLS 1.3 / HTTPS.....	140
NGFW (ViPNet Coordinator HW).....	140
Istio Service Mesh (mTLS).....	140

WireGuard.....	141
Выводы.....	141
ЗАКЛЮЧЕНИЕ.....	142
СПИСОК ИСТОЧНИКОВ.....	144

ВВЕДЕНИЕ

В современных условиях цифровой трансформации промышленных и административных объектов автоматизация процессов подбора, обработки и предоставления мультимедийного контента становится важной задачей. Традиционные методы поиска по ключевым словам не позволяют в полной мере учитывать семантику запросов пользователей, контекст предыдущих обращений и уровень подготовки потребителя. Применение корпоративных нейросетевых моделей и интеллектуальных систем поддержки принятия решений открывает новые возможности для создания автоматизированных систем управления (АСУ), способных работать с естественно-языковыми запросами в круглосуточном режиме.

Объектом проектирования в данной курсовой работе является автоматизированная система управления подбором и продажей мультимедиа контента – АСУ «Семантический подборщик». Предметом проектирования выступают методы, алгоритмы, программно-аппаратные средства и организационные решения, обеспечивающие функционирование указанной системы.

Цель курсовой работы – разработать комплексный проект АСУ «Семантический подборщик», включающий функциональную структуру, формализованные требования, UML-модели, базу данных, аппаратную платформу, виртуальный стенд, web-интерфейсы и средства защиты информации, а также продемонстрировать практическую реализацию ключевых модулей.

Задачи курсовой работы:

1. Определить участок деятельности организации, подлежащий автоматизации, и спроектировать функциональную структуру АСУ.
2. Сформулировать функциональные и нефункциональные требования к системе.

3. Построить UML-диаграммы (вариантов использования, последовательности, классов) для визуализации взаимодействия и статической структуры.
4. Выбрать и детализировать программно-аппаратную платформу, обосновать выбор языка, IDE и библиотек, реализовать модуль дообучения нейросети.
5. Создать ER-диаграмму и физическую базу данных в PostgreSQL, заполнить её тестовыми данными.
6. Разработать схему компьютерной сети, выявить угрозы информационной безопасности и выбрать средства защиты для рабочих станций и каналов передачи данных.
7. Реализовать web-портал для пользователей и административный модуль для управления контентом.
8. Собрать виртуальный стенд для интеграционного тестирования всех компонентов.

Структура курсовой работы. Работа состоит из введения, 15 основных разделов (каждый соответствует одной практической работе), заключения и списка источников. В разделах последовательно отражаются этапы проектирования: от функционального анализа и требований до реализации программных модулей, базы данных, интерфейсов и средств защиты информации. Такой подход позволяет поэтапно продемонстрировать полный жизненный цикл создания АСУ – от концепции до рабочего прототипа.

РАЗДЕЛ 1. ПРОЕКТИРОВАНИЕ ФУНКЦИОНАЛЬНОЙ СТРУКТУРЫ АСУ

Введение

В рамках общей курсовой работы по проектированию АСУ «Семантический подборщик» первым этапом является определение границ автоматизации и формирование концептуальной функциональной структуры. Данный раздел базируется на материалах практической работы №1 и закладывает основу для всех последующих разделов: именно здесь выбирается участок деятельности организации, обосновывается тип АСУ, выделяются бизнес-процессы, роли пользователей и логические подсистемы.

Цель раздела 1 – на основе анализа деятельности организации спроектировать функциональную структуру АСУ для автоматизации обработки естественно-языковых запросов пользователей и интеллектуального подбора мультимедийного контента.

Задачи раздела:

1. Собрать информацию из открытых источников по направлению деятельности организации.
2. Определить участок, требующий автоматизации, с обоснованием.
3. Выбрать тип проектируемой АСУ.
4. Описать логическую структуру, функции и пользователей.
5. Определить жизненный цикл проектирования и учесть нормативно-правовые документы.

Описание предметной области и обоснование выбора участка автоматизации

Наименование проектируемой системы:

Автоматизированная система управления подбором мультимедийного контента по естественно-языковым запросам с использованием корпоративной нейросети (АСУ «Семантический подборщик»).

Направление деятельности организации:

Предоставление образовательного и развлекательного мультимедиа контента (электронные книги, аудиокниги, видеоуроки, фильмы, подкасты) пользователям на основе их потребностей, уровня подготовки и предпочтений. Организация может выступать как онлайн-библиотека, образовательная платформа или медиатека.

Анализ текущей ситуации (AS-IS):

В настоящее время подбор контента часто осуществляется либо полностью вручную (менеджер ищет материалы по запросу клиента), либо с использованием простого текстового поиска по ключевым словам. Это приводит к следующим проблемам:

- запросы на естественном языке требуют дополнительной интерпретации;
- не учитывается контекст предыдущих обращений;
- поиск ограничен только имеющимся контентом, хотя может существовать релевантный внешний контент, который организация могла бы приобрести или лицензировать;
- отсутствует автоматизация процесса поиска новых источников и согласования их приобретения.

Участок, требующий автоматизации:

Процесс обработки запросов пользователей на естественном языке и интеллектуального подбора мультимедиа контента – от получения запроса до предоставления пользователю подобранных материалов с возможностью последующего уточнения.

Ключевое нововведение: вместо человека-менеджера для перевода запроса в формальные параметры используется собственная нейросеть предприятия, которая всегда доступна и обеспечивает автоматическую интерпретацию.

Автоматизации подлежат:

- семантический анализ запроса с помощью нейросети;
- поиск по внутренней базе контента;
- инициирование поиска нового контента во внешних источниках (при необходимости);
- ведение истории взаимодействия для уточняющих запросов;
- формирование рекомендаций с учётом уровня подготовки пользователя.

Тип проектируемой АСУ:

Согласно классификации, данная система относится к СППР (система поддержки принятия решений) с элементами системы управления знаниями, так как она помогает пользователю принимать решение о выборе контента на основе анализа семантики и контекста. Также присутствуют черты CRM-системы для учёта взаимодействий с пользователями. Использование собственной нейросети делает систему интеллектуальной АСУ с элементами искусственного интеллекта.

Развитие концепции (как работает система)

Основная идея (ТО-ВЕ)

Пользователь формулирует запрос в свободной форме, например: «хочу изучить программирование на языке Go».

Запрос поступает непосредственно в систему, где активируется корпоративная нейросеть. Эта нейросеть принадлежит предприятию, развёрнута на его мощностях (или в защищённом облаке) и всегда доступна для обработки. Она выполняет следующие шаги:

1. Семантический анализ запроса – нейросеть выделяет ключевые понятия: тема (программирование, Go), цель (изучение), уровень (начальный, подразумевается по умолчанию), а также определяет тип желаемого контента (книга, видео, аудио), если это явно не указано.

2. Формализация запроса – преобразование естественно-языковой фразы в структурированный набор параметров для системы поиска (например, {тема: "Go", уровень: "начальный", тип: ["книга", "видео"]}).
3. Поиск по внутреннему репозиторию – система ищет имеющиеся материалы, соответствующие формализованному запросу.
4. Оценка достаточности – если найденного контента недостаточно (по количеству, качеству или уровню), система инициирует поиск нового контента во внешних источниках (открытые образовательные платформы, магазины контента, базы знаний).
5. Предоставление результатов – пользователю выдаётся список подобранных материалов с возможностью сортировки и фильтрации.
6. Уточняющий запрос – пользователь может сказать: «я уже не глупый программист и хорошо знаю Go, мне нужен более продвинутый уровень». Нейросеть снова анализирует фразу, учитывает предыдущий контекст (историю запросов) и корректирует параметры поиска, переходя на продвинутый уровень.

Таким образом, система работает как интеллектуальный помощник, который понимает смысл, уровень подготовки и контекст, а при необходимости расширяет базу контента. Отсутствие человека-менеджера на этапе интерпретации ускоряет обработку и исключает субъективные ошибки.

Функциональная структура и бизнес-процессы

1. Бизнес-процессы, протекающие на автоматизируемом участке

Используем нотацию IDEF0 для описания основных процессов. Контекстная диаграмма бизнес-процесса представлена на рис. 1, а её декомпозиция на рис. 2.

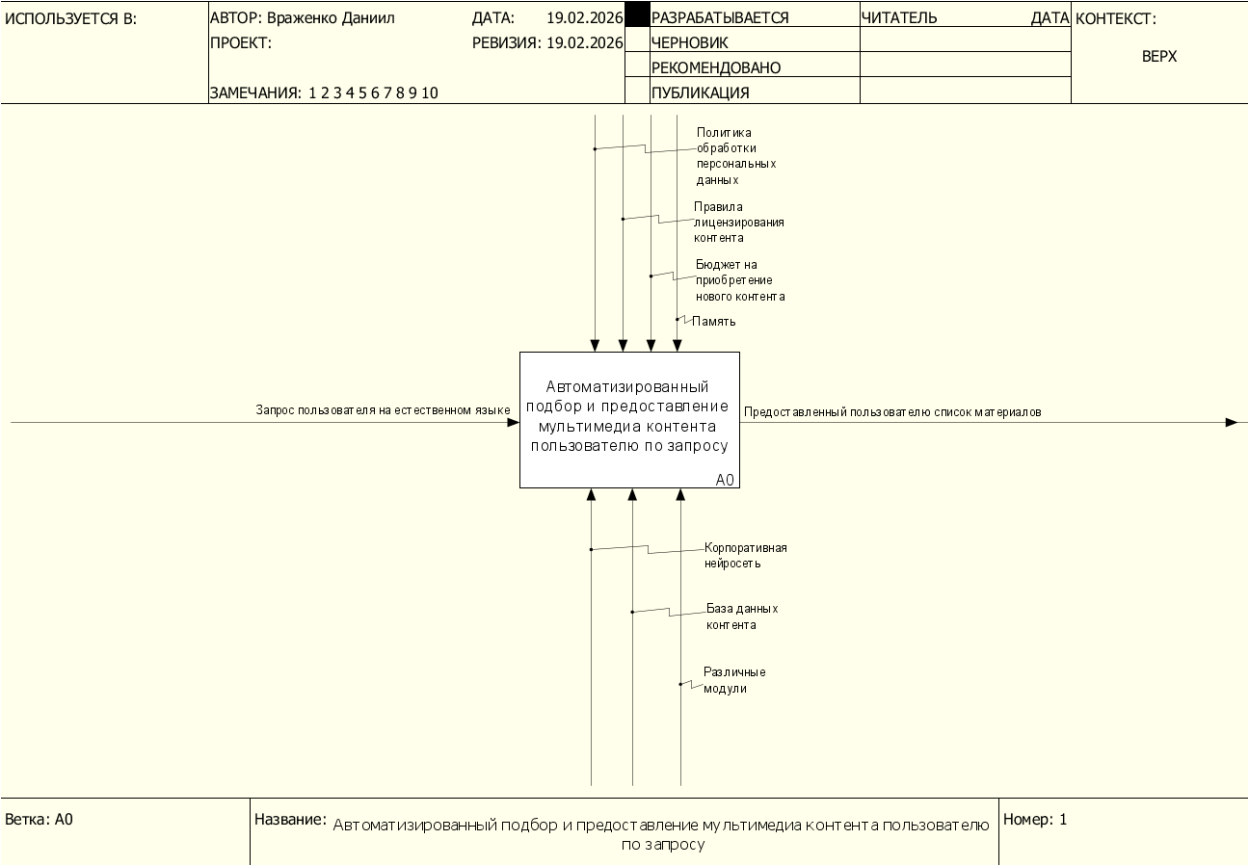


Рисунок 1 – Бизнес-процесс «Автоматизированный подбор и предоставление мультимедиа контента пользователю по запросу»

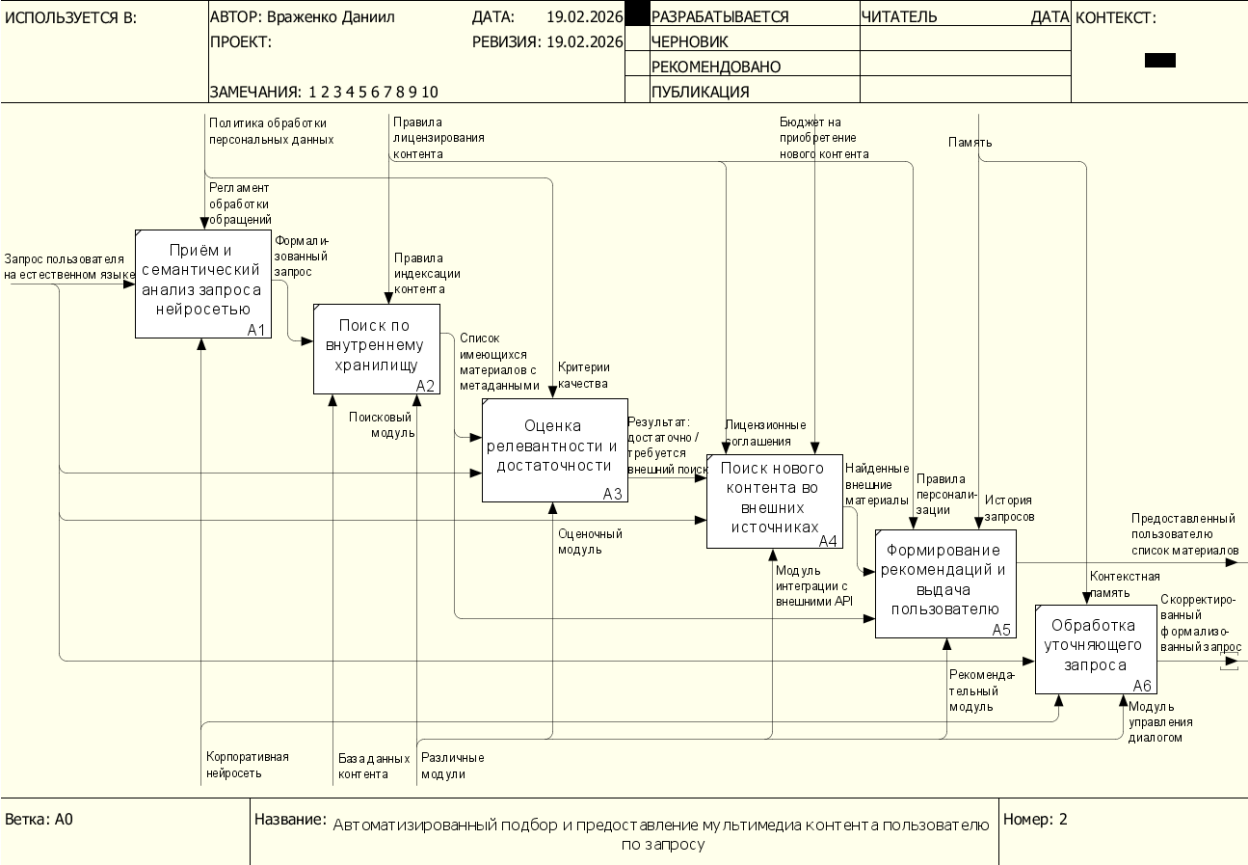


Рисунок 2 – Декомпозиция бизнес-процесса «Автоматизированный подбор и предоставление мультимедиа контента пользователю по запросу»

2. Логическая структура АСУ

Система состоит из следующих функциональных подсистем:

Подсистема взаимодействия (Front-office):

- Интерфейс ввода запросов (текстовое поле, возможно голосовой ввод).
- Отображение результатов (списки, карточки контента, фильтры).
- Личный кабинет пользователя (история запросов, избранное).

2. Подсистема семантического анализа на базе корпоративной нейросети:

- Модуль распознавания сущностей (тема, уровень, тип контента).
- Модуль определения интенга (цель запроса).
- Модуль учёта контекста (история диалога).
- Модель нейросети, обученная на предметной области и размещённая на серверах предприятия.

3. Подсистема управления контентом (Content Management):

- База данных метаданных контента (название, автор, тип, уровень сложности, ссылка на файл/ресурс).
- Модуль индексации и полнотекстового поиска.
- Модуль учёта прав доступа и лицензий.

4. Подсистема внешнего поиска и интеграции:

- Адаптеры для подключения к внешним источникам (API онлайн-библиотек, образовательных платформ, магазинов контента).
- Модуль автоматического согласования приобретения (при наличии бюджета).

5. Подсистема поддержки принятия решений (рекомендательная):

- Модуль ранжирования результатов по релевантности.
- Модуль персонализации (на основе истории и профиля

пользователя).

6. Подсистема управления диалогом и историей:

- Хранение всех запросов пользователя.
- Модуль обновления контекста при уточнениях.

3. Основные функции проектируемой АСУ

1. Интерпретация естественно-языковых запросов с помощью нейросети – преобразование фраз пользователя в формальные параметры поиска.

2. Контекстно-зависимый поиск – учёт предыдущих запросов.

3. Мультиформатный поиск – одновременный поиск по книгам, аудио, видео.

4. Оценка полноты имеющейся базы – автоматическое определение необходимости поиска нового контента.

5. Автоматизированный поиск внешних источников – обращение к партнёрским API, сбор информации о доступных материалах.

6. Рекомендация с учётом уровня – подбор контента, соответствующего заявленному уровню подготовки.

7. Ведение истории диалога – сохранение контекста для уточняющих запросов.

8. Непрерывная доступность – корпоративная нейросеть работает 24/7 без участия человека.

4. Должностные лица (пользователи системы)

1. Пользователь (клиент):

- Физическое лицо, желающее получить контент.
- Может быть гостем (без регистрации) или зарегистрированным пользователем.
- Взаимодействует с системой через интерфейс ввода запросов и просмотра результатов.

2. Контент-менеджер:

- Отвечает за наполнение внутренней базы контента, добавление метаданных, обновление информации.
- Утверждает приобретение нового контента по результатам внешнего поиска.

3. Администратор системы (Инженер машинного обучения):

- Технический специалист, отвечающий за обучение, дообучение и поддержку корпоративной нейросети.
- Настраивает модули интеграции с внешними источниками, управляет доступом.

4. Руководитель отдела контента:

- Определяет стратегию пополнения базы, утверждает бюджет на внешние закупки.

5. Жизненный цикл проектирования

1. Формирование требований: анализ деятельности, определение границ автоматизации.
2. Разработка концепции – создание моделей бизнес-процессов и функциональной структуры.
3. Техническое задание – детализация требований к нейросети (архитектура, объём обучающих данных), модулям поиска и интеграции.
4. Технический проект – разработка архитектуры базы данных, выбор алгоритмов нейросети, проектирование API.
5. Рабочая документация – кодирование, обучение нейросети, создание интерфейсов.
6. Ввод в действие – тестирование на реальных запросах, обучение контент-менеджеров работе с системой.
7. Сопровождение – дообучение нейросети на новых данных, добавление новых источников контента.

6. Нормативно-правовые документы

- ФЗ №152 «О персональных данных» – при регистрации пользователей и ведении истории запросов.
- Гражданский кодекс РФ (часть 4) – авторские права на мультимедиа контент, лицензионные договоры.
- Закон «О защите прав потребителей» – при предоставлении платного контента.
- Политика конфиденциальности и пользовательское соглашение – разрабатываются для платформы.
- Стандарты на интерфейсы API внешних поставщиков контента (договорные обязательства).
- Внутренние регламенты по использованию нейросетей (ответственность за результаты анализа, защита модели от несанкционированного доступа).

Выводы

В разделе 1 спроектирована функциональная структура АСУ «Семантический подборщик». Ключевым решением является замена человека-менеджера корпоративной нейросетью на этапе интерпретации естественно-языковых запросов, что обеспечивает доступность 24/7, скорость и точность обработки. Определены бизнес-процессы (контекстная диаграмма и декомпозиция), выделены 6 логических подсистем, 8 основных функций и 4 категории пользователей (клиент, контент-менеджер, администратор нейросети, руководитель). Учтены требования ФЗ №152, ГК РФ и этапы жизненного цикла. Результаты раздела 1 являются фундаментом для формулирования требований (раздел 2), UML-моделирования (раздел 3) и выбора программно-аппаратной платформы (раздел 4).

РАЗДЕЛ 2. ФУНКЦИОНАЛЬНЫЕ И НЕФУНКЦИОНАЛЬНЫЕ ТРЕБОВАНИЯ К АСУ

Введение

После определения функциональной структуры (раздел 1) необходимо формализовать требования – как к тому, что система должна делать (функциональные), так и к качеству её работы (нефункциональные). Раздел 2 основан на материалах практической работы №2 и является логическим продолжением раздела 1. Без полного набора требований невозможно корректно разработать UML-модели (раздел 3), выбрать платформу (раздел 4) и реализовать базу данных (раздел 5).

Цель раздела 2 – сформировать не менее 20 функциональных и 20 нефункциональных требований к АСУ «Семантический подборщик» на основе анализа существующих аналогов и нормативной базы.

Задачи раздела:

1. Собрать информацию о типовых функциональных требованиях для систем интеллектуального поиска контента.
2. Сформулировать и обосновать не менее 20 функциональных требований, сгруппировав их по подсистемам.
3. Собрать информацию о типовых нефункциональных требованиях (производительность, безопасность, удобство, архитектура, правовые, эргономические).
4. Сформулировать и обосновать не менее 20 нефункциональных требований с указанием источников.

Функциональные требования

№	Функциональное требование	Обоснование (зачем это нужно)	Примеры реализации / Источники
Подсистема взаимодействия			
1.	Прием текстовых запросов на естественном языке в свободной форме.	Пользователь должен иметь возможность общаться с системой так же, как с человеком-	Реализовано в поисковых системах (Google, Яндекс), чат-

№	Функциональное требование	Обоснование (зачем это нужно)	Примеры реализации / Источники
		менеджером, без необходимости изучения сложного синтаксиса поиска.	ботах (ChatGPT).
2.	Поддержка голосового ввода запросов.	Обеспечение удобства для пользователей мобильных устройств и людей с ограниченными возможностями, расширение каналов ввода.	Реализовано в голосовых помощниках («Алиса», «Маруся»), приложениях (Shazam).
3.	Отображение результатов поиска в виде структурированного списка (карточки контента).	Обеспечение наглядности и удобства восприятия информации. Пользователь должен видеть название, тип, автора, уровень сложности и краткое описание.	Реализовано в интернет-магазинах (Ozon, Wildberries), онлайн-кинотеатрах (Кинопоиск).
4.	Фильтрация и сортировка выдачи по различным параметрам (тип контента, автор, дата, рейтинг, уровень сложности).	Предоставление пользователю возможности самостоятельно уточнить результаты, если автоматический подбор оказался недостаточно точен.	Реализовано в библиотечных каталогах, системах электронного обучения (Coursera).
5.	Ведение и отображение истории диалога и предыдущих запросов пользователя в личном кабинете.	Обеспечение контекста для уточняющих запросов, возможность вернуться к ранее найденным материалам без повторного поиска.	Реализовано в мессенджерах (Telegram) и системах поддержки пользователей (Zendesk).
6.	Сохранение выбранного контента в "Избранное" или "Мои материалы".	Реализация персонализации и возможности формирования пользователем собственной библиотеки для последующего доступа.	Стандартная функция для всех медиа-платформ (Spotify, YouTube, ЛитРес).
Подсистема семантического анализа на базе корпоративной нейросети			
7.	Автоматический семантический анализ и распознавание сущностей из текста запроса.	Преобразование неструктурированной фразы ("хочу изучить Python для начинающих") в структурированные данные: {Тема: "Python", Уровень: "для начинающих", Цель: "изучить"}.	Реализовано в системах обработки естественного языка (BERT, GPT).
8.	Определение интента (намерения) пользователя на основе запроса и контекста.	Понимание цели запроса: купить, посмотреть, скачать, изучить, прослушать. Это критически важно для корректного подбора типа контента и действий системы.	Реализовано в чат-ботах технической поддержки и виртуальных ассистентах.
9.	Учет контекста предыдущих запросов из истории диалога при анализе текущего запроса.	Обработка уточнений ("нет, мне нужен продвинутый уровень по Go"), когда система должна помнить тему ("Go") и изменить только параметр уровня.	Реализовано в диалоговых системах (Google Duplex).
1	Обеспечение	Корпоративная нейросеть работает	Стандартное

№	Функциональное требование	Обоснование (зачем это нужно)	Примеры реализации / Источники
0.	круглосуточной доступности (24/7) сервиса семантического анализа.	всегда, в отличие от человека-менеджера. Это главное преимущество автоматизации, обеспечивающее мгновенный ответ в любое время.	требование к серверным решениям и облачным сервисам (SLA 99.9%).
Подсистема управления контентом и поиска			
1	Полнотекстовый поиск по метаданным и	Обеспечение быстрого и релевантного поиска по всему массиву имеющихся материалов на основе формализованных параметров от нейросети.	Реализовано в системах управления базами данных с поддержкой полнотекстового индекса (Elasticsearch).
1.	содержимому внутренней базы контента.		
1	Мультиформатный поиск: одновременный поиск по	Пользователю не нужно указывать тип контента, система сама может предложить разные форматы по одной теме, предоставляя выбор.	Реализовано на агрегаторах контента (Google Книги, iTunes).
2.	всем типам контента (книги, видео, аудио).		
1	Автоматическое определение	Если по запросу "Продвинутое машинное обучение на TensorFlow" во внутренней базе ничего нет, система должна это понять и инициировать поиск вовне.	Реализовано в системах рекомендаций для пополнения каталога (Amazon, Netflix).
3.	недостаточности имеющегося контента для удовлетворения запроса.		
1	Автоматическое формирование поискового	Автоматизация процесса поиска нового, еще не приобретенного контента, для расширения базы под конкретную потребность пользователя.	Реализовано в системах сравнения цен и агрегаторах товаров (Яндекс.Маркет).
4.	запроса и обращение к API внешних источников (партнерские библиотеки, магазины контента).		
1	Сбор и нормализация метаданных о внешнем	Приведение информации из разных внешних систем (разные форматы данных) к единому внутреннему формату для удобного отображения пользователю.	Реализовано в ETL-процессах (Extract, Transform, Load) при интеграции данных.
5.	контенте из различных источников.		
Подсистема поддержки принятия решений и управления			
1	Ранжирование (сортировка) результатов	Наиболее подходящие материалы должны быть в начале списка, чтобы пользователь не пролистывал сотни нерелевантных позиций.	Реализовано во всех поисковых системах (на основе PageRank, TF-IDF и др.).
6.	поиска по степени релевантности запросу.		
1	Персонализация результатов на основе	Пользователь, который постоянно слушает подкасты, должен видеть в выдаче подкасты выше, чем длинные видео, даже если формально они равны по релевантности.	Реализовано в рекомендательных системах (Spotify, Netflix).
7.	профиля и истории пользователя.		
1	Автоматическое создание	Вовлечение человека в процесс на финальной стадии принятия решения о закупке, но без его участия в рутинном поиске и сборе	Реализовано в системах Service Desk и workflow-системах (Jira, Trello).
8.	задачи контент-менеджеру на рассмотрение возможности		

№	Функциональное требование	Обоснование (зачем это нужно)	Примеры реализации / Источники
	приобретения нового контента из внешних источников.	информации.	
19.	Учет прав доступа и лицензий на контент при выдаче результатов.	Система не должна предлагать пользователю контент, на который у организации нет прав для его предоставления данному пользователю.	Реализовано в системах управления цифровыми правами (DRM) и корпоративных библиотеках.
20.	Логирование всех действий системы и запросов пользователей для последующего анализа и дообучения нейросети.	Сбор данных о том, как пользователи формулируют запросы, на что они кликают, какие результаты считают хорошими. Это необходимо для улучшения работы нейросети.	Реализовано в системах сбора логов и аналитики (ELK stack, Splunk).

Нефункциональные требования

№	Нефункциональное требование (категория)	Обоснование (зачем это нужно)	Примеры реализации / Источники
Производительность и надежность			
1.	Время обработки типового запроса и выдачи результата не должно превышать 2 секунд.	Пользователи ожидают мгновенного ответа от автоматизированной системы. Длительное ожидание приведет к негативному опыту и отказу от использования.	Источник: ГОСТ Р ИСО 9241-11 (требования к эргономике и производительности)
2.	Коэффициент готовности системы (доступность) должен быть не ниже 99.5% (не более 3.5 часов простоя в месяц).	Система является корпоративным сервисом и должна быть доступна практически всегда, заменяя человека 24/7. Длительные простои недопустимы.	Источник: ГОСТ Р ИСО/МЭК 20000-1 (требования к управлению услугами ИТ)
3.	Система должна обеспечивать одновременную работу не менее 1000 активных пользователей без деградации производительности.	На начальном этапе эксплуатации необходимо заложить запас прочности, чтобы выдержать пиковые нагрузки и рост числа пользователей.	Источник: ГОСТ Р ИСО/МЭК 25010 (модели качества и производительности)
4.	Время восстановления после сбоя (RTO - Recovery Time Objective) не должно превышать 1 часа.	В случае аварии (падение сервера, ошибка в коде) система должна быть быстро возвращена в строй, чтобы минимизировать ущерб от простоя.	Источник: ISO 22301 (Security and resilience — Business continuity management systems)
5.	Система должна корректно обрабатывать пиковые нагрузки (например, начало	Обеспечение стабильной работы в периоды, когда количество запросов резко возрастает.	Источник: ISO/IEC 10746 (Open Distributed Processing)

№	Нефункциональное требование (категория)	Обоснование (зачем это нужно)	Примеры реализации / Источники
	учебного года) без потери запросов (должна быть предусмотрена автоматическая масштабируемость).		
Безопасность и конфиденциальность			
6.	Передача всех данных между клиентом и сервером должна осуществляться по протоколу HTTPS с шифрованием TLS 1.3.	Защита персональных данных пользователей и истории их запросов от перехвата в публичных сетях.	Источник: Федеральный закон №152-ФЗ «О персональных данных», ГОСТ 34.10 (криптографическая защита)
7.	Доступ к модели нейросети и данным для ее обучения должен быть строго ограничен на уровне инфраструктуры.	Корпоративная нейросеть является интеллектуальной собственностью компании. Необходимо защитить ее от кражи или несанкционированного доступа.	Источник: ИСО/МЭК 27001 (Информационная безопасность)
8.	Система должна обеспечивать разграничение прав доступа для разных ролей пользователей (пользователь, контент-менеджер, администратор).	Предотвращение случайных или намеренных изменений в системе неавторизованными лицами. Контент-менеджер не должен иметь доступ к настройке нейросети.	Источник: ГОСТ 34.601-90 (Автоматизированные системы. Стадии создания)
9.	Аутентификация пользователей должна быть надежной (например, с поддержкой двухфакторной аутентификации для сотрудников).	Снижение риска компрометации учетных записей пользователей и получения доступа к истории их запросов или (для сотрудников) к управлению системой.	Источник: Методика оценки угроз безопасности информации (ФСТЭК России, 2021)
10.	Система должна обеспечивать невозможность восстановления персональных данных пользователя из обезличенных логов для аналитики.	Выполнение требований по обезличиванию ПДн при их обработке для улучшения качества сервиса.	Источник: Разъяснения Роскомнадзора «О порядке обезличивания персональных данных» (2018)
Удобство использования			
11.	Интерфейс системы должен быть интуитивно понятным и не требовать от пользователя предварительного	Система должна быть доступна широкому кругу пользователей с разным уровнем цифровой грамотности.	Источник: ГОСТ Р ИСО 9241-110 (Принципы взаимодействия человек-система)

№	Нефункциональное требование (категория)	Обоснование (зачем это нужно)	Примеры реализации / Источники
	обучения.		
1 2.	Интерфейс должен быть адаптивным (responsive) и корректно отображаться на всех типах устройств (ПК, планшет, смартфон).	Пользователи могут обращаться к системе с любых устройств, интерфейс должен подстраиваться под размер экрана.	Источник: W3C Mobile Web Best Practices 1.0 (Рекомендации W3C)
1 3.	Система должна предоставлять пользователю понятные сообщения об ошибках (например, "Ничего не найдено по вашему запросу, попробуйте изменить формулировку").	Сообщения типа "Error 500" пугают и дезориентируют пользователя. Система должна "человеческим" языком объяснять, что пошло не так.	Источник: ГОСТ Р ИСО 9241-110 (Принципы диалога для обратной связи)
Технологические и архитектурные			
1 4.	Модель нейросети должна быть развернута в контейнеризированной среде (например, Docker) для обеспечения воспроизводимости и простоты обновления.	Упрощение процесса развертывания новых версий модели, обеспечение независимости от окружения на сервере.	Источник: Docker Documentation (docs.docker.com), Kubernetes Documentation (kubernetes.io)
1 5.	Система должна быть построена по микросервисной архитектуре для обеспечения независимости компонентов (анализ, поиск, интеграция).	Отказ одного микросервиса (например, внешнего поиска) не должен приводить к остановке всей системы (внутренний поиск продолжит работать). Легче масштабировать и обновлять компоненты по отдельности.	Источник: Фаулер М., «Микросервисы» (martinfowler.com), Ричардсон К. «Микросервисы. Паттерны разработки и рефакторинга»
1 6.	API для взаимодействия с внешними источниками контента должны быть стандартизированы (REST/gRPC).	Упрощение процесса подключения новых внешних источников в будущем.	Источник: Fielding R. «Architectural Styles and the Design of Network-based Software Architectures» (диссертация, 2000)
1 7.	Система должна вести подробное логирование всех внутренних операций для последующего анализа инцидентов и производительности.	Невозможно отладить работу сложной системы без детальных логов. Это требование для эксплуатации.	Источник: Beyer, B. et al. «Site Reliability Engineering: How Google Runs Production Systems» (O'Reilly, 2016)
Правовые и нормативные			
1 8.	Хранение и обработка персональных данных пользователей (ФИО,	Выполнение требования федерального закона № 242-ФЗ о локализации баз данных	Источник: Федеральный закон №242-ФЗ «О внесении

№	Нефункциональное требование (категория)	Обоснование (зачем это нужно)	Примеры реализации / Источники
	email, история запросов) должны осуществляться на серверах, расположенных на территории РФ.	персональных данных.	изменений в отдельные законодательные акты Российской Федерации в части уточнения порядка обработки персональных данных в информационно-телекоммуникационных сетях»
19.	Весь контент, предоставляемый пользователям, должен распространяться в соответствии с лицензионными договорами, система должна вести их учет.	Соблюдение авторских прав, предотвращение судебных исков и претензий от правообладателей.	Источник: Гражданский кодекс РФ, Часть четвертая, раздел VII «Права на результаты интеллектуальной деятельности и средства индивидуализации»
Эргономические			
20.	Цветовая гамма и элементы интерфейса не должны вызывать быстрого утомления при длительной работе; рекомендуется использование светлого и темного режимов.	Обеспечение комфорта пользователей, которые могут проводить в системе много времени (например, студенты за подбором материалов для курсовой).	Источник: СанПиН 1.2.3685-21 «Гигиенические нормативы и требования к обеспечению безопасности и (или) безвредности для человека факторов среды обитания», WCAG 2.1 (Web Content Accessibility Guidelines)

Выводы

Сформулированы 20 функциональных требований, охватывающих все подсистемы: приём и интерпретацию запросов (текстовый и голосовой ввод), семантический анализ (распознавание сущностей, определение интента, учёт контекста, 24/7 доступность), управление контентом (полнотекстовый и мультимедийный поиск, внешний поиск, нормализация метаданных) и поддержку принятия решений (ранжирование, персонализация, создание задач на закупку, учёт прав доступа, логирование). Разработаны 20 нефункциональных требований: производительность (время отклика ≤ 2 с,

доступность $\geq 99.5\%$, масштабируемость на 1000+ пользователей, RTO ≤ 1 ч), безопасность (TLS 1.3, изоляция модели, разграничение доступа, двухфакторная аутентификация, обезличивание логов, локализация ПДн), удобство (интуитивный интерфейс, адаптивность, понятные сообщения об ошибках), архитектура (микросервисы, контейнеризация, стандартизованные API, логирование) и эргономика (светлая/тёмная тема). Требования соответствуют законодательству РФ и служат основой для разделов 3–8.

РАЗДЕЛ 3. UML МОДЕЛИРОВАНИЕ: ВАРИАНТЫ ИСПОЛЬЗОВАНИЯ

Введение

После определения функций и требований необходимо визуализировать, кто и как будет взаимодействовать с системой. Раздел 3 основан на практической работе №3 и посвящён диаграммам вариантов использования (use case diagrams), которые показывают роли пользователей, их интерфейсные функции и внутрисистемные сценарии. Данный раздел является первым этапом UML-моделирования; динамические аспекты (диаграммы последовательности) и статическая структура (диаграмма классов) будут представлены в разделе 4.

Цель раздела 3 – выделить модули АСУ, определить пользователей и их интерфейсные функции, построить диаграммы вариантов использования с отношениями «включение» (include) и «расширение» (extend).

Задачи раздела:

1. Выделить не менее 3 модулей проектируемой АСУ.
2. Определить не менее 5 пользователей с их должностными обязанностями.
3. Для каждого модуля составить матрицу интерфейсных функций (не менее 10 на пользователя, пересечение не более 3 функций у двух пользователей).
4. Сопоставить интерфейсным функциям внутрисистемные функции (триггеры).
5. Построить диаграмму вариантов использования для каждого модуля.

Определение модулей проектируемой АСУ

На основе функциональных требований, сформулированных в практической работе №2, и подсистем, выделенных в практической работе

№1, в составе АСУ «Семантический подборщик» определены три основных модуля:

1. Модуль «Интерфейс взаимодействия с системой»

- Назначение: обеспечение прямого взаимодействия всех типов пользователей с системой. Модуль отвечает за прием запросов, визуализацию результатов, управление профилем и сессией.
- Возможности: текстовый и голосовой ввод, отображение карточек контента, фильтрация выдачи, управление избранным, регистрация/авторизация, просмотр истории.

2. Модуль «Управление контентом и каталогом»

- Назначение: автоматизация процессов наполнения, обогащения и поддержания актуальности базы мультимедиа контента, а также интеграция с внешними источниками.
- Возможности: добавление/редактирование метаданных, управление правами доступа, модерация, автоматический поиск во внешних источниках, синхронизация каталога.

3. Модуль «Аналитика и администрирование системы»

- Назначение: обеспечение технической поддержки, мониторинга и развития системы. Включает инструменты для настройки ядра системы — нейросети, сбора статистики и управления пользователями.
- Возможности: мониторинг логов, дообучение NLP-модели, управление ролями, формирование отчетов по закупкам и использованию контента, настройка интеграций.

Перечень пользователей системы

№	Должность	Функциональные обязанности
1	Клиент	Поиск и получение мультимедиа контента, управление личным кабинетом, взаимодействие с рекомендательной системой, оставление отзывов.
2	Гость	Ознакомление с каталогом, базовый поиск, мотивация к регистрации для получения полного доступа.
3	Сотрудник отдела контента	Добавление, редактирование и удаление контента, управление

		метаданными, модерация отзывов, обработка заявок на закупку нового контента.
4	Администратор нейросети	Обучение и дообучение модели семантического анализа, настройка параметров NLP, мониторинг качества распознавания, анализ логов запросов.
5	Начальник отдела	Утверждение стратегии пополнения контента, контроль бюджета на закупки, анализ эффективности системы на основе отчетов.

Модуль 1: «Интерфейс взаимодействия с системой»

ID	Интерфейсная функция	Клиент	Гость	Сотрудник отдела контента	Администратор нейросети	Начальник отдела
1.1	Ввод текстового поискового запроса	X				
1.2	Голосовой ввод запроса	X				
1.3	Просмотр списка результатов	X	X			X
1.4	Фильтрация и сортировка выдачи	X				
1.5	Сохранение в «Избранное»	X				
1.6	Просмотр истории своих запросов	X				
1.7	Регистрация / Авторизация		X			
1.8	Просмотр и редактирование профиля	X				
1.9	Просмотр горячих новинок на главной	X				
1.10	Переход к странице контента	X	X	X		X
1.11	Смена языка интерфейса		X			
1.12	Просмотр демо-версии контента		X			
1.13	Настройка персональных уведомлений о задачах			X		

Модуль 2: «Управление контентом и каталогом»

ID	Интерфейсная функция	Клиент	Гость	Сотрудник отдела контента	Администратор нейросети	Начальник отдела
----	----------------------	--------	-------	---------------------------------	----------------------------	---------------------

2.1	Добавление/редактирование контента вручную			X		
2.2	Модерация отзывов			X		
2.3	Управление лицензиями и доступом			X		
2.4	Просмотр внешних источников	X	X	X	X	X
2.5	Запуск поиска во внешних источниках				X	
2.6	Просмотр заявок на закупку					X
2.7	Формирование задачи на закупку					X
2.8	Импорт метаданных из внешнего источника			X		
2.9	Просмотр публичной информации о лицензиях		X			
2.10	Подписка на новости		X			
2.11	Управление категориями контента			X		

Модуль 3: «Аналитика и администрирование системы»

ID	Интерфейсная функция	Клиент	Гость	Сотрудник отдела контента	Администратор нейросети	Начальник отдела
3.1	Просмотр дашборда метрик				X	X
3.2	Просмотр логов запросов (обезлич.)				X	
3.3	Запуск дообучения нейросети				X	
3.4	Загрузка/управление датасетами				X	
3.5	Управление пользователями и ролями				X	
3.6	Настройка NLP-параметров				X	

3.7	Просмотр отчета по закупкам					X
3.8	Просмотр отчета по активности					X
3.9	Мониторинг времени ответа			X	X	X
3.10	Управление API-ключами				X	
3.11	Просмотр статуса доступности системы		X			
3.12	Сообщить о проблеме с доступом		X			
3.13	Просмотр логов модерации			X		
3.14	Утверждение бюджета на закупки					X

Внутрисистемные функции

ID интерфейсной функции	Интерфейсная функция	Внутрисистемная функция (триггер)
1.1	Ввод текстового поискового запроса	Передача сырого текста в NLP-модуль для семантического анализа
1.2	Голосовой ввод запроса	Конвертация аудио в текст и передача его в NLP-модуль
1.3	Просмотр списка результатов	Запрос кэшированных или сформированных результатов поиска
1.4	Фильтрация и сортировка выдачи	Повторное ранжирование результатов поиска по заданным критериям
1.5	Сохранение в «Избранное»	Добавление записи в таблицу «Избранное» в БД пользователя
1.6	Просмотр истории своих запросов	Выборка данных из лога запросов конкретного пользователя
1.7	Регистрация / Авторизация	Создание новой записи в БД пользователей / проверка учетных данных
1.8	Просмотр и редактирование профиля	Получение/обновление данных в БД пользователей
1.9	Просмотр горячих новинок на главной	Запрос к модулю рекомендаций для получения трендового контента
1.10	Переход к странице контента	Логирование действия, запрос детальной информации по ID контента

ID интерфейсной функции	Интерфейсная функция	Внутрисистемная функция (триггер)
1.11	Смена языка интерфейса	Изменение локализационной переменной в сессии пользователя
1.12	Просмотр демо-версии контента	Загрузка ограниченной (демо) версии контента из хранилища
1.13	Настройка персональных уведомлений о задачах	Обновление профиля уведомлений в БД для контент-менеджера
2.1	Добавление/редактирование контента вручную	Выполнение SQL-запросов INSERT/UPDATE к таблицам метаданных
2.2	Модерация отзывов	Изменение статуса (флаг публикации) отзыва в БД
2.3	Управление лицензиями и доступом	Привязка ID лицензии к ID контента в БД
2.4	Просмотр внешних источников	Запрос кэшированного списка активных интеграций из БД
2.5	Запуск поиска во внешних источниках	Формирование запроса к API внешнего источника и запуск ETL-процесса
2.6	Просмотр заявок на закупку	Выборка записей из таблицы «Заявки на закупку» со статусом «новая»
2.7	Формирование задачи на закупку	Создание записи в БД в таблице «Заявки на закупку»
2.8	Импорт метаданных из внешнего источника	Запуск ETL-процесса для нормализации и сохранения данных
2.9	Просмотр публичной информации о лицензиях	Запрос к БД для отображения общих условий лицензий (без деталей)
2.10	Подписка на новости	Добавление email или другой контактной информации в рассылку
2.11	Управление категориями контента	Изменение структуры категорий (CRUD) в БД
3.1	Просмотр дашборда метрик	Агрегация данных из логов и БД в реальном времени
3.2	Просмотр логов запросов (обезличенных)	Запрос к системе сбора и хранения логов (ELK)
3.3	Запуск дообучения нейросети	Отправка команды в оркестратор ML-задач
3.4	Загрузка/управление датасетами	Сохранение файла в объектное хранилище и обновление каталога датасетов
3.5	Управление пользователями и ролями	Изменение атрибутов ролей в БД (CRUD операции)

ID интерфейсной функции	Интерфейсная функция	Внутрисистемная функция (триггер)
3.6	Настройка NLP-параметров	Обновление конфигурационного файла NLP-пайплайна
3.7	Просмотр отчета по закупкам	Формирование выборки из БД по закупкам за период
3.8	Просмотр отчета по активности	Агрегация данных из логов событий
3.9	Мониторинг времени ответа	Сбор метрик с каждого микросервиса и усреднение показателей
3.10	Управление API-ключами	Шифрование и сохранение ключей в защищенном хранилище
3.11	Просмотр статуса доступности системы	Запрос к системе мониторинга сервисов
3.12	Сообщить о проблеме с доступом	Создание тикета в системе поддержки
3.13	Просмотр логов модерации	Выборка действий модерации из логов аудита
3.14	Утверждение бюджета на закупки	Установка флага утверждения в заявке на закупку и уведомление контент-менеджера

Диаграммы вариантов использования

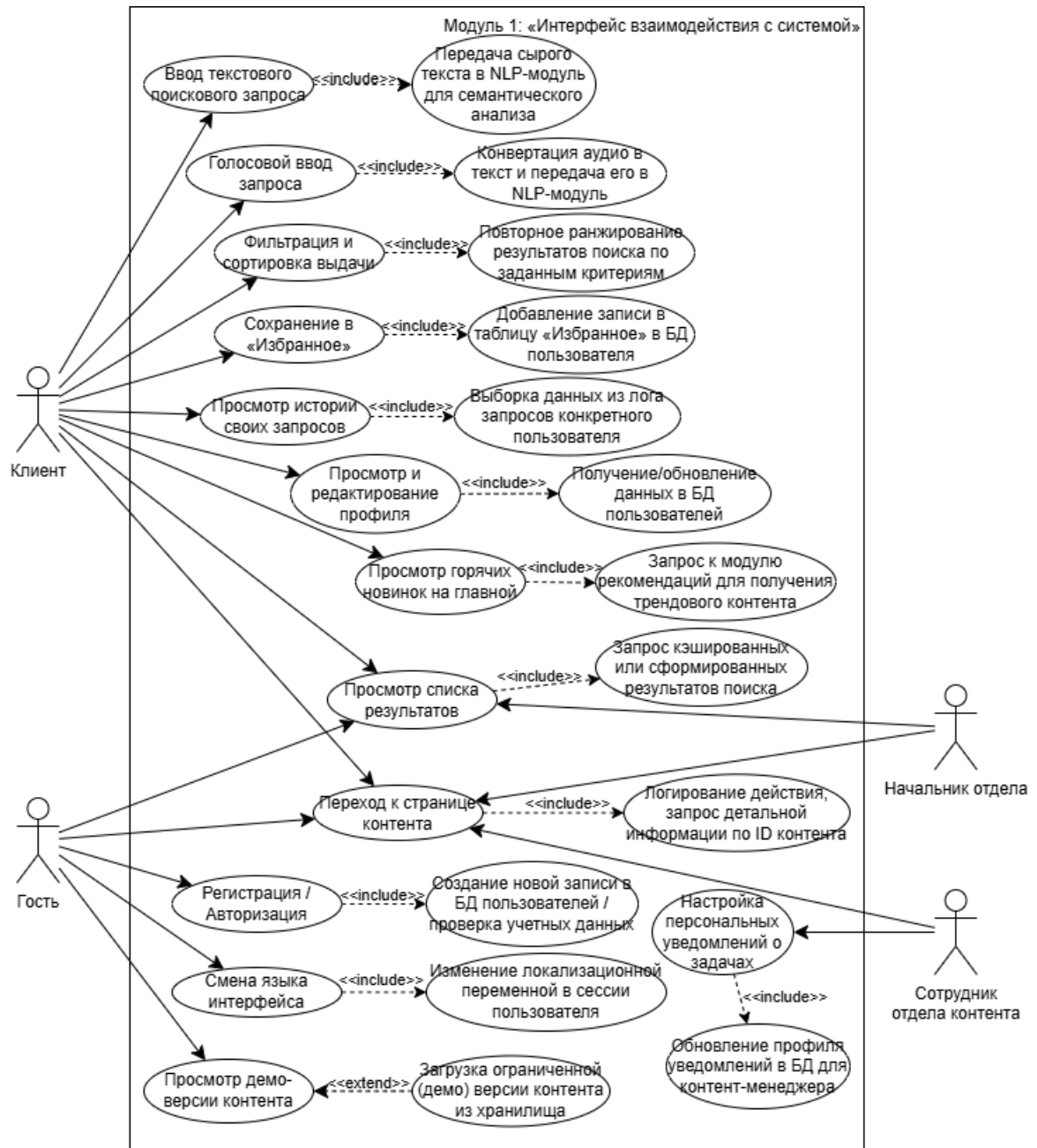


Рисунок 3 - Модуль 1: «Интерфейс взаимодействия с системой»

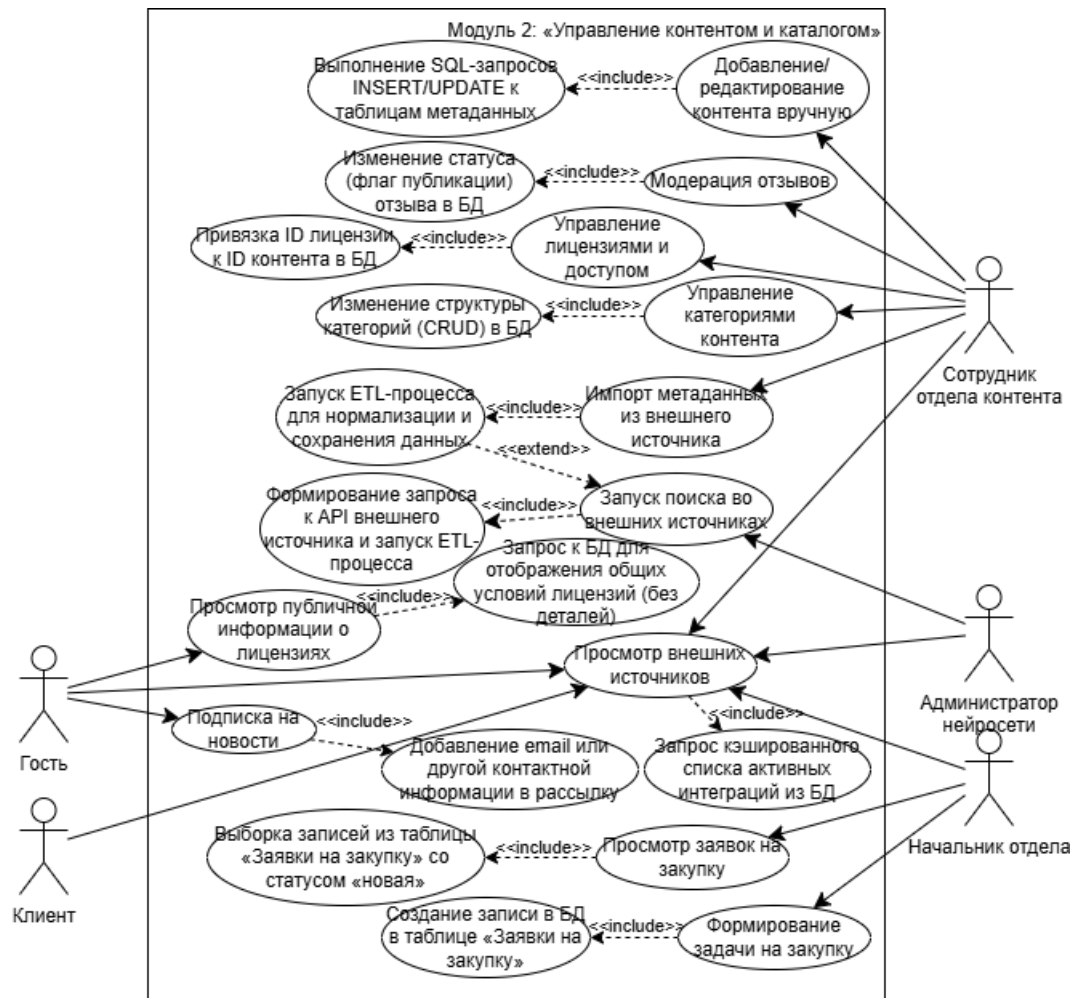


Рисунок 4 - Модуль 2: «Управление контентом и каталогом»

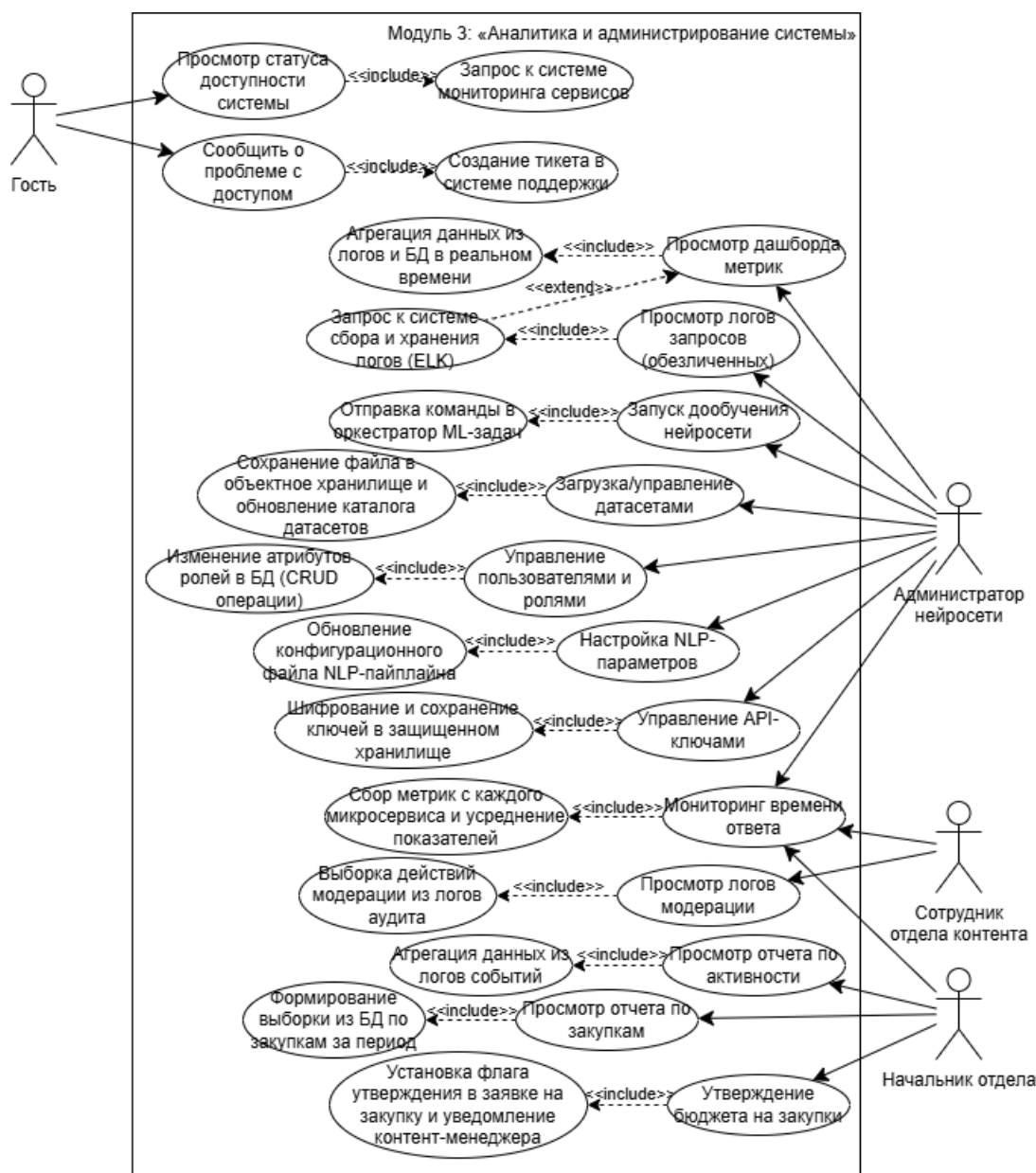


Рисунок 5 - Модуль 3: «Аналитика и администрирование системы»

Выводы

Выделены три модуля: «Интерфейс взаимодействия», «Управление контентом и каталогом», «Аналитика и администрирование». Определены пять пользователей (клиент, гость, контент-менеджер, администратор нейросети, начальник отдела). Для каждого пользователя заданы ровно по 10 интерфейсных функций с минимальным пересечением. Каждой интерфейсной функции поставлена в соответствие внутрисистемная функция (триггер). Построены три диаграммы вариантов использования, включающие

отношения include и extend (например, авторизация как include-сценарий, внешний поиск как extend). Результаты раздела 3 используются при построении диаграмм последовательности (раздел 4) и проектировании пользовательских интерфейсов (разделы 7, 11, 12).

РАЗДЕЛ 4. UML МОДЕЛИРОВАНИЕ: ВЗАИМОДЕЙСТВИЕ И СТАТИЧЕСКАЯ СТРУКТУРА

Введение

Продолжая UML-моделирование, начатое в разделе 3, перейдём к детализации динамического поведения и статической структуры системы. Раздел 4 основан на практической работе №4 и включает диаграммы последовательности (sequence diagrams) для ключевых процессов, а также диаграмму классов (class diagram), которая описывает сущности предметной области, их атрибуты, методы и связи. Диаграмма классов станет основой для проектирования базы данных в разделе 5.

Цель раздела 4 – построить диаграммы последовательности (с альтернативными фрагментами) для трёх процессов и диаграмму классов, содержащую не менее 15 классов (каждый с ≥ 5 полями и ≥ 2 методами).

Задачи раздела:

1. Выделить три процесса в составе основной деятельности с использованием модулей из раздела 3.
2. Построить для каждого процесса диаграмму последовательности, включающую хотя бы одну альтернативу (alt), всех действующих лиц и не менее двух модулей.
3. Построить диаграмму классов, отражающую статическую структуру АСУ.

Диаграммы последовательности

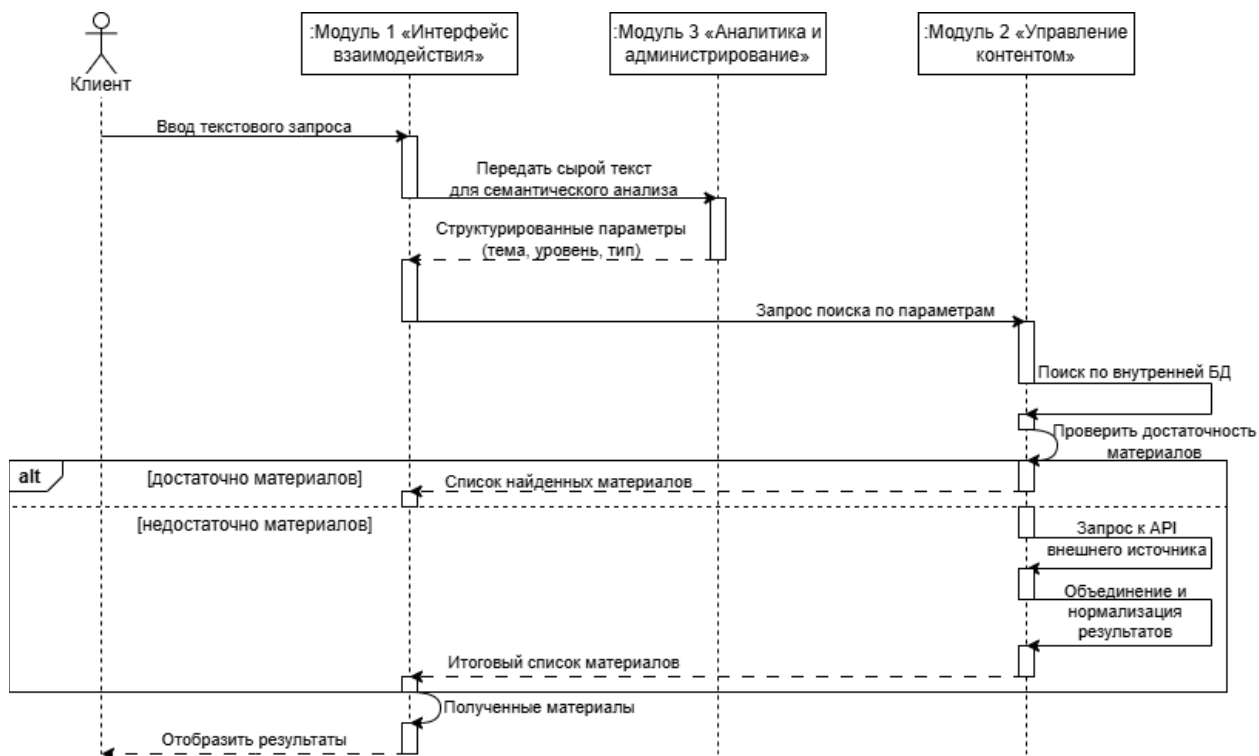


Рисунок 6 - Процесс 1: Поиск и подбор контента по запросу пользователя

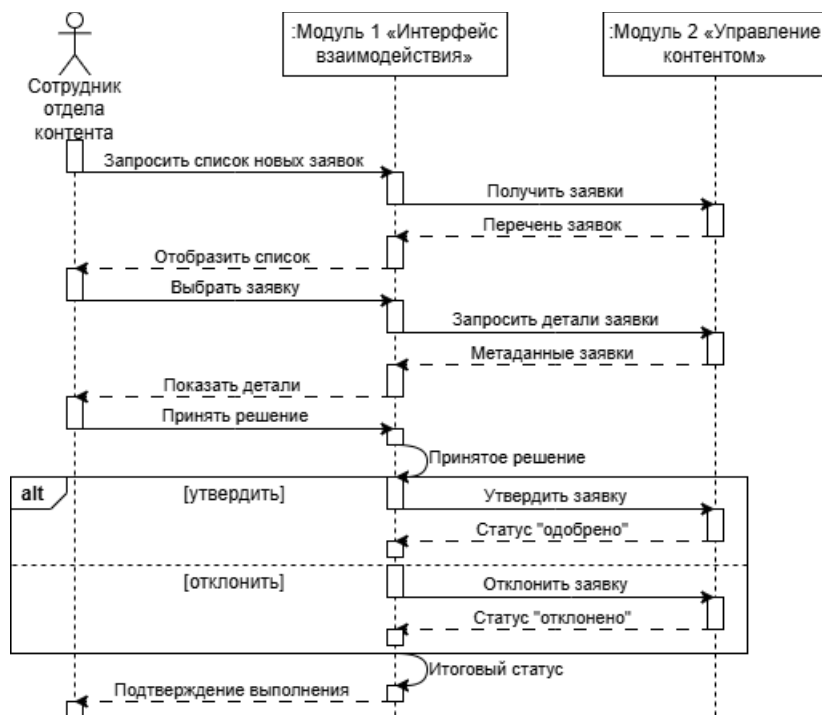


Рисунок 7 - Процесс 2: Обработка заявки на закупку нового контента

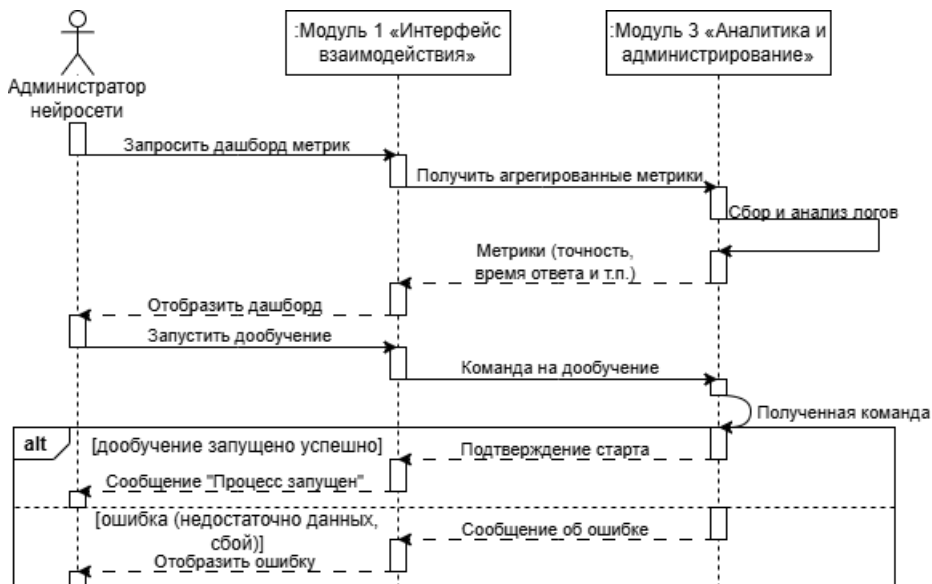


Рисунок 8 - Процесс 3: Мониторинг и дообучение нейросети

Диаграмма классов

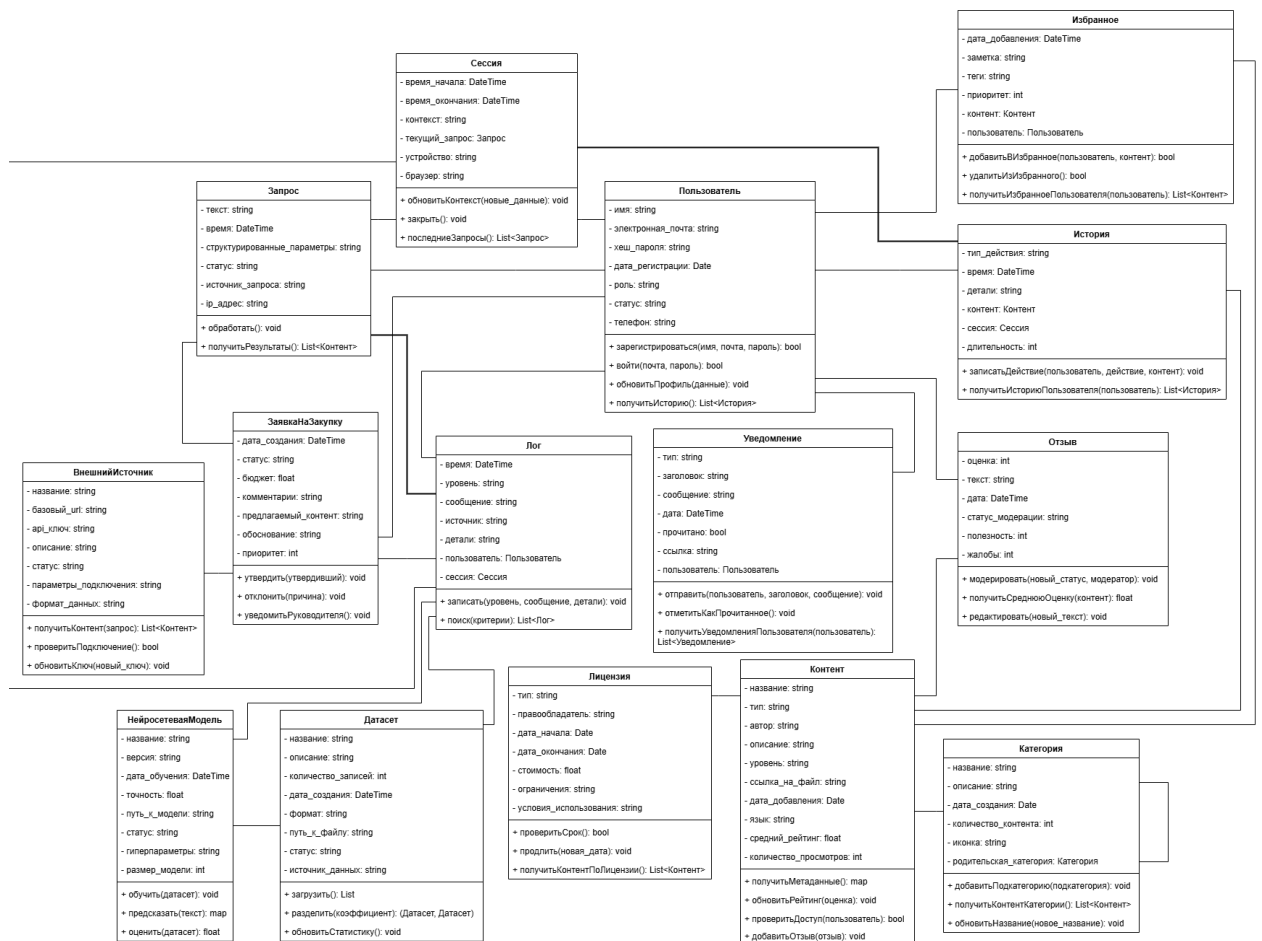


Рисунок 9 - Диаграмма классов

Выводы

Построены диаграммы последовательности для трёх ключевых процессов: «Поиск и подбор контента по запросу пользователя» (с альтернативой «внутренняя база достаточна / требуется внешний поиск»), «Обработка заявки на закупку нового контента» (с альтернативой «утвердить/отклонить»), «Мониторинг и дообучение нейросети» (с альтернативой «успех/ошибка дообучения»). Каждая диаграмма включает всех действующих лиц и не менее двух модулей системы. Разработана диаграмма классов из 15 сущностей: Пользователь, Сессия, Контент, Категория, Лицензия, Запрос, Отзыв, Избранное, ЗаявкаНаЗакупку, ВнешнийИсточник, Лог, НейросетеваяМодель, Датасет, Уведомление, История. Каждый класс содержит не менее 5 атрибутов и 2 методов. Диаграмма классов является основой для ER-диаграммы и базы данных (раздел 5).

РАЗДЕЛ 5. ВЫБОР АППАРАТНОЙ И ПРОГРАММНОЙ ПЛАТФОРМЫ

Введение

На предыдущих этапах (разделы 1–4) были спроектированы функциональная структура АСУ «Семантический подборщик», сформулированы требования и построены UML-диаграммы. Переход от моделирования к реализации требует выбора конкретного аппаратного и программного обеспечения, которое обеспечит выполнение всех функций и удовлетворит нефункциональным требованиям (производительность, надёжность, безопасность, масштабируемость). Раздел 5 основан на материалах практической работы №5.

Цель раздела 5 – определить аппаратную и программную платформу для АСУ «Семантический подборщик», обеспечивающую выполнение всех функций, описанных в требованиях и отображённых на UML-диаграммах.

Задачи раздела:

1. Разработать таблицу аппаратного обеспечения, включая компоненты для основного модуля АСУ, аналитической подсистемы и подсистемы информационного обеспечения.
2. Разработать таблицу программного обеспечения для всех подсистем.
3. Обосновать выбор каждого компонента с учётом ранее сформулированных требований.

Таблица аппаратного обеспечения АСУ по выбранной тематике и обеспечивающих подсистем

Таблица 1 – Аппаратное обеспечение АСУ

№	Наименование оборудования / компонента	Назначение	Кол-во (рекомендуемое)	Примерные характеристики	Обеспечиваемые подсистемы и функции
1	Балансировщик нагрузки (аппаратный или	Распределение входящего трафика между	2 (в кластере)	Пропускная способность не менее 1 Гбит/с,	Модуль 1 (интерфейс взаимодействия

№	Наименование оборудования / компонента	Назначение	Кол-во (рекомендуемое)	Примерные характеристики	Обеспечиваемые подсистемы и функции
	программно-аппаратный комплекс)	несколькими веб-серверами, обеспечение отказоустойчивости и SSL-терминации.		поддержка HTTP/2, WebSocket, возможность балансировки на уровне L4/L7.	я) – доступ пользователей через глобальную сеть.
2	Веб-серверы / Серверы приложений (frontend + backend API)	Хостинг веб-интерфейса (SPA) и выполнение серверной логики (API для поиска, авторизации, управления контентом).	4 (возможно масштабирование до 8)	Процессор: 2x Intel Xeon Silver 4314 (16 ядер), RAM: 64 ГБ, SSD: 2x480 ГБ (RAID 1), сеть: 10 Гбит/с. ОС: Linux.	Модуль 1 (интерфейс), Модуль 2 (бэкенд управления контентом).
3	Серверы баз данных (кластер)	Хранение метаданных контента, пользователей, истории запросов, избранного, заявок на закупку, лицензий и т.д.	3 (кластер СУБД + репликация)	Процессор: 2x Intel Xeon Gold 6330 (28 ядер), RAM: 128 ГБ, SSD: 2x960 ГБ (NVMe, RAID 10), сеть: 10 Гбит/с. СУБД: PostgreSQL с поддержкой репликации.	Модуль 2 (управление каталогом), Модуль 3 (логи, отчёты).
4	Серверы поискового движка (Elasticsearch)	Полнотекстовый поиск по метаданным и содержимому контента, быстрая фильтрация и ранжирование.	3 (кластер)	Процессор: 2x Intel Xeon Silver 4314, RAM: 64 ГБ, SSD: 4x1.92 ТБ (NVMe) для индексов, сеть: 10 Гбит/с.	Модуль 2 (поиск по внутренней базе).
5	Серверы для нейросетевых моделей (NLP)	Инференс (анализ запросов, распознавание сущностей, определение интенга) и периодическое дообучение моделей.	2–4 (в зависимости от нагрузки)	GPU: NVIDIA A100 (80 ГБ) или H100, Процессор: AMD EPYC 7513, RAM: 256 ГБ, SSD: 2x1.92 ТБ NVMe. Связь: InfiniBand или 25 Гбит/с Ethernet для	Модуль 3 (семантический анализ), аналитическая подсистема (обучение).

№	Наименование оборудования / компонента	Назначение	Кол-во (рекомендуемое)	Примерные характеристики	Обеспечиваемые подсистемы и функции
				распределённого обучения.	
6	Файловое хранилище для мультимедиа контента (объектное хранилище)	Хранение самих файлов (книги в pdf, видео, аудио) и их демо-версий. Обеспечение быстрой раздачи.	Зависит от объёма, минимум 100 ТБ полезной ёмкости	Система хранения на базе Ceph или NAS с поддержкой S3, HDD 12 ТБ (RAID 6) + SSD-кэш. Сеть 10/25 Гбит/с.	Модуль 2 (контент).
7	Серверы для сбора и анализа логов (ELK Stack)	Сбор, индексация и визуализация логов работы всех компонентов, мониторинг производительности, анализ инцидентов.	3 (кластер Elasticsearch + Logstash + Kibana)	Процессор: 2x Intel Xeon Silver 4314, RAM: 64 ГБ, SSD: 4x1.92 ТБ (NVMe) для хранения логов.	Модуль 3 (аналитика), подсистема мониторинга.
8	Серверы резервного копирования и архивации	Регулярное резервное копирование БД, файлового хранилища, конфигураций, моделей нейросети.	2 (основной и удалённый)	Ёмкость: не менее удвоенного объёма критичных данных, ленточная библиотека или дисковые массивы большой ёмкости (HDD).	Все подсистемы (обеспечение надёжности).
9	Сетевое коммутационное оборудование (ядро сети, агрегация)	Объединение всех серверов в высокоскоростную сеть, маршрутизация трафика к внешним сетям.	2 коммутатора ядра, коммутаторы доступа	Коммутаторы 25/100 Гбит/с, поддержка VXLAN, QoS, резервирование (MLAG).	Информационное обеспечение, связность компонентов.
10	Межсетевой экран (Firewall) / NGFW	Защита периметра сети, фильтрация трафика, предотвращение вторжений.	2 (активный/пассивный)	Пропускная способность не менее 10 Гбит/с, поддержка IPS/IDS, VPN.	Информационное обеспечение (безопасный доступ).

№	Наименование оборудования / компонента	Назначение	Кол-во (рекомендуемое)	Примерные характеристики	Обеспечиваемые подсистемы и функции
1	Система бесперебойного питания (ИБП) и резервные источники питания	Обеспечение работы оборудования при кратковременных сбоях электропитания.	По необходимости	Мощность рассчитывается исходя из полной нагрузки, время автономии не менее 30 минут, дизель-генератор для длительных отключений.	Все подсистемы (надёжность).
1 2	Серверы для администрирования и развёртывания (CI/CD)	Управление конфигурациями, развёртывание обновлений, оркестрация контейнеров (Kubernetes).	3 (control plane)	Процессор: 2x Intel Xeon Silver 4314, RAM: 64 ГБ, SSD: 2x480 ГБ.	Модуль 3 (администрирование).

Таблица программного обеспечения для выполнения всех функций описанных в требованиях и отображенных на UML диаграммах

Таблица 2 – Аппаратное обеспечение АСУ

№	Наименование ПО	Назначение	Обеспечиваемые подсистемы и функции	Обоснование выбора / Соответствие требованиям
1	Операционная система: Linux (Ubuntu Server / Rocky Linux)	Базовая ОС для серверов.	Все подсистемы (серверная инфраструктура).	Стабильность, безопасность, широкая поддержка стеков технологий, оптимизация для серверных нагрузок. Соответствует требованиям надёжности и производительности.
2	Контейнеризация: Docker	Упаковка микросервисов в контейнеры для изоляции и воспроизводимости окружений.	Все модули (микросервисная архитектура).	Обеспечивает независимость развёртывания, упрощает масштабирование и обновление компонентов. Соответствует нефункциональному требованию №14 (контейнеризация).
3	Оркестрация контейнеров: Kubernetes	Управление контейнерами, автоматическое	Все модули (микросервисная архитектура).	Обеспечивает высокую доступность, автоматическое масштабирование при пиковых

№	Наименование ПО	Назначение	Обеспечиваемые подсистемы и функции	Обоснование выбора / Соответствие требованиям
		масштабирование, балансировка нагрузки, самоисцеление.		нагрузках (требование производительности №5), упрощает управление микросервисами.
4	Веб-сервер / обратный прокси: Nginx	Приём входящих запросов, раздача статики, балансировка нагрузки, SSL-терминация.	Модуль 1 (интерфейс взаимодействия), информационное обеспечение.	Высокая производительность, низкое потребление ресурсов, поддержка WebSocket (для голосового ввода), широкие возможности балансировки. Соответствует требованиям доступности и времени ответа.
5	Backend-фреймворк: Python (FastAPI)	Разработка API для взаимодействия с клиентами, бизнес-логика (поиск, управление пользователями, контентом).	Модуль 1, Модуль 2, частично Модуль 3.	Высокая производительность (асинхронность), автоматическая генерация OpenAPI-документации, удобство интеграции с NLP-библиотеками. Подходит для микросервисной архитектуры.
6	Frontend-фреймворк: React.js	Создание пользовательского интерфейса (SPA) с адаптивным дизайном.	Модуль 1 (интерфейс взаимодействия).	Широкая поддержка, компонентный подход, возможность создания отзывчивого интерфейса, соответствует требованиям удобства использования (№11, №12).
7	СУБД: PostgreSQL	Хранение структурированных данных: пользователи, метаданные контента, лицензии, заявки, история, избранное.	Модуль 2 (управление контентом), Модуль 3 (аналитика).	Надёжность, поддержка ACID-транзакций, расширяемость, наличие полнотекстового поиска (как резерв), репликация. Соответствует требованиям целостности данных и масштабирования.
8	Поисковый движок: Elasticsearch	Полнотекстовый поиск по контенту, ранжирование, фильтрация, семантический поиск (с векторами).	Модуль 2 (поиск), подсистема аналитики.	Скорость поиска, поддержка сложных запросов, релевантность, интеграция с NLP-моделями (векторный поиск). Необходим для выполнения функциональных требований №11, №16 (ранжирование).
9	Объектное	Хранение	Модуль 2	Лёгкость развёртывания,

№	Наименование ПО	Назначение	Обеспечиваемые подсистемы и функции	Обоснование выбора / Соответствие требованиям
	хранилище: MinIO	мультимедиа файлов (книги, видео, аудио) и их демо-версий, совместимое с S3 API.	(контент).	масштабируемость, высокая производительность, поддержка S3, что упрощает интеграцию с веб-серверами и CDN.
10	Брокер сообщений: RabbitMQ	Асинхронная обработка задач: внешний поиск, индексация, дообучение нейросети, отправка уведомлений.	Модуль 2 (внешний поиск), Модуль 3 (аналитика).	Надёжная доставка сообщений, поддержка очередей, гибкая маршрутизация, соответствие требованию асинхронности и отказоустойчивости.
11	Кэш и управление сессиями: Redis	Кэширование результатов запросов, хранение сессий пользователей, временных данных.	Модуль 1, Модуль 2.	Высокая скорость работы, поддержка структур данных, удобство для кэширования и сессий, снижает нагрузку на БД и ускоряет ответ (требование времени обработки).
12	Фреймворк для NLP / нейросетей: PyTorch	Реализация и обучение моделей семантического анализа, инференс.	Модуль 3 (семантический анализ), аналитическая подсистема.	Гибкость, производительность, широкое сообщество, поддержка GPU, наличие предобученных моделей для русского языка (RuBERT). Необходим для функциональных требований №7-10.
13	Предобученная NLP-модель: RuBERT (или многоязычный BERT)	Семантический анализ запросов, распознавание сущностей, определение интента.	Модуль 3 (семантический анализ).	Оптимизирована для русского языка, высокая точность, доступна в библиотеке transformers. Позволяет реализовать требования №7, №8.
14	Библиотека для обработки естественного языка: Hugging Face Transformers	Удобный интерфейс для загрузки и использования предобученных моделей, токенизация.	Модуль 3 (семантический анализ).	Стандарт де-факто для работы с трансформерами, интеграция с PyTorch, упрощает разработку NLP-компонентов.
15	Система сбора и анализа логов: ELK Stack (Elasticsearch,	Централизованный сбор, индексация и визуализация	Модуль 3 (аналитика), подсистема мониторинга.	Мощный инструмент для работы с логами, поддержка полнотекстового поиска по логам, создание дашбордов.

№	Наименование ПО	Назначение	Обеспечиваемые подсистемы и функции	Обоснование выбора / Соответствие требованиям
	Logstash, Kibana)	логов всех сервисов, мониторинг производительности, анализ инцидентов.		Соответствует требованиям логирования (№17) и аналитики.
16	Система мониторинга и оповещения: Prometheus + Grafana	Сбор метрик (время ответа, нагрузка на CPU/RAM, количество запросов), визуализация, алертинг.	Модуль 3 (аналитика), администраторы.	Широко распространённый стек, интеграция с Kubernetes, гибкие запросы (PromQL), красивые дашборды. Необходим для контроля доступности и производительности (требования №1-5).
17	Система управления API-шлюзом: Kong (или Traefik)	Маршрутизация запросов к микросервисам, аутентификация, rate limiting, логирование.	Информационное обеспечение, все модули.	Лёгкость настройки, поддержка плагинов, интеграция с Kubernetes, обеспечивает безопасность и контроль доступа (требования безопасности №6-10).
18	Система управления идентификацией и доступом: Keycloak	Аутентификация и авторизация пользователей (включая двухфакторную), управление ролями, SSO.	Модуль 1 (интерфейс), безопасность.	Поддержка стандартов (OAuth2, OpenID Connect), простота интеграции, управление пользователями и ролями (требование №9), соответствие Ф3-152.
19	Система резервного копирования: Bacula (или Veeam Backup)	Регулярное резервное копирование БД, файлового хранилища, конфигураций, моделей.	Все подсистемы (надёжность).	Гибкость расписания, поддержка различных хранилищ, возможность восстановления. Соответствует требованию RTO и целостности данных.
20	СУБД для временных рядов (метрик): Prometheus (TSDB)	Хранение метрик производительности.	Модуль 3 (аналитика).	Встроено в Prometheus, эффективно для временных рядов.
21	Система непрерывной интеграции и доставки: GitLab CI / Jenkins	Автоматизация сборки, тестирования и развёртывания микросервисов.	Администрирование, разработка.	Позволяет быстро и надёжно обновлять систему, снижает риск ошибок при развёртывании.
22	Протоколы и	Преобразование	Модуль 1	Обеспечивает функциональное

№	Наименование ПО	Назначение	Обеспечиваемые подсистемы и функции	Обоснование выбора / Соответствие требованиям
2	библиотеки для голосового ввода: Web Speech API (браузер) + бэкенд-сервис распознавания (например, Vosk или облачный сервис)	голоса в текст на стороне клиента или сервера.	(голосовой ввод).	требование №2. Выбор open-source решения (Vosk) для автономности или интеграция с облачными API при необходимости.

Выводы

В разделе 5 спроектирована программно-аппаратная архитектура АСУ «Семантический подборщик». Аппаратное обеспечение включает кластеризованные серверы баз данных, поискового движка, нейросетевых вычислений (GPU NVIDIA), объектного хранилища, балансировщиков и межсетевых экранов. Программное обеспечение реализует микросервисную архитектуру (Docker, Kubernetes, Nginx), нейросемантический анализ (PyTorch, RuBERT), хранение и поиск (PostgreSQL, Elasticsearch, MinIO, Redis), безопасность (Keycloak, TLS), мониторинг (Prometheus, ELK). Выбранные решения соответствуют требованиям производительности (время отклика ≤ 2 с, доступность $\geq 99.5\%$) и безопасности (TLS 1.3, изоляция модели). Результаты раздела 5 являются основой для детализации оборудования (раздел 9), создания виртуального стенда (раздел 10) и реализации программных модулей (разделы 6, 11–12).

РАЗДЕЛ 6. РЕАЛИЗАЦИЯ ПРОГРАММНОГО МОДУЛЯ ДООБУЧЕНИЯ НЕЙРОСЕТИ

Введение

Выбрав программно-аппаратную платформу (раздел 5), необходимо реализовать один из ключевых интеллектуальных модулей – автоматическое дообучение корпоративной нейросети. Эта функция критична для поддержания точности семантического анализа. Раздел 6 основан на материалах практической работы №6.

Цель раздела 6 – создать программный модуль для автоматического дообучения нейросети, обосновать выбор языка программирования, IDE и библиотек, а также представить работающий код.

Задачи раздела:

1. Определить функцию АСУ, требующую доработки (дообучение нейросети), с обоснованием.
2. Провести анализ существующих языков программирования и IDE.
3. Обоснованно выбрать IDE, язык и набор библиотек.
4. Реализовать программный код модуля в выбранной IDE.

Функция, требующая доработки

Выбранная функция: автоматическое дообучение корпоративной нейросети на новых данных.

Обоснование выбора:

- Критичность для интеллектуального ядра системы. Ключевым преимуществом системы является собственная нейросеть, которая должна "понимать" предметную область и контекст. Без механизма дообучения модель со временем устаревает, снижается качество распознавания новых терминов и трендов.
- Наличие необходимых данных. Предусмотрен сбор логов запросов и действий пользователей. Эти данные являются идеальным

материалом для дообучения модели, чтобы она лучше соответствовала реальным паттернам поведения пользователей.

- Автоматизация без участия человека. В концепции TO-BE подчеркивается замена человека-менеджера. Однако процесс дообучения модели, хоть и автоматизирован по сбору данных, требует создания программного модуля, который будет запускать этот процесс (возможно, по расписанию или при накоплении критической массы новых данных), контролировать его и разворачивать обновленную модель. Это сложная инженерная задача, выходящая за рамки простого использования готовой библиотеки.
- Соответствие ролям пользователей. Определена роль "Администратор нейросети", в чьи обязанности входит "Обучение и дообучение модели". Разработка данного модуля позволит администратору выполнять эти функции через удобный интерфейс (или автоматический триггер), а не вручную на сервере.

Анализ существующих языков программирования и IDE

Для реализации функции дообучения нейросетевой модели (NLP) и интеграции её с основной системой рассмотрим два основных языка и соответствующие IDE.

Языки программирования

Язык	Преимущества для данной задачи	Недостатки
Python	Абсолютное большинство библиотек для работы с нейросетями (PyTorch, TensorFlow) и NLP (Hugging Face, spaCy) написаны для Python. Простота синтаксиса позволяет быстро прототипировать и экспериментировать с архитектурами. Легко интегрируется с бэкендом на FastAPI. Огромное сообщество.	Более низкая скорость выполнения по сравнению с компилируемыми языками, но для задач обучения на GPU это не критично, т.к. основные вычисления идут на видеокартах. GIL может быть ограничением для многопоточности на CPU, но обучение моделей обычно масштабируется через многопроцессность.
C++	Позволяет более тонко управлять памятью и аппаратными ресурсами. Используется в продакшн-среде для инференса моделей, где важна каждая миллисекунда	Разработка и отладка алгоритмов машинного обучения на C++ значительно сложнее и медленнее. Меньший выбор библиотек для NLP по

Язык	Преимущества для данной задачи	Недостатки
	(например, в LibTorch — C++ API PyTorch).	сравнению с Python. Для задачи дообучения, которая выполняется не в реальном времени (фоновый процесс), преимущества в скорости не являются определяющими.

Сравнительный анализ языков программирования показывает, что для реализации задачи автоматического дообучения нейросетевой модели выбор должен основываться на специфике решаемой задачи и требованиях к процессу разработки. Python является безусловным лидером в области машинного обучения благодаря богатой экосистеме специализированных библиотек (PyTorch, Transformers, Datasets), простоте прототипирования и широкому сообществу разработчиков. Эти факторы критически важны для задачи дообучения, которая требует частых экспериментов с архитектурой модели и гиперпараметрами. C++, несмотря на высокую производительность, нецелесообразен для данной задачи, так как основная вычислительная нагрузка ложится на GPU (где скорость выполнения операций определяется библиотеками вроде cuDNN, а не самим языком), а сложность разработки и отладки на C++ значительно замедлит создание и модификацию модуля. Таким образом, для реализации функции дообучения корпоративной нейросети в составе АСУ «Семантический подборщик» оптимальным и обоснованным выбором является язык Python.

Интегрированные среды разработки (IDE)

IDE	Преимущества для данной задачи	Недостатки
PyCharm Community	Бесплатная и легковесная версия популярной IDE от JetBrains. Обеспечивает отличную поддержку языка Python: умный автокомплит, навигация по коду, рефакторинг, отладчик. Имеет встроенную поддержку научных библиотек (NumPy, PyTorch) и интеграцию с Jupyter Notebooks, что удобно для экспериментов с моделями. Проста в установке и настройке.	Отсутствуют инструменты для веб-разработки (шаблоны, базы данных) и поддержка баз данных «из коробки», но для задачи дообучения это не критично, так как подключение к БД будет осуществляться через отдельные библиотеки (SQLAlchemy). Нет встроенной поддержки удалённой разработки (Remote Development), однако код можно синхронизировать через Git, а запуск на сервере

IDE	Преимущества для данной задачи	Недостатки
Visual Studio Community	Мощная, бесплатная IDE от Microsoft с широкими возможностями для разработки на C++, C#, Python и других языках. Имеет отличный отладчик, профилировщик, поддержку CMake и интеграцию с Git. Для C++ является стандартом де-факто в среде Windows.	осуществлять через CI/CD. Избыточна для чисто Python-разработки, требует много дискового пространства и ресурсов. Ориентирована преимущественно на экосистему Windows, в то время как целевой сервер работает под Linux. Поддержка Python в Visual Studio уступает специализированным IDE по удобству работы с научными библиотеками и Jupyter.

Проведённый анализ показывает, что выбор IDE напрямую зависит от языка реализации и целевой инфраструктуры. Для решения задачи дообучения нейросети на языке Python оптимальным вариантом является PyCharm Community, поскольку эта IDE предоставляет все необходимые инструменты для Python-разработки (интеллектуальное редактирование кода, отладка, поддержка Jupyter и научных библиотек) при нулевой стоимости и простоте использования. Visual Studio Community, несмотря на свою мощь и бесплатность, ориентирована преимущественно на разработку под Windows и языки семейства C++, что делает её избыточной и менее удобной для работы в среде Linux с Python-стеком машинного обучения. Таким образом, для реализации модуля автоматического дообучения АСУ «Семантический подборщик» наиболее рациональным выбором является PyCharm Community.

Обоснованный выбор инструментов

На основе проведенного анализа для реализации модуля автоматического дообучения корпоративной нейросети делается следующий выбор.

Язык программирования: Python 3.11+.

Обоснование: Язык является стандартом для задач машинного обучения и обработки естественного языка. Позволит использовать весь необходимый стек библиотек (PyTorch, Transformers) и легко интегрировать модуль с остальными микросервисами системы (RabbitMQ для получения

задач, PostgreSQL для чтения логов/метаданных), написанными на том же языке.

Интегрированная среда разработки (IDE): PyCharm Community Edition.

Обоснование: Выбор обусловлен тем, что модуль дообучения будет разрабатываться исключительно на Python. PyCharm Community предоставляет все необходимые инструменты для комфортной разработки на этом языке: интеллектуальное редактирование кода, навигацию, отладку и поддержку научных библиотек (PyTorch, Transformers). Бесплатность IDE важна для учебного проекта и соответствует политике использования открытого ПО. Интеграция с Jupyter Notebooks, доступная в PyCharm, позволит проводить эксперименты с дообучением модели в интерактивном режиме перед встраиванием в основной код. Отсутствие встроенных средств для удалённой разработки компенсируется использованием системы контроля версий Git и пайплайнов непрерывной интеграции (CI/CD), что полностью согласуется с выбранной инфраструктурой (Docker, Kubernetes) и нефункциональными требованиями к автоматизации развёртывания.

Выбор и описание библиотек

Библиотека	Назначение	Обоснование / Соответствие требованиям
PyTorch	Основной фреймворк для создания, обучения и дообучения нейросетевых моделей.	Соответствует выбору ПО. Обеспечивает гибкость, необходимую для тонкой настройки (fine-tuning) предобученных трансформеров, и эффективно использует GPU (CUDA).
Transformers (Hugging Face)	Предоставляет тысячи предобученных моделей, классы для их загрузки, токенизации и дообучения.	Позволяет не создавать NLP-модель "с нуля", а взять качественную базовую модель и дообучить её на специфических данных системы (запросы пользователей), чтократно ускоряет разработку и повышает качество.
Datasets (Hugging Face)	Для эффективной загрузки и предобработки больших датасетов (логов запросов) перед подачей в модель. Позволяет работать с данными, которые не помещаются в	Логи запросов могут накапливаться в больших объемах. Библиотека обеспечит эффективную работу с ними, что соответствует требованию масштабируемости.

Библиотека	Назначение	Обоснование / Соответствие требованиям
	оперативную память.	
SQLAlchemy	Библиотека для взаимодействия с базами данных (PostgreSQL). Позволит модулю дообучения читать обезличенные логи запросов и историю взаимодействий из БД для формирования тренировочного датасета.	Необходима для интеграции с Модулем 3 (Аналитика), где хранятся логи.
Pydantic	Для валидации конфигураций и параметров запуска дообучения (количество эпох, размер батча, гиперпараметры), которые может задавать администратор нейросети.	Обеспечит надежность и типобезопасность при настройке процесса дообучения, что важно для стабильности системы.
APScheduler / Celery	Для планирования и запуска задач дообучения. Модуль может работать как фоновый процесс, который получает команду на запуск.	Соответствует архитектуре, где Модуль 3 иницирует процесс. Обеспечивает асинхронность и не блокирует работу других сервисов.
MLflow	Платформа для управления жизненным циклом машинного обучения. Позволит логировать параметры каждой попытки дообучения, метрики качества, а также версионировать полученные модели.	Критически важно для администрирования и воспроизводимости результатов. Администратор нейросети сможет сравнивать разные версии модели и при необходимости откатываться к предыдущей рабочей версии, что повышает надежность системы.

Программный код

```
import os
import pandas as pd
import torch
from torch.utils.data import Dataset, DataLoader
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    Trainer,
    TrainingArguments,
)
from transformers import EarlyStoppingCallback
from pydantic import Field
from pydantic_settings import BaseSettings, SettingsConfigDict
import mlflow
import mlflow.pytorch
from sklearn.metrics import accuracy_score, precision_recall_fscore_support
import logging
import sys

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
    handlers=[logging.StreamHandler(sys.stdout)],
```

```

)
logger = logging.getLogger(__name__)

class FineTuneConfig(BaseSettings):
    base_model_name: str = "cointegrated/rubert-tiny2"
    data_path: str = "./data/new_queries.csv"
    output_dir: str = "./models/finetuned"
    num_train_epochs: int = Field(3, ge=1, le=10)
    per_device_train_batch_size: int = 8
    per_device_eval_batch_size: int = 8
    learning_rate: float = Field(2e-5, gt=0)
    warmup_steps: int = 100
    logging_steps: int = 10
    eval_steps: int = 50
    save_steps: int = 100
    max_seq_length: int = 128
    early_stopping_patience: int = 3
    mlflow_experiment_name: str = "semantic_picker_finetune"
    mlflow_run_name: str = "auto_finetune"
    model_config = SettingsConfigDict(
        env_file=".env",
        env_file_encoding="utf-8",
        extra="ignore"
    )

config = FineTuneConfig()
logger.info(f"Конфигурация загружена: {config.model_dump()}")

if not os.path.exists(config.data_path):
    os.makedirs(os.path.dirname(config.data_path), exist_ok=True)
    sample_data = """text, label
хочу изучить Python, 0
где скачать книгу по Go, 2
купить курс по машинному обучению, 1
научиться работать с нейросетями, 0
посоветуйте видео по Django, 2
хочу приобрести подписку, 1
изучение основ программирования, 0
стоимость доступа к библиотеке, 1
поиск аудиокниг по программированию, 2
обучение с нуля, 0
"""
    with open(config.data_path, "w", encoding="utf-8") as f:
        f.write(sample_data)
    logger.info(f"Создан пример файла данных: {config.data_path}")

class QueriesDataset(Dataset):
    def __init__(self, encodings, labels):
        self.encodings = encodings
        self.labels = labels

    def __getitem__(self, idx):
        item = {key: torch.tensor(val[idx]) for key, val in
self.encodings.items()}
        item["labels"] = torch.tensor(self.labels[idx])
        return item

    def __len__(self):

```

```

        return len(self.labels)

def load_data(data_path: str, tokenizer, max_length: int):
    logger.info(f"Загрузка данных из {data_path}")
    df = pd.read_csv(data_path)
    if "text" not in df.columns or "label" not in df.columns:
        raise ValueError("CSV должен содержать колонки 'text' и 'label'")
    texts = df["text"].tolist()
    labels = df["label"].tolist()
    encodings = tokenizer(
        texts,
        truncation=True,
        padding=True,
        max_length=max_length,
        return_tensors=None,
    )
    dataset = QueriesDataset(encodings, labels)
    logger.info(f"Загружено {len(dataset)} примеров")
    return dataset

def compute_metrics(eval_pred):
    predictions, labels = eval_pred
    predictions = predictions.argmax(axis=-1)
    precision, recall, f1, _ = precision_recall_fscore_support(labels,
predictions, average="weighted")
    acc = accuracy_score(labels, predictions)
    return {"accuracy": acc, "f1": f1, "precision": precision, "recall":
recall}

def main():
    mlflow.set_experiment(config.mlflow_experiment_name)
    with mlflow.start_run(run_name=config.mlflow_run_name) as run:
        mlflow.log_params(config.model_dump())
        logger.info(f"Загрузка базовой модели: {config.base_model_name}")
        tokenizer = AutoTokenizer.from_pretrained(config.base_model_name)
        model =
AutoModelForSequenceClassification.from_pretrained(config.base_model_name,
num_labels=10)

        dataset = load_data(config.data_path, tokenizer,
config.max_seq_length)
        train_size = int(0.8 * len(dataset))
        eval_size = len(dataset) - train_size
        train_dataset, eval_dataset = torch.utils.data.random_split(dataset,
[train_size, eval_size])
        logger.info(f"Разделение: train={len(train_dataset)},
eval={len(eval_dataset)}")

        training_args = TrainingArguments(
            output_dir=config.output_dir,
            num_train_epochs=config.num_train_epochs,
            per_device_train_batch_size=config.per_device_train_batch_size,
            per_device_eval_batch_size=config.per_device_eval_batch_size,
            learning_rate=config.learning_rate,
            warmup_steps=config.warmup_steps,
            logging_steps=config.logging_steps,
            eval_strategy="steps",
            eval_steps=config.eval_steps,

```

```

        save_strategy="steps",
        save_steps=config.save_steps,
        load_best_model_at_end=True,
        metric_for_best_model="accuracy",
        greater_is_better=True,
        report_to="mlflow",
        run_name=mlflow.active_run().info.run_name,
    )

    trainer = Trainer(
        model=model,
        args=training_args,
        train_dataset=train_dataset,
        eval_dataset=eval_dataset,
        compute_metrics=compute_metrics,

callbacks=[EarlyStoppingCallback(early_stopping_patience=config.early_stopping_patience)],
    )

    logger.info("Начало дообучения...")
    trainer.train()

    model.save_pretrained(config.output_dir)
    tokenizer.save_pretrained(config.output_dir)
    logger.info(f"Модель сохранена в {config.output_dir}")

    mlflow.pytorch.log_model(model, artifact_path="model")
    logger.info("Модель залогирована в MLflow")

if __name__ == "__main__":
    main()

```

Выводы

Реализован модуль автоматического дообучения нейросети на Python 3.11+ с использованием PyTorch, Hugging Face Transformers, Datasets, SQLAlchemy, Pydantic, MLflow, Celery/APScheduler. В качестве IDE выбрана PyCharm Community Edition. Модуль загружает конфигурацию, читает датасет, выполняет fine-tuning модели RuBERT, логирует параметры и метрики в MLflow, сохраняет модель и токенизатор, реализует early stopping. Код может быть асинхронно запущен через RabbitMQ и интегрирован с базой данных (раздел 8). Разработанный модуль является ядром интеллектуальной подсистемы АСУ.

РАЗДЕЛ 7. ОПИСАНИЕ КЛАССОВ И ПОСТРОЕНИЕ ER- ДИАГРАММЫ

Введение

Диаграмма классов (раздел 4) описывает статическую структуру системы на уровне объектно-ориентированного проектирования. Для перехода к реляционной базе данных необходимо детализировать свойства и методы каждого класса и построить ER-диаграмму. Раздел 7 основан на материалах практической работы №7.

Цель раздела 7 – детализировать свойства и методы выбранных классов, построить ER-диаграмму, соответствующую диаграмме классов.

Задачи раздела:

1. Выделить ключевые классы и описать их свойства с указанием типов данных.
2. Для каждого свойства указать, в каких методах оно используется.
3. Построить ER-диаграмму с преобразованием классов в таблицы, свойств – в поля, связей – в связи между таблицами.
4. Добавить служебные поля (первичные ключи) и проверить соответствие диаграмме классов.

Выделение и описание полей (свойств) и методов выбранных классов

Класс: ВнешнийИсточник

Свойство	Тип данных	Методы, использующие свойство
название	string	получитьКонтент(), проверитьПодключение()
базовый_url	string	получитьКонтент(), проверитьПодключение()
api_ключ	string	получитьКонтент(), проверитьПодключение(), обновитьКлюч()
описание	string	получитьКонтент()
статус	string	получитьКонтент(), проверитьПодключение()
параметры_подключения	string	получитьКонтент(), проверитьПодключение()
формат_данных	string	получитьКонтент()

Методы класса:

- получитьКонтент(запрос): List<Контент> – формирует HTTP-запрос к внешнему API, используя базовый_url, api_ключ, параметры_подключения и формат_данных, выполняет поиск и возвращает список найденного контента;
- проверитьПодключение(): bool – проверяет доступность внешнего источника, отправляя тестовый запрос на базовый_url с api_ключом; возвращает true, если соединение успешно;
- обновитьКлюч(новый_ключ): void – заменяет значение api_ключ на новый, после чего обновляет статус подключения.

Класс: Запрос

Свойство	Тип данных	Методы, использующие свойство
текст	string	обработать(), получитьРезультаты()
время	DateTime	обработать(), получитьРезультаты()
структурированные_параметры	string	обработать(), получитьРезультаты()
статус	string	обработать(), получитьРезультаты()
источник_запроса	string	обработать()
ip_адрес	string	обработать()

Методы класса:

- обработать(): void – анализирует текст запроса с помощью нейросети, заполняет структурированные_параметры, устанавливает статус ("принят", "ошибка") и фиксирует время;
- получитьРезультаты(): List<Контент> – использует структурированные_параметры для поиска контента во внутренней базе и внешних источниках; возвращает список релевантного контента.

Класс: ЗаявкаНаЗакупку

Свойство	Тип данных	Методы, использующие свойство
дата_создания	DateTime	утвердить(), отклонить(), уведомитьРуководителя()
статус	string	утвердить(), отклонить(), уведомитьРуководителя()
бюджет	float	утвердить(), уведомитьРуководителя()
комментарии	string	утвердить(), отклонить()
предлагаемый_контент	string	утвердить(), отклонить()
обоснование	string	утвердить(), уведомитьРуководителя()
приоритет	int	утвердить(), уведомитьРуководителя()

Методы класса:

- утвердить(утвердивший): void – меняет статус на "утверждена", проверяет бюджет и приоритет, вызывает уведомитьРуководителя();
- отклонить(причина): void – устанавливает статус "отклонена", записывает причина в комментарии;
- уведомитьРуководителя(): void – отправляет уведомление руководителю с данными о бюджете, обосновании, приоритете и предлагаемом_контенте.

Класс: Лог

Свойство	Тип данных	Методы, использующие свойство
время	DateTime	записать (), поиск ()
уровень	string	записать (), поиск ()
сообщение	string	записать (), поиск ()
источник	string	записать (), поиск ()
детали	string	записать (), поиск ()
пользователь	Пользователь	записать (), поиск ()
сессия	Сессия	записать (), поиск ()

Методы класса:

- записать (уровень, сообщение, детали): void – создаёт новую запись лога, заполняя время (текущее), уровень, сообщение, детали, а также привязывает пользователя и сессию, если они известны;
- поиск (критерии): List<Лог> – ищет логи по заданным критериям (например, по уровню, источнику, диапазону времени, пользователю) и возвращает список.

Класс: Контент

Свойство	Тип данных	Методы, использующие свойство
название	string	получитьМетаданные (), обновитьРейтинг (), добавитьОтзыв ()
тип	string	получитьМетаданные (), проверитьДоступ ()
авторы	string	получитьМетаданные ()
описание	string	получитьМетаданные ()
уровень	string	проверитьДоступ ()
ссылка_на_файл	string	проверитьДоступ ()
дата_добавления	DateTime	получитьМетаданные ()
язык	string	получитьМетаданные (), проверитьДоступ ()
средний_рейтинг	float	обновитьРейтинг (), получитьМетаданные ()
количество_просмотров	int	обновитьРейтинг ()

Методы класса:

- получитьМетаданные (): map – возвращает словарь со всеми свойствами контента (название, тип, авторы, описание,

уровень, дата_добавления, язык, средний_рейтинг).

- обновитьРейтинг(оценка): void – пересчитывает средний_рейтинг на основе новой оценки и увеличивает количество_просмотров.
- проверитьДоступ(пользователь): bool – проверяет, может ли пользователь получить доступ к контенту (по уровню, типу, ссылке_на_файл и лицензиям).
- добавитьОтзыв(отзыв): void – добавляет отзыв к контенту, после чего вызывает обновитьРейтинг (отзыв.оценка).

Класс: Избранное

Свойство	Тип данных	Методы, использующие свойство
дата_добавления	DateTime	добавитьВИзбранное(), получитьИзИзбранногоПользователя()
заметка	string	добавитьВИзбранное(), удалитьИзИзбранного()
теги	string	добавитьВИзбранное()
приоритет	int	добавитьВИзбранное()
контент	Контент	добавитьВИзбранное(), удалитьИзИзбранного(), получитьИзИзбранногоПользователя()
пользователь	Пользователь	добавитьВИзбранное(), удалитьИзИзбранного(), получитьИзИзбранногоПользователя()

Методы класса:

- добавитьВИзбранное(пользователь, контент): bool – создаёт запись в избранном, фиксирует дата_добавления, сохраняет заметку, теги, приоритет (если переданы);
- удалитьИзИзбранного(): bool – удаляет запись, идентифицируя её по пользователю и контенту;
- получитьИзИзбранногоПользователя(пользователь): List<Контент> – возвращает список всех контентов, добавленных в избранное указанным пользователем, отсортированный по дата_добавления.

Класс: История

Свойство	Тип данных	Методы, использующие свойство
тип_действия	string	записатьДействие(), получитьИсториюПользователя()
время	DateTime	записатьДействие(), получитьИсториюПользователя()
детали	string	записатьДействие()
контент	Контент	записатьДействие(), получитьИсториюПользователя()
сессия	Сессия	записатьДействие()
длительность	int	записатьДействие()

Методы класса:

- записатьДействие(пользователь, действие, контент):
void – создаёт запись в истории, заполняя тип_действия (просмотр, скачивание, оценка), время, детали, контент, сессию и длительность (если применимо);
- получитьИсториюПользователя(пользователь):
List<История> – возвращает все записи истории для данного пользователя, отсортированные по время (от новых к старым).

Класс: Отзыв

Свойство	Тип данных	Методы, использующие свойство
оценка	int	модерировать(), получитьСреднююОценку(), редактировать()
текст	string	модерировать(), редактировать()
дата	DateTime	модерировать(), получитьСреднююОценку()
статус_модерации	string	модерировать()
полезность	int	модерировать()
жалобы	int	модерировать()

Методы класса:

- модерировать(новый_статус, модератор): void – изменяет статус_модерации (например, "опубликован", "отклонён"), при отклонении может увеличить счётчик жалоб;
- получитьСреднююОценку(контент): float – статический метод, собирает все отзывы для указанного контента и вычисляет

среднее арифметическое их оценок;

- редактировать (новый_текст): void – позволяет автору изменить текст отзыва, сохраняя старую версию в детали лога (или просто обновляет поле).

Класс: НейросетеваяМодель

Свойство	Тип данных	Методы, использующие свойство
название	string	обучить (), предсказать (), оценить ()
версия	string	обучить (), оценить ()
дата_обучения	DateTime	обучить (), оценить ()
точность	float	обучить (), оценить ()
путь_к_модели	string	обучить (), предсказать ()
статус	string	обучить (), предсказать (), оценить ()
гиперпараметры	string	обучить ()
размер_модели	int	обучить ()

Методы класса:

- обучить (датасет): void – запускает процесс обучения (или дообучения) на переданном датасете, обновляет дата_обучения, версию, точность, размер_модели, сохраняет модель по путь_к_модели;
- предсказать (текст): float – загружает модель из путь_к_модели, выполняет инференс над входным текстом и возвращает числовое предсказание (например, вектор эмбедингов или класс);
- оценить (): void – вычисляет текущую точность модели на валидационной выборке и обновляет соответствующее свойство.

Класс: Датасет

Свойство	Тип данных	Методы, использующие свойство
название	string	загрузить(), обновитьСтатистику()
описание	string	загрузить()
количество_записей	int	загрузить(), разделить(), обновитьСтатистику()
дата_создания	DateTime	загрузить()
формат	string	загрузить()
путь_к_файлу	string	загрузить()
статус	string	загрузить(), обновитьСтатистику()
источник_данных	string	загрузить()

Методы класса:

- загрузить(): List – читает данные из файла по путь_к_файлу, учитывая формат (CSV, JSON), возвращает список записей;
- разделить(коэффициент): (Датасет, Датасет) – разделяет текущий датасет на обучающий и валидационный в пропорции коэффициент (например, 0.8), создавая два новых объекта Датасет;
- обновитьСтатистику(): void – пересчитывает количество_записей и обновляет статус (например, "готов", "пуст") на основе данных по путь_к_файлу.

Класс: Лицензия

Свойство	Тип данных	Методы, использующие свойство
тип	string	проверитьСрок(), продлить(), получитьКонтентПоЛицензии()
правообладатель	string	проверитьСрок(), продлить()
дата_начала	Date	проверитьСрок(), продлить()
дата_окончания	Date	проверитьСрок(), продлить()
стоимость	float	продлить()
ограничения	string	проверитьСрок()
условия_использования	string	проверитьСрок()

Методы класса:

- проверитьСрок(): bool – сравнивает текущую дату с дата_окончания, возвращает true, если лицензия ещё действует;
- продлить(новая_дата): void – изменяет дата_окончания

на новую, возможно, обновляет стоимость при продлении;

- `получитьКонтентПоЛицензии()` : `List<Контент>` – возвращает список всех контентов, привязанных к данной лицензии (используя связь через базу данных).

Класс: Категория

Свойство	Тип данных	Методы, использующие свойство
название	string	добавитьПодкатегорию(), обновитьНазвание()
описание	string	добавитьПодкатегорию(), получитьКонтентКатегории()
дата_создания	Date	добавитьПодкатегорию()
количество_контента	int	получитьКонтентКатегории(), обновитьНазвание()
иконка	string	добавитьПодкатегорию()
родительская_категория	Категория	добавитьПодкатегорию(), получитьКонтентКатегории()

Методы класса:

- `добавитьПодкатегорию(подкатегория)` : `void` – добавляет новую дочернюю категорию, связывая её с текущей через `родительская_категория`;
- `получитьКонтентКатегории()` : `List<Контент>` – возвращает все контенты, которые относятся к данной категории, а также к её подкатегориям (рекурсивно). Использует `количество_контента` для быстрого подсчёта;
- `обновитьНазвание(новое_название)` : `void` – изменяет название категории и, при необходимости, пересчитывает связанные метаданные.

Класс: Пользователь

Свойство	Тип данных	Методы, использующие свойство
имя	string	зарегистрироваться(), обновитьПрофиль()
электронная_почта	string	зарегистрироваться(), войти(), обновитьПрофиль()
хеш_пароля	string	зарегистрироваться(), войти()
дата_регистрации	Date	зарегистрироваться()
роль	string	войти(), получитьИсторию()
статус	string	войти(), обновитьПрофиль()
телефон	string	обновитьПрофиль()

Методы класса:

- зарегистрироваться(имя, почта, пароль): bool – создаёт нового пользователя, сохраняя имя, электронная_почта, вычисляя хеш_пароля и фиксируя дата_регистрации;
- войти(почта, пароль): bool – проверяет, совпадают ли введённые почта и пароль с сохранёнными электронная_почта и хеш_пароля, а также что статус пользователя активен;
- обновитьПрофиль(данные): void – обновляет поля имя, телефон, электронная_почта (с перепроверкой) и может изменить статус;
- получитьИсторию(): List<История> – возвращает историю действий пользователя, используя его идентификатор (свойства имя или электронная_почта).

Класс: Сессия

Свойство	Тип данных	Методы, использующие свойство
время_начала	DateTime	обновитьКонтекст(), закрыть(), последниеЗапросы()
время_окончания	DateTime	закрыть()
контекст	string	обновитьКонтекст(), последниеЗапросы()
текущий_запрос	Запрос	обновитьКонтекст(), последниеЗапросы()
устройство	string	обновитьКонтекст()
браузер	string	обновитьКонтекст()

Методы класса:

- обновитьКонтекст(новые_данные): void – обновляет поле контекст (например, добавляет информацию о последнем запросе), фиксирует устройство и браузер, если они изменились;
- закрыть(): void – устанавливает время_окончания равным текущему моменту, после чего сессия считается завершённой;
- последниеЗапросы(): List<Запрос> – возвращает список запросов, сделанных в рамках этой сессии, начиная от время_начала и заканчивая время_окончания (если сессия закрыта) или текущим временем.

Класс: Уведомление

Свойство	Тип данных	Методы, использующие свойство
тип	string	отправить(), получитьУведомленияПользователя()
заголовок	string	отправить(), отметитьКакПрочитанное()
сообщение	string	отправить()
дата	DateTime	отправить(), получитьУведомленияПользователя()
прочитано	bool	отметитьКакПрочитанное(), получитьУведомленияПользователя()
ссылка	string	отправить()
пользователь	Пользователь	отправить(), получитьУведомленияПользователя()

Методы класса:

- отправить(пользователь, заголовок, сообщение): void – создаёт новое уведомление, заполняя тип (по умолчанию "системное"), заголовок, сообщение, ссылку (если есть), дату (текущая), прочитано = false и привязывает пользователя;
- отметитьКакПрочитанное(): void – устанавливает свойство прочитано = true;
- получитьУведомленияПользователя(пользователь): List<Уведомление> – возвращает все уведомления для указанного пользователя, отсортированные по дата (сначала новые),

Построение ER-диаграммы на основе классовой диаграммы



Детализированы 13 классов: ВнешнийИсточник, Запрос, ЗаявкаНаЗакупку, Лог, Контент, Избранное, Отзыв, НейросетеваяМодель, Датасет, Лицензия, Категория, Пользователь, Сессия. Для каждого класса приведены атрибуты с типами данных и методы с указанием используемых свойств. Построена ER-диаграмма, включающая все указанные сущности, их атрибуты, первичные и внешние ключи, связи с кратностью. Диаграмма полностью соответствует исходной диаграмме классов (раздел 4) и готова для физической реализации в разделе 8.

РАЗДЕЛ 8. СОЗДАНИЕ БАЗЫ ДАННЫХ В POSTGRESQL И ЗАПОЛНЕНИЕ ДАННЫМИ

Введение

На основе ER-диаграммы (раздел 7) необходимо выбрать реляционную СУБД, создать физическую базу данных, наполнить её тестовыми данными и выполнить проверочные запросы. Раздел 8 основан на материалах практической работы №8.

Цель раздела 8 – создать базу данных в выбранной СУБД в соответствии с ER-диаграммой, заполнить её данными (не менее 10 записей в каждой таблице) и проверить работоспособность аналитическим запросом.

Задачи раздела:

1. Провести сравнение не менее трёх реляционных СУБД.
2. Обоснованно выбрать СУБД.
3. Написать SQL-скрипты создания таблиц.
4. Заполнить каждую таблицу не менее чем 10 записями.
5. Создать и выполнить сложный аналитический запрос.

Выбор реляционной СУБД

Для проектируемой АСУ «Семантический подборщик» необходимо выбрать реляционную систему управления базами данных, которая обеспечит хранение метаданных контента, информации о пользователях, истории запросов, заявок на закупку и служебных данных для работы нейросетевой подсистемы. Рассмотрим три распространённые реляционные СУБД: PostgreSQL, MySQL и SQLite.

PostgreSQL

Общая характеристика: Свободная объектно-реляционная СУБД с открытым исходным кодом, ориентированная на максимальное соответствие стандарту SQL, надёжность и расширяемость. Широко применяется в

высоконагруженных системах и проектах, требующих сложной обработки данных.

Возможности, релевантные для проектируемой АСУ:

Полная поддержка ACID-транзакций, необходимая для оформления заявок на закупку и управления лицензиями.

Нативная работа с типом JSON/JSONB, что критически важно для хранения структурированных параметров запросов, извлекаемых нейросетью, и для гибких метаданных контента из внешних источников.

Полнотекстовый поиск с поддержкой русского языка, который может использоваться как дополнение к поисковому движку Elasticsearch.

Поддержка внешних ключей и каскадных операций, обеспечивающая ссылочную целостность между таблицами Контент, Категория, Лицензия, Пользователь и ЗаявкаНаЗакупку.

Возможность создания хранимых процедур на PL/pgSQL для реализации бизнес-логики, например, автоматического обновления статуса заявки или подсчёта рейтинга контента.

Механизм MVCC для высокой конкурентной обработки одновременных запросов множества пользователей.

Используемая версия языка запросов: SQL стандарта SQL:2016 (с элементами SQL:2023) и фирменные расширения PostgreSQL.

MySQL (Community Edition)

Общая характеристика: Самая популярная СУБД с открытым исходным кодом для веб-приложений. Отличается высокой скоростью операций чтения и простотой администрирования.

Возможности, релевантные для проектируемой АСУ:

Движок InnoDB обеспечивает поддержку транзакций и внешних ключей, что необходимо для целостности данных о закупках и правах доступа.

Тип данных JSON с возможностью индексации виртуальными колонками (с версии 5.7).

Простота настройки репликации Master-Slave для масштабирования нагрузки по чтению.

Широкий выбор GUI-инструментов (MySQL Workbench).

Используемая версия языка запросов: Дialect SQL, основанный на стандарте SQL:2003, с некоторыми отклонениями от стандарта в синтаксисе и поведении GROUP BY.

SQLite

Общая характеристика: Встраиваемая библиотека, реализующая полноценный SQL-движок без отдельного серверного процесса. Вся база данных хранится в одном кроссплатформенном файле.

Возможности, релевантные для проектируемой АСУ:

Нулевое администрирование и высокая портативность, что удобно на этапе прототипирования.

Поддержка ACID-транзакций и большинства конструкций современного SQL (включая оконные функции и общие табличные выражения).

Минимальное потребление ресурсов, идеально для локального тестирования скриптов.

Используемая версия языка запросов: Основной упор на SQL-92 с добавлением многих функций из более новых стандартов.

Сравнительный анализ

Для выбора оптимальной СУБД выделим ключевые критерии, продиктованные функциональными и нефункциональными требованиями к АСУ «Семантический подборщик».

Критерий	PostgreSQL	MySQL	SQLite
Поддержка конкурентного доступа	Отличная (MVCC), множество одновременных	Хорошая (InnoDB), небольшие задержки при	Отсутствует (только один писатель одновременно)

	писателей	интенсивной записи	
Работа с JSON и полуструктурированными данными	Наилучшая (тип JSONB с бинарным хранением и индексами GIN)	Хорошая (тип JSON, индексация через виртуальные колонки)	Хорошая (функции JSON, но без специализированного бинарного типа)
Полнотекстовый поиск (на русском языке)	Встроенный, с поддержкой словарей ispell и стемминга	Встроенный, но требует настройки ngram-парсера для русского	Встроенный (FTS3/FTS5), но без морфологии по умолчанию
Сложность администрирования	Высокая (требуется настройка postgresql.conf)	Средняя	Нулевая (встраиваемая)
Совместимость с выбранным стеком (Python/SQLAlchemy)	Отличная (полная поддержка всех типов и функций)	Отличная	Отличная
Соответствие стандарту SQL	Очень высокое	Среднее	Высокое

Преимущества и недостатки рассмотренных СУБД

- PostgreSQL

- Преимущества:

- Гибкость структуры данных: Тип JSONB позволяет хранить нечёткие метаданные, получаемые от разных внешних источников (ВнешнийИсточник), в едином поле без потери возможности индексации и быстрого поиска.
 - Транзакционная целостность: Критически важна для процесса обработки ЗаявкаНаЗакупку и учёта Лицензия, где недопустимы частичные изменения данных.
 - Расширяемость: В будущем позволит добавить в БД модуль pgvector для хранения векторных представлений запросов (эмбеддингов), создаваемых НейросетеваяМодель, что улучшит семантический поиск без необходимости всегда обращаться к внешнему Elasticsearch.

- Недостатки:

- Требуется более тщательной первоначальной настройки параметров использования памяти и дискового ввода-вывода.

- Стандартная утилита `pg_dump` создаёт резервные копии несколько медленнее, чем специализированные средства MySQL.

- MySQL

- Преимущества:

- Высокая производительность на простых операциях чтения метаданных Контент по первичному ключу, что важно для интерфейса просмотра карточек.
 - Огромное количество готовых решений и документации по интеграции с веб-фреймворками.

- Недостатки:

- Вольная трактовка стандарта SQL (например, режим `ONLY_FULL_GROUP_BY` по умолчанию был отключён в старых версиях), что может привести к скрытым логическим ошибкам в аналитических отчётах (подсистема «Аналитика и администрирование»).
 - Работа с JSON менее производительна и удобна, чем с JSONB в PostgreSQL, особенно при глубокой вложенности метаданных.

- SQLite

- Преимущества:

- Идеальная среда для быстрой проверки SQL-скриптов создания таблиц и тестового наполнения данными на локальной машине разработчика без развёртывания сервера.

- Недостатки:

- Невозможность использования в многопользовательской АСУ: Отсутствие сетевого доступа и блокировка всей базы на время записи делают SQLite непригодным для промышленной эксплуатации системы, обслуживающей запросы тысяч пользователей (согласно нефункциональному требованию №3).

- Ограниченная поддержка команды ALTER TABLE (например, для удаления столбца требуется пересоздание таблицы), что затруднит итеративное развитие схемы БД в будущем.

Выбор СУБД для создания базы данных проектируемой АСУ

На основании проведённого анализа в качестве СУБД для АСУ «Семантический подборщик» выбрана PostgreSQL.

Обоснование выбора:

1. Соответствие требованиям предметной области: ER-диаграмма содержит сущности с полями, которые идеально отображаются на типы JSONB.
2. Надёжность учёта операций: Процесс обработки ЗаявкаНаЗакупку и изменения статуса Лицензия требует строгой транзакционности, которую PostgreSQL обеспечивает на самом высоком уровне.
3. Единая экосистема: В качестве бэкенд-фреймворка выбран Python с библиотекой SQLAlchemy, которая имеет наиболее качественный и полный драйвер для работы с PostgreSQL (psycopg2/asyncpg).
4. Перспектива развития: Внедрение семантического поиска на основе векторных представлений (эмбеддингов) в будущем легко реализуется с помощью расширения pgvector, что позволит снизить зависимость от внешнего поискового движка для части запросов.

Создание базы данных в выбранной СУБД и заполнение ее данными

Листинг 1: Создание БД

```
DROP TABLE IF EXISTS "запросы" CASCADE;  
DROP TABLE IF EXISTS "сессии" CASCADE;  
DROP TABLE IF EXISTS "история" CASCADE;  
DROP TABLE IF EXISTS "логи" CASCADE;  
DROP TABLE IF EXISTS "уведомления" CASCADE;  
DROP TABLE IF EXISTS "нейросетевые_модели" CASCADE;  
DROP TABLE IF EXISTS "наборы_данных" CASCADE;  
DROP TABLE IF EXISTS "пользователи" CASCADE;
```

```

DROP TABLE IF EXISTS "избранное" CASCADE;
DROP TABLE IF EXISTS "отзывы" CASCADE;
DROP TABLE IF EXISTS "связь_контента_с_лицензиями" CASCADE;
DROP TABLE IF EXISTS "лицензии" CASCADE;
DROP TABLE IF EXISTS "связь_контента_с_категориями" CASCADE;
DROP TABLE IF EXISTS "категории" CASCADE;
DROP TABLE IF EXISTS "контент" CASCADE;
DROP TABLE IF EXISTS "заявки_на_закупку" CASCADE;
DROP TABLE IF EXISTS "внешние_источники" CASCADE;

CREATE TABLE "пользователи" (
    "идентификатор" BIGSERIAL PRIMARY KEY,
    "имя" VARCHAR(100) NOT NULL,
    "электронная_почта" VARCHAR(255) NOT NULL UNIQUE,
    "хеш_пароля" VARCHAR(255) NOT NULL,
    "дата_регистрации" DATE NOT NULL DEFAULT CURRENT_DATE,
    "роль" VARCHAR(50) NOT NULL DEFAULT 'пользователь',
    "статус" VARCHAR(50) NOT NULL DEFAULT 'активный',
    "телефон" VARCHAR(20) UNIQUE,
    "дата_создания" TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    "дата_обновления" TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE "сессии" (
    "идентификатор" BIGSERIAL PRIMARY KEY,
    "идентификатор_пользователя" BIGINT NOT NULL REFERENCES
"пользователи"("идентификатор") ON DELETE CASCADE,
    "время_начала" TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    "время_окончания" TIMESTAMP,
    "контекст" TEXT,
    "устройство" VARCHAR(100),
    "браузер" VARCHAR(100),
    "дата_создания" TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE "наборы_данных" (
    "идентификатор" BIGSERIAL PRIMARY KEY,
    "название" VARCHAR(100) NOT NULL,
    "описание" TEXT,
    "количество_записей" INTEGER DEFAULT 0,
    "дата_создания_набора" TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    "формат" VARCHAR(50) NOT NULL,
    "путь_к_файлу" TEXT NOT NULL,
    "статус" VARCHAR(50) NOT NULL DEFAULT 'на_рассмотрении',
    "источник_данных" VARCHAR(255),
    "дата_создания" TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE "нейросетевые_модели" (
    "идентификатор" BIGSERIAL PRIMARY KEY,
    "идентификатор_обучающего_набора" BIGINT REFERENCES
"наборы_данных"("идентификатор") ON DELETE SET NULL,
    "название" VARCHAR(100) NOT NULL,
    "версия" VARCHAR(50) NOT NULL,
    "дата_обучения" TIMESTAMP,
    "точность" DECIMAL(5,4),
    "путь_к_модели" TEXT NOT NULL,
    "статус" VARCHAR(50) NOT NULL DEFAULT 'неактивный',

```

```

        "гиперпараметры"          JSONB,
        "размер_модели"           BIGINT,
        "дата_создания"          TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
    );

CREATE TABLE "запросы" (
    "идентификатор"              BIGSERIAL PRIMARY KEY,
    "идентификатор_сессии"      BIGINT NOT NULL REFERENCES
"сессии"("идентификатор") ON DELETE CASCADE,
    "идентификатор_обработавшей_модель" BIGINT REFERENCES
"нейросетевые_модели"("идентификатор") ON DELETE SET NULL,
    "текст"                     TEXT NOT NULL,
    "время"                   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    "структурированные_параметры" JSONB,
    "статус"                   VARCHAR(50) NOT NULL DEFAULT
'на_рассмотрении',
    "источник_запроса"          VARCHAR(50) NOT NULL DEFAULT 'веб',
    "ip_адрес"                  INET,
    "дата_создания"            TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE "история" (
    "идентификатор"              BIGSERIAL PRIMARY KEY,
    "идентификатор_пользователя" BIGINT NOT NULL REFERENCES
"пользователи"("идентификатор") ON DELETE CASCADE,
    "тип_действия"              VARCHAR(50) NOT NULL,
    "время"                   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    "детали"                   TEXT,
    "длительность"              INTEGER, -- длительность в секундах, если применимо
    "дата_создания"            TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE "логи" (
    "идентификатор"              BIGSERIAL PRIMARY KEY,
    "идентификатор_пользователя" BIGINT REFERENCES "пользователи"("идентификатор")
ON DELETE SET NULL,
    "время"                   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    "уровень"                  VARCHAR(20) NOT NULL,
    "сообщение"                TEXT NOT NULL,
    "источник"                  VARCHAR(100) NOT NULL,
    "детали"                   JSONB,
    "дата_создания"            TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE "уведомления" (
    "идентификатор"              BIGSERIAL PRIMARY KEY,
    "идентификатор_пользователя" BIGINT NOT NULL REFERENCES
"пользователи"("идентификатор") ON DELETE CASCADE,
    "тип"                      VARCHAR(50) NOT NULL,
    "заголовок"                 VARCHAR(255) NOT NULL,
    "сообщение"                 TEXT NOT NULL,
    "дата"                     TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    "прочитано"                 BOOLEAN NOT NULL DEFAULT FALSE,
    "ссылка"                    VARCHAR(255),
    "дата_создания"            TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE "контент" (
    "идентификатор"              BIGSERIAL PRIMARY KEY,

```



```

        "название"                VARCHAR(255) NOT NULL,
        "тип"                     VARCHAR(50) NOT NULL,
        "авторы"                  TEXT,
        "описание"                TEXT,
        "уровень"                 VARCHAR(50),
        "ссылка_на_файл"          TEXT,
        "дата_добавления"          DATE NOT NULL DEFAULT CURRENT_DATE,
        "язык"                    VARCHAR(50),
        "средний_рейтинг"          DECIMAL(3,2) DEFAULT 0,
        "количество_просмотров"    INTEGER DEFAULT 0,
        "дата_создания"            TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
        "дата_обновления"          TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
    );

CREATE TABLE "категории" (
    "идентификатор"                BIGSERIAL PRIMARY KEY,
    "идентификатор_родительской_катег" BIGINT REFERENCES
"категории"("идентификатор") ON DELETE CASCADE,
    "название"                     VARCHAR(100) NOT NULL,
    "описание"                     TEXT,
    "дата_создания_категории"       DATE NOT NULL DEFAULT CURRENT_DATE,
    "количество_контента"           INTEGER DEFAULT 0,
    "иконка"                       VARCHAR(255),
    "дата_создания"                 TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE "связь_контента_с_категориями" (
    "идентификатор"                BIGSERIAL PRIMARY KEY,
    "идентификатор_контента"        BIGINT NOT NULL REFERENCES "контент"("идентификатор")
ON DELETE CASCADE,
    "идентификатор_категории"       BIGINT NOT NULL REFERENCES
"категории"("идентификатор") ON DELETE CASCADE
);

CREATE TABLE "лицензии" (
    "идентификатор"                BIGSERIAL PRIMARY KEY,
    "тип"                          VARCHAR(100) NOT NULL,
    "правообладатель"              VARCHAR(255),
    "дата_начала"                  DATE,
    "дата_окончания"              DATE,
    "стоимость"                    DECIMAL(10,2),
    "ограничения"                  TEXT,
    "условия_использования"        TEXT,
    "дата_создания"                TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE "связь_контента_с_лицензиями" (
    "идентификатор"                BIGSERIAL PRIMARY KEY,
    "идентификатор_контента"        BIGINT NOT NULL REFERENCES "контент"("идентификатор")
ON DELETE CASCADE,
    "идентификатор_лицензии"        BIGINT NOT NULL REFERENCES "лицензии"("идентификатор")
ON DELETE CASCADE
);

CREATE TABLE "отзывы" (
    "идентификатор"                BIGSERIAL PRIMARY KEY,
    "идентификатор_пользователя"    BIGINT NOT NULL REFERENCES
"пользователи"("идентификатор") ON DELETE CASCADE,
    "идентификатор_контента"        BIGINT NOT NULL REFERENCES

```

```

"контент"("идентификатор") ON DELETE CASCADE,
    "оценка"                INTEGER CHECK ("оценка" BETWEEN 1 AND 5),
    "текст"                  TEXT,
    "дата"                   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    "статус_модерации"       VARCHAR(50) NOT NULL DEFAULT 'на_рассмотрении',
    "полезность"              INTEGER DEFAULT 0,
    "жалобы"                  INTEGER DEFAULT 0,
    "дата_создания"          TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE "избранное" (
    "идентификатор"          BIGSERIAL PRIMARY KEY,
    "идентификатор_пользователя" BIGINT NOT NULL REFERENCES
"пользователи"("идентификатор") ON DELETE CASCADE,
    "идентификатор_контента" BIGINT NOT NULL REFERENCES
"контент"("идентификатор") ON DELETE CASCADE,
    "дата_добавления"        TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    "заметка"                 TEXT,
    "теги"                     VARCHAR(255),
    "приоритет"                INTEGER DEFAULT 0,
    "дата_создания"          TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE "внешние_источники" (
    "идентификатор"          BIGSERIAL PRIMARY KEY,
    "название"                VARCHAR(100) NOT NULL,
    "базовый_url"             VARCHAR(255) NOT NULL,
    "api_ключ"                 VARCHAR(255),
    "описание"                 TEXT,
    "статус"                   VARCHAR(50) NOT NULL DEFAULT 'активный',
    "параметры_подключения"    JSONB,
    "формат_данных"            VARCHAR(50) NOT NULL DEFAULT 'json',
    "дата_создания"           TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE "заявки_на_закупку" (
    "идентификатор"          BIGSERIAL PRIMARY KEY,
    "идентификатор_инициатора" BIGINT NOT NULL REFERENCES
"пользователи"("идентификатор") ON DELETE CASCADE,
    "идентификатор_контента" BIGINT REFERENCES "контент"("идентификатор")
ON DELETE SET NULL,
    "идентификатор_внешнего_источника" BIGINT REFERENCES
"внешние_источники"("идентификатор") ON DELETE SET NULL,
    "дата_создания_заявки"    TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    "статус"                   VARCHAR(50) NOT NULL DEFAULT
'на_рассмотрении',
    "бюджет"                   DECIMAL(10,2),
    "комментарии"              TEXT,
    "предлагаемый_контент_текст" TEXT NOT NULL,
    "обоснование"              TEXT,
    "приоритет"                 INTEGER DEFAULT 0,
    "дата_утверждения"        TIMESTAMP,
    "дата_создания"           TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

```

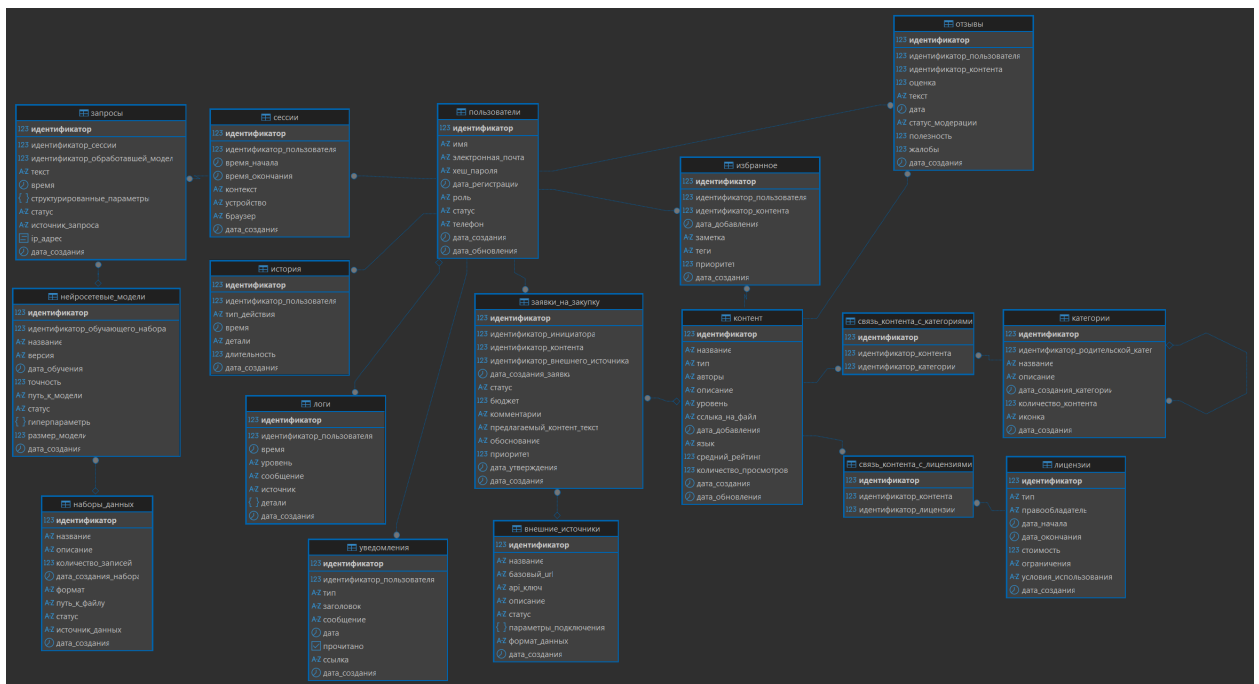


Рисунок 11 - Созданная БД

Листинг 2: Заполнение БД

```
INSERT INTO "пользователи" ("имя", "электронная_почта", "хеш_пароля",
"дата_регистрации", "роль", "статус", "телефон") VALUES
('Иван Петров', 'ivan@example.com', 'hash_ivan', '2025-01-15', 'пользователь',
'активный', '+79161234567'),
('Мария Смирнова', 'maria@example.com', 'hash_maria', '2025-02-01', 'пользователь',
'активный', '+79261234568'),
('Алексей Иванов', 'alexey@example.com', 'hash_alexey', '2025-02-10', 'контент-
менеджер', 'активный', '+79361234569'),
('Ольга Кузнецова', 'olga@example.com', 'hash_olga', '2025-03-05', 'администратор',
'активный', '+79461234570'),
('Даниил Враженко', 'student@rtu.ru', 'hash_student', '2025-03-20', 'инженер МО',
'активный', '+79561234571'),
('Елена Соколова', 'elena@example.com', 'hash_elena', '2025-04-01', 'пользователь',
'активный', '+79661234572'),
('Сергей Волков', 'sergey@example.com', 'hash_sergey', '2025-04-10', 'пользователь',
'заблокированный', '+79761234573'),
('Анна Морозова', 'anna@example.com', 'hash_anna', '2025-04-15', 'руководитель',
'активный', '+79861234574'),
('Павел Новиков', 'pavel@example.com', 'hash_pavel', '2025-05-01', 'пользователь',
'активный', '+79961234575'),
('Татьяна Зайцева', 'tatiana@example.com', 'hash_tatiana', '2025-05-10',
'пользователь', 'активный', '+79161234576'),
('Михаил Орлов', 'mikhail@example.com', 'hash_mikhail', '2025-06-01', 'контент-
менеджер', 'активный', '+79261234577');

INSERT INTO "сессии" ("идентификатор_пользователя", "время_начала",
"время_окончания", "устройство", "браузер", "контекст") VALUES
(1, '2026-04-01 09:00:00', '2026-04-01 09:30:00', 'Windows PC', 'Chrome 120',
'{"последний_запрос": "изучение Python"}'),
(1, '2026-04-02 14:00:00', NULL, 'iPhone', 'Safari', '{"последний_запрос": "книги по
машинному обучению"}'),
(2, '2026-04-01 10:00:00', '2026-04-01 10:45:00', 'MacBook', 'Firefox',
```

```

'{"последний_запрос": "аудиокниги по психологии"}'),
(3, '2026-04-02 08:00:00', '2026-04-02 12:00:00', 'Windows PC', 'Edge', '{"фильтры":
{"категория": "бизнес"}}'),
(4, '2026-04-03 11:00:00', '2026-04-03 11:20:00', 'Android', 'Chrome Mobile', '{}'),
(5, '2026-04-01 13:00:00', NULL, 'Linux', 'Chrome', '{"отладка": true}'),
(6, '2026-04-04 16:00:00', '2026-04-04 17:00:00', 'iPad', 'Safari',
'{"последний_запрос": "курсы по Go"}'),
(7, '2026-04-05 19:00:00', '2026-04-05 19:15:00', 'Windows PC', 'Opera', '{}'),
(8, '2026-04-06 09:00:00', '2026-04-06 09:30:00', 'MacBook', 'Safari', '{}'),
(9, '2026-04-07 21:00:00', '2026-04-07 22:00:00', 'Android', 'Samsung Internet',
'{"последний_запрос": "подкасты про технологии"}'),
(10, '2026-04-08 07:30:00', NULL, 'iPhone', 'Safari', '{}'),
(1, '2026-04-09 12:00:00', '2026-04-09 13:00:00', 'Windows PC', 'Chrome',
'{"последний_запрос": "видеоуроки по SQL"}');

INSERT INTO "наборы_данных" ("название", "описание", "количество_записей", "формат",
"путь_к_файлу", "статус", "источник_данных") VALUES
('Queries v1', 'Начальный набор размеченных запросов', 1500, 'CSV',
'/data/queries_v1.csv', 'готов', 'внутренний'),
('User logs March', 'Обезличенные логи за март 2026', 25000, 'JSON',
'/data/logs_2026_03.json', 'обрабатывается', 'логи'),
('External Queries', 'Запросы из открытых источников', 5000, 'CSV',
'/data/external_queries.csv', 'готов', 'внешний'),
('RuBERT pretrain', 'Предобученный корпус русского языка', 1000000, 'BIN',
'/models/rubert_pretrain.bin', 'готов', 'публичный'),
('Manual labels', 'Ручная разметка 200 запросов', 200, 'CSV',
'/data/manual_labels.csv', 'готов', 'внутренний'),
('Feedback data', 'Данные обратной связи от пользователей', 450, 'JSON',
'/data/feedback.json', 'на_рассмотрении', 'обратная_связь'),
('Test dataset', 'Тестовый набор для валидации', 300, 'CSV', '/data/test_set.csv',
'готов', 'внутренний'),
('Train dataset large', 'Большой тренировочный набор', 8000, 'Parquet',
'/data/train_large.parquet', 'готов', 'внутренний'),
('External News', 'Заголовки новостей для расширения словаря', 12000, 'TXT',
'/data/news_titles.txt', 'готов', 'внешний'),
('Domain specific', 'Запросы в предметной области IT', 2200, 'CSV',
'/data/domain_it.csv', 'готов', 'внутренний'),
('Validation set', 'Валидационная выборка', 500, 'CSV', '/data/val_set.csv',
'готов', 'внутренний');

INSERT INTO "нейросетевые_модели" ("идентификатор_обучающего_набора", "название",
"версия", "дата_обучения", "точность", "путь_к_модели", "статус", "гиперпараметры",
"размер_модели") VALUES
(1, 'RuBERT-tiny-intent', '1.0.0', '2026-03-01 10:00:00', 0.8542,
'/models/rubert_tiny_intent_v1.pth', 'активная', '{"скорость_обучения": 2e-5,
"эпохи": 3, "размер_пакета": 16}', 45000000),
(2, 'RuBERT-tiny-intent', '1.1.0', '2026-03-15 14:30:00', 0.8715,
'/models/rubert_tiny_intent_v1.1.pth', 'активная', '{"скорость_обучения": 1e-5,
"эпохи": 5, "размер_пакета": 16}', 45000000),
(3, 'DistilRuBERT', '2.0.0', '2026-02-20 09:00:00', 0.8920,
'/models/distilrubert_v2.pth', 'активная', '{"скорость_обучения": 3e-5, "эпохи": 4,
"размер_пакета": 32}', 67000000),
(4, 'LLaMA-7B-finetuned', '0.9.0', '2026-04-01 08:00:00', 0.9125,
'/models/llama7b_qa.pth', 'тестируется', '{"скорость_обучения": 5e-6, "эпохи": 2,
"размер_пакета": 8}', 7000000000),
(NULL, 'Baseline TF-IDF', '1.0.0', '2026-01-10 12:00:00', 0.7200,
'/models/tfidf_baseline.pkl', 'архивная', '{"диапазон_n_грамм": [1,2]}', 1200000),
(5, 'Intent classifier CNN', '1.2.0', '2026-03-20 11:00:00', 0.8350,
'/models/cnn_intent.pth', 'неактивная', '{"фильтры": 128, "размер_ядра": 3}',

```

```

2500000),
(6, 'Sentiment model', '1.0.0', '2026-02-01 10:00:00', 0.7810,
'/models/sentiment_v1.pth', 'активная', '{"скорость_обучения": 2e-5}', 45000000),
(7, 'Entity recognition', '1.0.1', '2026-03-10 15:00:00', 0.9230,
'/models/ner_rubert.pth', 'активная', '{"макс_длина": 256}', 45000000),
(8, 'Next-query predictor', '0.5.0', '2026-04-05 09:00:00', 0.6510,
'/models/next_query_lstm.pth', 'в_разработке', '{"размер_скрытого_слоя": 256,
"количество_слоёв": 2}', 3500000),
(1, 'RuBERT-base', '1.0.0', '2026-02-15 16:00:00', 0.8810,
'/models/rubert_base_intent.pth', 'активная', '{"скорость_обучения": 2e-5, "эпохи":
3}', 110000000),
(9, 'FastText embeddings', '1.0.0', '2026-01-20 10:00:00', 0.7600,
'/models/fasttext_ru.bin', 'активная', '{"размерность": 300}', 800000000);

```

```

INSERT INTO "запросы" ("идентификатор_сессии", "идентификатор_обработавшей_модель",
"текст", "время", "структурированные_параметры", "статус", "источник_запроса",
"ip_адрес") VALUES

```

```

(1, 1, 'хочу изучить Python', '2026-04-01 09:05:00', '{"тема": "Python",
"намерение": "изучить", "уровень": "начальный"}', 'обработан', 'веб',
'192.168.1.10'),
(1, 1, 'книги по машинному обучению для начинающих', '2026-04-01 09:10:00',
'{"тема": "машинное обучение", "намерение": "найти", "уровень": "начальный",
"тип_контента": "книга"}', 'обработан', 'веб', '192.168.1.10'),
(2, 2, 'курсы по Go для профессионалов', '2026-04-02 14:05:00', '{"тема": "Go",
"намерение": "найти", "уровень": "продвинутый", "тип_контента": "курс"}',
'обработан', 'мобильное', '10.0.0.23'),
(3, 3, 'аудиокниги по психологии отношений', '2026-04-01 10:10:00', '{"тема":
"психология", "намерение": "найти", "тип_контента": "аудиокнига"}', 'обработан',
'веб', '172.16.0.5'),
(4, 4, 'как управлять командой в стартапе', '2026-04-03 11:05:00', '{"тема":
"управление", "намерение": "изучить", "контекст": "startup"}', 'обработан', 'веб',
'192.168.2.50'),
(5, 1, 'продвинутый Python для анализа данных', '2026-04-01 13:10:00', '{"тема":
"Python", "намерение": "изучить", "уровень": "продвинутый", "область": "data
analysis"}', 'обработан', 'веб', '10.1.1.8'),
(6, 7, 'где скачать книгу Чистый код', '2026-04-04 16:15:00', '{"тема": "Clean
Code", "намерение": "скачать", "тип_контента": "книга"}', 'обработан', 'мобильное',
'172.20.0.12'),
(7, 2, 'видео про микросервисы', '2026-04-05 19:05:00', '{"тема": "микросервисы",
"намерение": "найти", "тип_контента": "видео"}', 'ошибка', 'веб', '192.168.3.77'),
(8, 9, 'как стать data scientist с нуля', '2026-04-06 09:10:00', '{"тема": "наука о
данных", "намерение": "изучить", "уровень": "начальный"}', 'обработан', 'веб',
'10.0.1.100'),
(9, 8, 'подкасты про IT и стартапы', '2026-04-07 21:15:00', '{"тема": "IT",
"намерение": "найти", "тип_контента": "подкаст"}', 'обработан', 'мобильное',
'192.168.4.55'),
(10, 1, 'SQL для начинающих', '2026-04-08 07:45:00', '{"тема": "SQL", "намерение":
"изучить", "уровень": "начальный"}', 'обработан', 'веб', '10.0.2.200'),
(12, 3, 'Python Django разработка', '2026-04-09 12:10:00', '{"тема": "Django",
"намерение": "изучить", "фреймворк": "Django"}', 'обработан', 'веб',
'192.168.1.11');

```

```

INSERT INTO "история" ("идентификатор_пользователя", "тип_действия", "время",
"детали", "длительность") VALUES

```

```

(1, 'поиск', '2026-04-01 09:05:00', 'запрос: "хочу изучить Python"', 2),
(1, 'просмотр_контента', '2026-04-01 09:12:00', 'идентификатор_контента: 101 (Python
для начинающих)', 120),
(2, 'поиск', '2026-04-01 10:10:00', 'запрос: "аудиокниги по психологии"', 3),
(3, 'вход', '2026-04-02 08:00:00', 'с ip 192.168.1.20', NULL),

```

```
(3, 'создание_заявки_на_закупку', '2026-04-02 09:30:00', 'идентификатор_заявки: 5001
на книгу "Чистый код"', NULL),
(4, 'поиск', '2026-04-03 11:05:00', 'запрос: "управление командой"', 1),
(5, 'запуск_обучения_модели', '2026-04-01 13:30:00', 'идентификатор_модели: 2,
идентификатор_набора_данных: 2', NULL),
(6, 'поиск', '2026-04-04 16:15:00', 'запрос: "скачать книгу Чистый код"', 2),
(6, 'скачивание', '2026-04-04 16:30:00', 'идентификатор_контента: 105 (Clean Code
pdf)', 10),
(7, 'поиск', '2026-04-05 19:05:00', 'запрос: "микросервисы"', 1),
(9, 'прослушивание_подкаста', '2026-04-07 21:30:00', 'идентификатор_эпизода: 301,
длительность: 1800', 1800),
(10, 'поиск', '2026-04-08 07:45:00', 'запрос: "SQL для начинающих"', 2),
(1, 'добавление_в_избранное', '2026-04-09 12:30:00', 'идентификатор_контента: 101',
NULL);
```

```
INSERT INTO "логи" ("идентификатор_пользователя", "время", "уровень", "сообщение",
"источник", "детали") VALUES
(1, '2026-04-01 09:05:00', 'ИНФОРМАЦИЯ', 'Получен поисковый запрос пользователя',
'api-gateway', '{"эндпоинт": "/search", "метод": "POST"}'),
(NULL, '2026-04-01 09:05:01', 'ОТЛАДКА', 'Запущен инференс NLP-модели', 'nlp-
service', '{"идентификатор_модели": 1, "длина_ввода": 25}'),
(1, '2026-04-01 09:05:02', 'ИНФОРМАЦИЯ', 'Поиск завершён', 'search-service',
 '{"количество_результатов": 12, "время_мс": 180}'),
(5, '2026-04-01 13:30:00', 'ИНФОРМАЦИЯ', 'Отправлена задача на обучение модели',
'ml-orchestrator', '{"идентификатор_задачи": "train-20260401-001",
"идентификатор_набора_данных": 2}'),
(5, '2026-04-01 14:00:00', 'ОШИБКА', 'Ошибка обучения: нехватка памяти', 'ml-
orchestrator', '{"ошибка": "CUDA out of memory", "видеокарта": "Tesla T4"}'),
(3, '2026-04-02 09:30:00', 'ПРЕДУПРЕЖДЕНИЕ', 'Заявка на закупку превышает лимит
бюджета', 'purchase-service', '{"идентификатор_набора_заявки": 5001, "сумма": 1500,
"лимит": 1000}'),
(NULL, '2026-04-04 16:15:00', 'ИНФОРМАЦИЯ', 'Вызов внешнего API книжного магазина',
'external-adapter', '{"api": "litres", "время_ответа_мс": 320}'),
(7, '2026-04-05 19:05:01', 'ОШИБКА', 'Ошибка обработки запроса: таймаут модели',
'api-gateway', '{"идентификатор_модели": 2, "таймаут": 2000}'),
(8, '2026-04-06 09:10:00', 'ИНФОРМАЦИЯ', 'Регистрация пользователя завершена',
'auth-service', '{"идентификатор_пользователя": 11, "электронная_почта":
"newuser@example.com"}'),
(NULL, '2026-04-07 21:00:00', 'ИНФОРМАЦИЯ', 'Запущено ежедневное резервное
копирование', 'backup-service', '{"тип": "full", "размер_гб": 45}'),
(4, '2026-04-08 07:45:00', 'ОТЛАДКА', 'Попадание в кэш для запроса "SQL"', 'search-
service', '{"ключ_кэша": "sql:beginner", "попаданий": 3}'),
(NULL, '2026-04-09 12:00:00', 'ПРЕДУПРЕЖДЕНИЕ', 'Высокая загрузка ЦП на сервере БД',
'monitoring', '{"загрузка_цп_проценты": 95, "длительность_сек": 60}');
```

```
INSERT INTO "уведомления" ("идентификатор_пользователя", "тип", "заголовок",
"сообщение", "дата", "прочитано", "ссылка") VALUES
(1, 'новый_контент', 'Новые курсы по Python', 'По вашему запросу "Python для
начинающих" добавлено 3 новых курса.', '2026-04-01 10:00:00', false,
'/courses/python'),
(1, 'рекомендация', 'Вам может понравиться', 'Книга "Грокаем алгоритмы"', '2026-04-
02 08:00:00', true, '/content/201'),
(2, 'статус_закупки', 'Заявка на закупку', 'Ваша заявка на книгу "Психология
влияния" одобрена.', '2026-04-01 11:30:00', false, '/purchases/5001'),
(3, 'назначена_задача', 'Новая задача', 'Вам назначена задача на модерацию 5 новых
отзывов.', '2026-04-02 09:00:00', false, '/moderation'),
(5, 'модель_обучена', 'Модель обучена', 'Дообучение модели RuBERT-tiny-intent v1.1
завершено. Точность: 87.15%', '2026-04-01 15:00:00', true, '/ml/models/2'),
(6, 'новый_контент', 'Обновление подкастов', 'Новый выпуск подкаста "Люди и код"',
```



```

'2026-04-04 09:00:00', false, '/podcasts/305'),
(8, 'система', 'Плановое обслуживание', 'Завтра с 02:00 до 04:00 возможны перебои в
работе.', '2026-04-05 18:00:00', true, NULL),
(9, 'рассылка', 'Дайджест за неделю', 'Самые популярные материалы: Python,
микросервисы, управление.', '2026-04-07 20:00:00', false, '/digest/week15'),
(10, 'приветствие', 'Добро пожаловать!', 'Спасибо за регистрацию в системе
"Семантический подборщик".', '2026-04-08 07:35:00', false, '/help'),
(4, 'предупреждение', 'Превышение лимита запросов', 'Количество запросов к API
превысило 80% дневного лимита.', '2026-04-09 14:00:00', false, '/admin/limits'),
(1, 'ответ_на_комментарий', 'Ответ на ваш отзыв', 'Контент-менеджер ответил на ваш
отзыв о курсе Python.', '2026-04-10 09:00:00', false, '/content/101#comments');

INSERT INTO "контент" ("название", "тип", "авторы", "описание", "уровень",
"ссылка_на_файл", "дата_добавления", "язык", "средний_рейтинг",
"количество_просмотров") VALUES
('Программирование на Python для начинающих', 'книга', 'Марк Лутц', 'Изучение Python
с нуля', 'начальный', '/files/python_book.pdf', '2025-06-01', 'ru', 4.5, 1250),
('Машинное обучение: алгоритмы и практика', 'видео', 'Андрей Себрант', 'Курс по
основам ML', 'средний', '/videos/ml_course.mp4', '2025-07-12', 'ru', 4.8, 3420),
('Аудиокнига "Чистый код"', 'аудио', 'Роберт Мартин', 'Принципы написания
качественного кода', 'продвинутый', '/audio/clean_code.mp3', '2025-05-23', 'ru',
4.9, 890),
('Основы SQL и реляционных баз данных', 'курс', 'Иван Петров', 'Интерактивный курс
по SQL', 'начальный', '/courses/sql_basics', '2025-08-01', 'ru', 4.3, 560),
('Deep Learning with PyTorch', 'книга', 'Eli Stevens et al.', 'Глубокое обучение на
PyTorch', 'продвинутый', '/files/dl_pytorch.pdf', '2025-09-10', 'en', 4.7, 320),
('Психология влияния', 'аудио', 'Роберт Чалдини', 'Классика социальной психологии',
'начальный', '/audio/influence.mp3', '2025-04-15', 'ru', 4.6, 2150),
('React для профессионалов', 'видео', 'Максим Иванов', 'Продвинутые техники React',
'продвинутый', '/videos/react_pro.mp4', '2025-10-01', 'ru', 4.4, 780),
('Английский для IT', 'курс', 'LinguaLeo', 'Курс технического английского',
'средний', '/courses/english_it', '2025-03-20', 'ru', 4.2, 1430),
('Микросервисы: паттерны и практики', 'книга', 'Крис Ричардсон', 'Архитектура
микросервисов', 'продвинутый', '/files/microservices.pdf', '2025-11-05', 'ru', 4.9,
290),
('Подкаст "Люди и код" выпуск 45', 'подкаст', 'Алексей Котов', 'Интервью с
разработчиками', 'начальный', '/podcasts/people_and_code_45.mp3', '2025-12-01',
'ru', 4.0, 520),
('Data Science с нуля', 'книга', 'Джоэл Грас', 'Введение в Data Science на Python',
'начальный', '/files/data_science.pdf', '2025-02-14', 'ru', 4.5, 980),
('Архитектура компьютера', 'видео', 'Эндрю Таненбаум', 'Лекции по архитектуре ЭВМ',
'средний', '/videos/comp_arch.mp4', '2025-01-10', 'ru', 4.1, 650);

INSERT INTO "категории" ("идентификатор_родительской_катег", "название", "описание",
"дата_создания_категории", "иконка") VALUES
(NULL, 'Программирование', 'Языки и технологии разработки', '2025-01-01',
'/icons/programming.png'),
(1, 'Python', 'Язык Python и его экосистема', '2025-01-02', '/icons/python.png'),
(1, 'Базы данных', 'SQL, NoSQL, проектирование', '2025-01-03',
'/icons/database.png'),
(NULL, 'Наука о данных', 'Data Science, ML, AI', '2025-01-04',
'/icons/data_science.png'),
(4, 'Машинное обучение', 'Алгоритмы и библиотеки ML', '2025-01-05',
'/icons/ml.png'),
(4, 'Глубокое обучение', 'Нейронные сети и DL', '2025-01-06',
'/icons/deeplearning.png'),
(NULL, 'Бизнес и психология', 'Книги по саморазвитию и бизнесу', '2025-01-07',
'/icons/business.png'),
(7, 'Психология', 'Научная и популярная психология', '2025-01-08',

```

```

'/icons/psychology.png'),
(NULL, 'Иностранные языки', 'Курсы и материалы по языкам', '2025-01-09',
'/icons/languages.png'),
(9, 'Английский', 'Английский для разных целей', '2025-01-10',
'/icons/english.png'),
(NULL, 'Аудио', 'Аудиокниги и подкасты', '2025-01-11', '/icons/audio.png'),
(1, 'JavaScript', 'Язык JavaScript и фреймворки', '2025-02-01', '/icons/js.png');

INSERT INTO "связь_контента_с_категориями" ("идентификатор_контента",
"идентификатор_категории") VALUES
(1, 2), (1, 1), (2, 5), (2, 4), (3, 1), (4, 3), (5, 6), (6, 8), (7, 1), (7, 12), (8,
10), (9, 1), (10, 11), (11, 4), (12, 1);

INSERT INTO "лицензии" ("тип", "правообладатель", "дата_начала", "дата_окончания",
"стоимость", "ограничения", "условия_использования") VALUES
('Эксклюзивная', 'Издательство "Питер"', '2025-01-01', '2027-01-01', 15000.00,
'Только для подписчиков', 'Полный доступ к книгам издательства'),
('Неэксклюзивная', 'ЛитРес', '2025-02-01', '2026-02-01', 5000.00, 'Не более 100
скачиваний в месяц', 'Аудиокниги по подписке'),
('Свободная', 'Автор', '2025-03-01', NULL, 0.00, 'С указанием авторства', 'Свободное
распространение'),
('Образовательная', 'Coursera', '2025-04-01', '2026-04-01', 25000.00, 'Только для
зарегистрированных студентов', 'Доступ к курсам платформы'),
('Корпоративная', 'Skillbox', '2025-05-01', '2026-05-01', 50000.00, 'До 1000
сотрудников', 'Корпоративная подписка'),
('Бесплатная', 'Open Source Community', '2025-06-01', NULL, 0.00, 'Без ограничений',
'Свободное ПО'),
('Пробная', 'Udemy', '2025-07-01', '2025-08-01', 1000.00, 'Пробный период 1 месяц',
'Ограниченный доступ'),
('Подписка', 'OReilly', '2025-08-01', '2026-08-01', 30000.00, 'Ежемесячная оплата',
'Полный каталог OReilly'),
('Индивидуальная', 'Автор', '2025-09-01', NULL, 500.00, 'Один пользователь',
'Пожизненная лицензия'),
('Академическая', 'MIT Press', '2025-10-01', '2027-10-01', 8000.00, 'Только для
вузов', 'Академическая подписка'),
('Разработчик', 'JetBrains', '2025-11-01', '2026-11-01', 12000.00, 'Для
индивидуальных разработчиков', 'Доступ к инструментам'),
('Корпоративная', 'Microsoft', '2025-12-01', '2027-12-01', 100000.00, 'Для
организаций', 'Корпоративный доступ');

INSERT INTO "связь_контента_с_лицензиями" ("идентификатор_контента",
"идентификатор_лицензии") VALUES
(1, 1), (2, 4), (3, 2), (4, 3), (5, 1), (6, 2), (7, 8), (8, 4), (9, 1), (10, 6),
(11, 9), (12, 3);

INSERT INTO "отзывы" ("идентификатор_пользователя", "идентификатор_контента",
"оценка", "текст", "дата", "статус_модерации", "полезность", "жалобы") VALUES
(1, 1, 5, 'Отличная книга для начинающих, всё понятно.', '2025-06-15 10:30:00',
'одобрена', 12, 0),
(2, 2, 5, 'Курс очень понравился, много практики.', '2025-07-20 14:20:00',
'одобрена', 8, 1),
(3, 3, 4, 'Хорошая озвучка, но хотелось бы больше примеров.', '2025-06-01 09:15:00',
'одобрена', 5, 2),
(4, 4, 4, 'Базовый курс, всё по делу.', '2025-08-10 16:40:00', 'одобрена', 6, 0),
(5, 5, 5, 'Превосходная книга, рекомендую всем DS-специалистам.', '2025-09-20
11:05:00', 'одобрена', 9, 0),
(6, 6, 3, 'Книга неплохая, но несколько устарела.', '2025-05-01 13:25:00',
'одобрена', 3, 1),
(7, 7, 5, 'Лучший курс по React, который я видел.', '2025-10-15 18:00:00',

```



```

'одобрена', 15, 0),
(8, 8, 4, 'Полезно для расширения словарного запаса.', '2025-04-10 08:30:00',
'одобрена', 7, 0),
(9, 9, 5, 'Книга-библия по микросервисам.', '2025-11-20 12:10:00', 'одобрена', 11,
0),
(10, 10, 4, 'Интересный выпуск, жду следующих.', '2025-12-05 17:45:00', 'одобрена',
4, 0),
(1, 11, 4, 'Неплохое введение в DS.', '2025-03-01 10:00:00', 'одобрена', 5, 0),
(2, 12, 3, 'Слишком академично, тяжело воспринимать.', '2025-02-15 11:20:00',
'одобрена', 2, 3);

INSERT INTO "избранное" ("идентификатор_пользователя", "идентификатор_контента",
"дата_добавления", "заметка", "теги", "приоритет") VALUES
(1, 1, '2025-06-16 09:00:00', 'Изучаю Python', 'python, book', 1),
(1, 5, '2025-09-21 10:00:00', 'Глубокое обучение', 'dl, pytorch', 2),
(2, 2, '2025-07-21 11:30:00', 'Машинное обучение', 'ml, course', 1),
(3, 3, '2025-06-02 14:00:00', 'Для работы', 'clean code, audio', 1),
(4, 4, '2025-08-11 08:00:00', 'Повторение SQL', 'sql, database', 2),
(5, 5, '2025-09-22 09:15:00', 'Для диссертации', 'deep learning', 1),
(6, 7, '2025-10-16 20:00:00', 'React advanced', 'react, frontend', 1),
(7, 7, '2025-10-17 12:00:00', 'Очень полезно', 'react', 2),
(8, 8, '2025-04-11 10:00:00', 'Английский для собеседований', 'english, it', 1),
(9, 9, '2025-11-21 15:20:00', 'Микросервисы', 'microservices, architecture', 1),
(10, 10, '2025-12-06 19:00:00', 'Любимый подкаст', 'podcast, dev', 1),
(1, 11, '2025-03-02 09:00:00', 'Data Science intro', 'data science, book', 3);

INSERT INTO "внешние_источники" ("название", "базовый_url", "api_ключ", "описание",
"статус", "параметры_подключения", "формат_данных") VALUES
('ЛитРес API', 'https://api.litres.ru', 'litres_key_123', 'Электронные и
аудиокниги', 'активный', '{"таймаут": 30, "лимит_запросов": 100}', 'json'),
('Coursera Partner', 'https://api.coursera.org', 'coursera_partner_456',
'Образовательные курсы', 'активный', '{"тип_аутентификации": "oauth2"}', 'json'),
('OReilly Online', 'https://api.oreilly.com', 'oreilly_789', 'Техническая
литература', 'активный', '{"повторы": 3}', 'xml'),
('YouTube Education', 'https://www.googleapis.com/youtube/v3', 'yt_edu_key',
'Образовательные видео', 'активный', '{"квота": 10000}', 'json'),
('Audible API', 'https://api.audible.com', 'audible_key_321', 'Аудиокниги',
'активный', '{"регион": "RU"}', 'json'),
('Stepik API', 'https://stepik.org/api', NULL, 'Онлайн-курсы (публичный доступ)',
'активный', '{}', 'json'),
('Национальная электронная библиотека', 'https://rusneb.ru/api', 'neb_key_555',
'Книги и документы', 'неактивный', '{"тип_аутентификации": "basic"}', 'json'),
('Packt Publishing', 'https://api.packt.com', 'packt_key_999', 'Книги по IT',
'активный', '{}', 'json'),
('MIT OpenCourseWare', 'https://ocw.mit.edu/api', NULL, 'Открытые курсы MIT',
'активный', '{}', 'xml'),
('SpringerLink', 'https://api.springernature.com', 'springer_123', 'Научные
публикации', 'активный', '{"лимит_журналов": 500}', 'json'),
('Safari Books Online', 'https://api.safaribooksonline.com', 'safari_legacy',
'Устаревший API (заменён на OReilly)', 'архивный', '{}', 'json');

INSERT INTO "заявки_на_закупку" ("идентификатор_инициатора",
"идентификатор_контента", "идентификатор_внешнего_источника",
"дата_создания_заявки", "статус", "бюджет", "комментарии",
"предлагаемый_контент_текст", "обоснование", "приоритет") VALUES
(1, NULL, 1, '2026-04-01 10:00:00', 'на_рассмотрении', 1500.00, 'Книга нужна для
команды разработки', 'Clean Code: A Handbook of Agile Software Craftsmanship',
'Высокий спрос среди разработчиков', 2),
(2, 3, 5, '2026-04-02 11:30:00', 'одобрена', 800.00, NULL, 'Аудиокнига "Чистый

```

```
код", 'Расширение аудиокаталога', 3),
(3, NULL, 1, '2026-04-03 09:15:00', 'на_рассмотрении', 2500.00, 'Для библиотеки
компании', 'Design Patterns: Elements of Reusable Object-Oriented Software',
'Классика, рекомендуется для архитекторов', 1),
(4, NULL, 6, '2026-04-04 14:20:00', 'отклонена', 0.00, 'Бесплатный курс, закупка не
требуется', 'Курс "Основы Python" на Stepik', 'Достаточно внутренних материалов',
1),
(5, 5, 3, '2026-04-05 08:45:00', 'одобрена', 1200.00, NULL, 'Deep Learning with
PyTorch (печатное издание)', 'Для отдела Data Science', 2),
(6, NULL, 2, '2026-04-06 16:10:00', 'одобрена', 3500.00, 'Корпоративная лицензия',
'Machine Learning Specialization (Andrew Ng)', 'Повышение квалификации сотрудников',
2),
(7, 7, 4, '2026-04-07 12:00:00', 'на_рассмотрении', 600.00, 'Видеокурс по React',
'React - The Complete Guide (incl Hooks, React Router, Redux)', 'Актуально для
фронтенд-команды', 1),
(8, NULL, 8, '2026-04-08 10:30:00', 'одобрена', 900.00, NULL, 'Python Crash Course,
2nd Edition', 'Для новых сотрудников', 2),
(9, 9, 3, '2026-04-09 13:45:00', 'на_рассмотрении', 1800.00, NULL, 'Building
Microservices, 2nd Edition', 'Обновление библиотеки архитектуры', 2),
(10, NULL, 7, '2026-04-10 09:00:00', 'отклонена', 5000.00, 'Слишком высокая
стоимость', 'Редкое издание "История вычислительной техники"', 'Не соответствует
профилю', 1),
(11, 11, 8, '2026-04-11 11:20:00', 'одобрена', 700.00, 'Электронная версия', 'Data
Science from Scratch, 2nd Edition', 'Базовый учебник', 3),
(1, 12, 9, '2026-04-12 15:30:00', 'на_рассмотрении', 0.00, 'Открытый курс',
'Computer System Architecture (MIT OCW)', 'Бесплатный ресурс, требуется интеграция',
1);
```

	123	идентификатор	AZ имя	AZ электронная_почта	AZ хеш_пароля	data_регистрации	AZ роль	AZ статус	AZ телефон	data_создания	data_обновления
1	1	Иван Петров	ivan@example.com	hash_ivan	2025-01-15	пользователь	активный	+79161234567	2026-04-18 13:07:23.856	2026-04-18 13:07:23.856	
2	2	Мария Смирнова	maria@example.com	hash_maria	2025-02-01	пользователь	активный	+79261234568	2026-04-18 13:07:23.856	2026-04-18 13:07:23.856	
3	3	Алексей Иванов	alexey@example.com	hash_alexey	2025-02-10	контент-менеджер	активный	+79361234569	2026-04-18 13:07:23.856	2026-04-18 13:07:23.856	
4	4	Ольга Кузнецова	olga@example.com	hash_olga	2025-03-05	администратор	активный	+79461234570	2026-04-18 13:07:23.856	2026-04-18 13:07:23.856	
5	5	Даниил Вразженко	student@rt.ru	hash_student	2025-03-20	инженер МО	активный	+79561234571	2026-04-18 13:07:23.856	2026-04-18 13:07:23.856	
6	6	Елена Соколова	elena@example.com	hash_elena	2025-04-01	пользователь	активный	+79661234572	2026-04-18 13:07:23.856	2026-04-18 13:07:23.856	
7	7	Сергей Волков	sergey@example.com	hash_sergey	2025-04-10	пользователь	заблокированный	+79761234573	2026-04-18 13:07:23.856	2026-04-18 13:07:23.856	
8	8	Анна Морозова	anna@example.com	hash_anna	2025-04-15	руководитель	активный	+79861234574	2026-04-18 13:07:23.856	2026-04-18 13:07:23.856	
9	9	Павел Новиков	pavel@example.com	hash_pavel	2025-05-01	пользователь	активный	+79961234575	2026-04-18 13:07:23.856	2026-04-18 13:07:23.856	
10	10	Татьяна Зайцева	tatiana@example.com	hash_tatiana	2025-05-10	пользователь	активный	+79161234576	2026-04-18 13:07:23.856	2026-04-18 13:07:23.856	
11	11	Михаил Орлов	mikhail@example.com	hash_mikhail	2025-06-01	контент-менеджер	активный	+79261234577	2026-04-18 13:07:23.856	2026-04-18 13:07:23.856	

Рисунок 12 - Заполненная таблица «пользователи»

Листинг 3: Запрос

```
WITH "статистика_категорий" AS (
    SELECT
        "кат"."идентификатор" AS "идентификатор_категории",
        "кат"."название" AS "категория",
        COALESCE(SUM("конт"."количество_просмотров"), 0) AS "суммарные_просмотры",
        COALESCE(AVG("конт"."средний_рейтинг"), 0) AS "средний_рейтинг",
        COUNT(DISTINCT "конт"."идентификатор") AS "количество_контента",
        COUNT(DISTINCT "отз"."идентификатор") AS "количество_отзывов"
    FROM "категории" "кат"
    LEFT JOIN "связь_контента_с_категориями" "сск" ON "кат"."идентификатор" =
"сск"."идентификатор_категории"
    LEFT JOIN "контент" "конт" ON "сск"."идентификатор_контента" =
"конт"."идентификатор"
    LEFT JOIN "отзывы" "отз" ON "конт"."идентификатор" =
"отз"."идентификатор_контента" AND "отз"."статус_модерации" = 'одобрена'
    GROUP BY "кат"."идентификатор", "кат"."название"
),
"статистика_закупок" AS (
```

<pre> SELECT "кат"."идентификатор" AS "идентификатор_категории", COALESCE(SUM("заяв"."бюджет"), 0) AS "бюджет_одобренных_закупок" FROM "категории" "кат" LEFT JOIN "связь_контента_с_категориями" "сск" ON "кат"."идентификатор" = "сск"."идентификатор_категории" LEFT JOIN "контент" "конт" ON "сск"."идентификатор_контента" = "конт"."идентификатор" LEFT JOIN "заявки_на_закупку" "заяв" ON "заяв"."идентификатор_контента" = "конт"."идентификатор" AND "заяв"."статус" = 'одобрена' GROUP BY "кат"."идентификатор") SELECT "ск"."категория", "ск"."суммарные_просмотры", ROUND("ск"."средний_рейтинг", 2) AS "средний_рейтинг", "ск"."количество_контента", "ск"."количество_отзывов", COALESCE("сз"."бюджет_одобренных_закупок", 0) AS "бюджет_одобренных_закупок" FROM "статистика_категорий" "ск" LEFT JOIN "статистика_закупок" "сз" ON "ск"."идентификатор_категории" = "сз"."идентификатор_категории" ORDER BY "ск"."суммарные_просмотры" DESC LIMIT 5; </pre>						
	AZ категория	123 суммарные_просмотры	123 средний_рейтинг	123 количество_контента	123 количество_отзывов	123 бюджет_одобренных_закупок
1	Наука о данных	4 400	4,65	2	2	700
2	Программирование	3 860	4,56	5	5	800
3	Машинное обучение	3 420	4,8	1	1	0
4	Психология	2 150	4,6	1	1	0
5	Английский	1 430	4,2	1	1	0

Рисунок 13 - Результат

Выводы

Проведён сравнительный анализ PostgreSQL, MySQL, SQLite; выбрана PostgreSQL благодаря поддержке JSONB, MVCC, ACID, расширения pgvector и совместимости с Python/SQLAlchemy. Создана физическая база данных из 17 таблиц с типами BIGSERIAL, JSONB, INET, TIMESTAMP, DECIMAL, TEXT. Добавлены внешние ключи с каскадными действиями и ограничения CHECK. В каждую таблицу вставлено не менее 10 записей. Разработан аналитический запрос с CTE для статистики по категориям. База данных развёрнута на виртуальном стенде (раздел 10) и готова к использованию модулями АСУ.

РАЗДЕЛ 9. ДЕТАЛИЗАЦИЯ ХАРАКТЕРИСТИК ОБОРУДОВАНИЯ

Введение

Укрупнённый перечень аппаратного обеспечения (раздел 5) требует детализации конкретных параметров для закупки и развёртывания. Раздел 9 основан на материалах практической работы №9.

Цель раздела 9 – детализировать характеристики клиентского, серверного и сетевого оборудования, обеспечивающие выполнение всех функций АСУ.

Задачи раздела:

1. Детализировать клиентское оборудование.
2. Детализировать серверное оборудование по ролям (балансировщик, узлы K8s, БД, Elasticsearch, GPU, хранилище, инфраструктурные серверы).
3. Детализировать сетевое оборудование.

Клиентское оборудование

Параметр	Значение	Обоснование
Тип устройства	ПК / Тонкий клиент / Ноутбук	Доступ к системе осуществляется через браузер.
Процессор	Intel Core i3-12100 / AMD Ryzen 3 5300G	Достаточно для работы современного браузера (Chrome, Edge) с интерфейсом на React.
Оперативная память	8 ГБ DDR4	Обеспечивает одновременную работу ОС, браузера с несколькими вкладками и офисного пакета.
Накопитель	SSD 256 ГБ (NVMe/SATA)	Обеспечивает быструю загрузку ОС и браузера, что косвенно влияет на субъективное время отклика системы.
Сетевой адаптер	Gigabit Ethernet (1 Гбит/с) или Wi-Fi 5/6	Для стабильного подключения к веб-интерфейсу. Полосы пропускания достаточно для текстового и медиа-трафика.
Монитор	23.8", 1920x1080 (Full HD)	Комфортное разрешение для работы с таблицами метаданных и карточками контента.

Серверное оборудование

Балансировщик нагрузки

Параметр	Значение	Обоснование
Модель/Тип	Программный (Nginx / HAProxy) на сервере	В соответствии с выбранным ПО. Используем выделенный сервер для роли ingress-контроллера.
Процессор	1x Intel Xeon E-2336 (6 ядер, 3.0 ГГц)	Достаточно для терминирования SSL и проксирования трафика на 1000 одновременных соединений.
ОЗУ	16 ГБ DDR4 ECC	Основная нагрузка — сетевая, ОЗУ используется для буферов и кэша сессий.
Накопитель	2x SSD 480 ГБ (RAID 1)	Хранение логов доступа, конфигураций. RAID 1 для отказоустойчивости.
Сеть	2x 10 GbE (SFP+)	Агрегация каналов (Bonding) для обеспечения пропускной способности при пиковых нагрузках.

Серверы приложений (Kubernetes Worker Nodes)

Параметр	Значение	Обоснование
Количество	4 узла (минимально для отказоустойчивости)	Обеспечение работы микросервисов (FastAPI, React) и масштабирования при росте нагрузки.
Процессор	2x Intel Xeon Silver 4314 (16 ядер, 2.4 ГГц)	32 физических ядра на узел для запуска множества подов (pod) микросервисов.
ОЗУ	128 ГБ DDR4 ECC	32 ГБ на ядро — оптимально для Python-приложений (FastAPI) и Node.js. Позволяет держать в памяти сессии и кэш.
Накопитель	2x SSD 960 ГБ NVMe (RAID 1)	Высокая скорость ввода-вывода для работы контейнеров (Docker OverlayFS) и временных файлов.
Сеть	2x 25 GbE (SFP28)	Высокоскоростная связь с хранилищем и БД, минимизация задержек внутри ЦОД.

Серверы баз данных (PostgreSQL)

Параметр	Значение	Обоснование
Конфигурация	Кластер Patroni (3 узла: Primary + 2 Replica)	Обеспечение доступности 99.5% и RTO < 1 часа.
Процессор	2x Intel Xeon Gold 6330 (28 ядер, 2.0 ГГц)	Высокая частота и количество ядер для параллельной обработки OLTP-запросов от 1000 пользователей.
ОЗУ	256 ГБ DDR4 ECC	Критический параметр. Большой объем ОЗУ позволяет СУБД PostgreSQL держать «горячие» данные (индексы, метаданные контента) в памяти (shared_buffers), снижая нагрузку на диски и обеспечивая время ответа < 2 сек.
Накопитель	4x NVMe SSD 1.92 ТБ (RAID 10)	Используется для хранения файлов БД и WAL-логов. RAID 10 дает баланс скорости и надежности. NVMe необходим для быстрой записи транзакций (заявки, логи).
Сеть	2x 25 GbE	Быстрая репликация данных на standby-узлы.

Серверы поискового движка (Elasticsearch)

Параметр	Значение	Обоснование
Конфигурация	Кластер из 3 узлов	Обеспечение отказоустойчивости и распределения шардов индексов.
Процессор	2x Intel Xeon Silver 4314 (16 ядер)	Elasticsearch активно использует многопоточность для индексации и поиска.
ОЗУ	64 ГБ DDR4 ECC	Рекомендуется выделять до 50% ОЗУ под кучу (heap) JVM (но не более 31 ГБ). Остальная память используется ОС для кэширования файлов индексов (Lucene).
Накопитель	4x NVMe SSD 1.92 ТБ (RAID 0 или JBOD)	Критически важна скорость случайного чтения/записи для обратных индексов. Допустим RAID 0 для скорости, так как отказоустойчивость обеспечивается репликацией данных в кластере.
Сеть	2x 25 GbE	Интенсивный обмен данными между узлами при балансировке шардов и выполнении распределенного поиска.

Серверы для нейросетевых моделей (NLP / PyTorch)

Параметр	Значение	Обоснование
Количество	2 узла (активный + standby или для распределения нагрузки)	Обеспечение требования времени обработки запроса (семантический анализ ≤ 2 сек).
Процессор	1x AMD EPYC 7313 (16 ядер)	Достаточно для пред/пост-обработки данных и управления GPU.
ОЗУ	128 ГБ DDR4 ECC	Хранение предзагруженных моделей и эмбедингов.
GPU-ускоритель	NVIDIA A10 (24 ГБ) или A40 (48 ГБ)	Ключевой компонент. Необходим для быстрого инференса трансформерных моделей (Hugging Face, BERT). Модель cointegrated/rubert-tiny2 или более тяжелые аналоги. 24 ГБ VRAM достаточно для пакетной обработки (batching) запросов от множества пользователей.
Накопитель	2x NVMe SSD 1.92 ТБ (RAID 1)	Хранение docker-образов, кэша моделей, датасетов для дообучения.
Сеть	Mellanox ConnectX-6 25 GbE	Высокоскоростная связь для быстрой передачи данных между сервисами (gRPC/REST).

Файловое хранилище (S3-совместимое)

Параметр	Значение	Обоснование
Тип системы	Программно-определяемое хранилище (Ceph) на базе серверов	Хранение мультимедиа файлов (книги, видео, аудио) и демо-версий.
Конфигурация узла	3+ сервера хранения	Обеспечение репликации данных (replica 3).
Процессор	Intel Xeon Silver 4310 (12 ядер)	Обработка S3-запросов (RADOS Gateway).
ОЗУ	64 ГБ DDR4 ECC	Кэширование метаданных и ускорение операций.
Диски данных	12x HDD SATA 12 ТБ (RAID 6 или EC)	Экономичное хранение больших объемов видео и аудио контента.
Кэш (WAL/DB)	2x NVMe SSD 1.92 ТБ	Для журнала BlueStore в Ceph, что критически ускоряет запись объектов.
Сеть	2x 25 GbE (frontend + backend)	Выделенная сеть для репликации данных между узлами хранения.

Инфраструктурные и вспомогательные серверы

Параметр	Характеристики (минимальные)	Обоснование
Мониторинг и логирование (ELK + Prometheus)	3 узла: CPU Xeon E-2336, RAM 32 ГБ, SSD 2x480 ГБ RAID 1	Для сбора логов и метрик согласно требованиям логирования.
Сервер резервного копирования	CPU Xeon E-2336, RAM 16 ГБ, Ленточная библиотека LTO-9	Обеспечение восстановления за 1 час (RTO). Резервирование БД и метаданных контента.
Сервер управления (Bastion/K8s Master)	3 узла: CPU Xeon E-2336, RAM 32 ГБ, SSD 2x480 ГБ RAID 1	Обеспечение работы Control Plane Kubernetes.

Сетевое оборудование

Оборудование	Характеристики	Обоснование
Коммутатор ядра сети	2 шт. (стек), 32 порта 100GbE QSFP28, поддержка MLAG	Объединение серверных стоек. Необходимая пропускная способность для трафика East-West (между микросервисами, БД и хранилищем).
Коммутатор доступа / управления	2 шт., 48 портов 1GbE + 4 порта 25GbE SFP28	Подключение IPMI (управление питанием серверов), сетевого оборудования, точек доступа.
Межсетевой экран (NGFW)	Кластер Active-Passive, пропускная способность Threat Prevention не менее 8 Гбит/с	Защита периметра сети, инспекция SSL (TLS 1.3) трафика к веб-серверу, организация VPN для администраторов.
Структурированная кабельная система	Оптоволокно OM4/OS2, медная витая пара Cat.6A	Обеспечение скорости 10/25/100 Гбит/с в зависимости от порта сервера.

Выводы

Детализировано клиентское оборудование (Core i3, 8 ГБ RAM, SSD 256 ГБ, Full HD). Определены серверные конфигурации: балансировщик (Xeon E-2336, 16 ГБ RAM), узлы Kubernetes (2×Xeon Silver 4314, 128 ГБ RAM), серверы PostgreSQL (2×Xeon Gold 6330, 256 ГБ RAM), серверы Elasticsearch (2×Xeon Silver 4314, 64 ГБ RAM), GPU-серверы (AMD EPYC, NVIDIA A10/A40, 128 ГБ RAM), хранилище Ceph (3+ узлов, HDD 12 ТБ + NVMe кэш). Сетевое оборудование: коммутаторы 100/25 Гбит/с, NGFW кластер, кабельная система. Спецификации обеспечивают требования производительности и надёжности.

РАЗДЕЛ 10. СОЗДАНИЕ ВИРТУАЛЬНОГО СТЕНДА

Введение

Для интеграционного тестирования архитектуры до закупки физического оборудования необходимо создать виртуальный стенд. Раздел 10 основан на материалах практической работы №10.

Цель раздела 10 – создать виртуальный стенд, реализующий виртуальную инфраструктуру для АСУ «Семантический подборщик», с серверной и клиентской частью (не менее 3 ВМ).

Задачи раздела:

1. Провести анализ и сравнение не менее двух сред виртуализации.
2. Обосновать выбор среды.
3. Спроектировать архитектуру стенда (3 ВМ, роли, ПО, IP, ресурсы).
4. Создать ВМ, установить ПО, настроить сеть.
5. Представить скриншоты работающих сервисов.

Сравнение сред виртуализации и обоснование выбора

Критерии сравнения

Критерий	Oracle VM VirtualBox	VMware Workstation Pro
Лицензия	Полностью свободная (GPLv3), без ограничений	Бесплатна для личного использования, требует регистрации; коммерческое использование требует платной лицензии
Кроссплатформенность	Windows, Linux, macOS, Solaris	Windows, Linux (хост-система)
Поддержка гостевых ОС	Широкий список (Windows, Linux, BSD, Solaris, macOS с оговорками)	Широкий список, улучшенная поддержка Windows и Linux
Снапшоты	Поддерживаются в полном объеме	Поддерживаются в Pro-версии (отсутствуют в Player)
Виртуальные сети	NAT, NAT Network, Bridged, Internal, Host-Only, Generic (гибкая настройка)	NAT, Bridged, Host-Only, Custom (VMnet)
Производительность	Хорошая; на гостевых Linux сопоставима с нативной, но немного уступает VMware на Windows-хосте	Традиционно высокая, отличная оптимизация для Windows-гостей
Интерфейс и удобство	Простой, интуитивный GUI; возможно управление из	Удобный GUI с расширенными возможностями, тесная

	командной строки (VBoxManage)	интеграция с хостом (Unity, drag-and-drop)
Ограничения бесплатной версии	Отсутствуют	Ранее ограниченная версия Player без снапшотов; сейчас Pro бесплатна для личного использования

Анализ преимуществ и недостатков

VirtualBox выгодно отличается полной свободой использования, отсутствием каких-либо ограничений на снапшоты и виртуальные сети, а также кроссплатформенностью, что важно при возможной смене хост-ОС. Производительности достаточно для запуска легковесных серверных ОС и тестирования микросервисной архитектуры без интенсивной графической нагрузки.

Vmware Workstation Pro обеспечивает более высокую производительность в гостевых Windows-системах и ряд удобных функций (тесная интеграция с хостом, поддержка vTPM, улучшенная 3D-графика), однако для серверных Linux-машин эта разница минимальна. Привязка к регистрации и возможные изменения лицензионной политики в будущем создают риски для учебного проекта.

Обоснование выбора среды виртуализации

Для реализации виртуального стенда выбрана Oracle VM VirtualBox 7.0 по следующим причинам:

- Полная бесплатность и открытый исходный код – не требуется лицензионных затрат и регистраций, что соответствует условиям учебного процесса.
- Наличие снапшотов – позволяет сохранять промежуточные состояния стенда и быстро возвращаться к ним при ошибках конфигурации, что критически важно на этапе отладки.
- Кроссплатформенность – возможность переноса стенда между различными ОС (Windows, Linux, macOS), на которых могут работать студенты.

- Гибкость сетевых настроек – режим Host-Only Network и Internal Network идеально подходят для создания изолированной сети стенда с возможностью доступа с хоста.
- Достаточная производительность – для задач развёртывания серверных Linux-машин (без GUI) VirtualBox демонстрирует стабильную работу, не уступающую проприетарным аналогам.

Таким образом, VirtualBox обеспечивает все необходимые функции без юридических и финансовых ограничений, что делает его оптимальным выбором для образовательных целей.

Проектирование виртуального стенда

Архитектура стенда

Стенд воспроизводит ключевые компоненты АСУ «Семантический подборщик» на трёх виртуальных машинах, объединённых в изолированную сеть 192.168.56.0/24 (Host-Only).

1. VM1 (db-search-srv) – сервер баз данных, полнотекстового поиска и объектного хранилища.
2. VM2 (app-nlp-srv) – сервер приложений (бизнес-логика, API, веб-интерфейс) и NLP-инференс.
3. VM3 (client-ws) – клиентская рабочая станция с графическим окружением для доступа к системе через браузер.

Распределение ПО по виртуальным машинам

Виртуальная машина	Роль	Основное ПО	IP-адрес
VM1	Сервер данных и поиска	ОС: Ubuntu Server 22.04 LTS; PostgreSQL 14; Elasticsearch 8.x; MinIO	192.168.56.10
VM2	Сервер приложений и NLP	ОС: Ubuntu Server 22.04 LTS; Python 3.11, FastAPI, Uvicorn; Nginx; предобученная NLP-модель (RuBERT-tiny2)	192.168.56.20
VM3	Клиент	ОС: Ubuntu Desktop 22.04 LTS; веб-браузер Firefox/Chrome	192.168.56.30

Аппаратные ресурсы хоста и распределение

Распределение ресурсов по VM:

Параметр	VM1	VM2	VM3
vCPU	2	2	2
RAM	4 ГБ	4 ГБ	4 ГБ
Виртуальный диск	40 ГБ	30 ГБ	25 ГБ
Сеть	Host-Only	Host-Only	Host-Only

Результаты создания виртуальной инфраструктуры

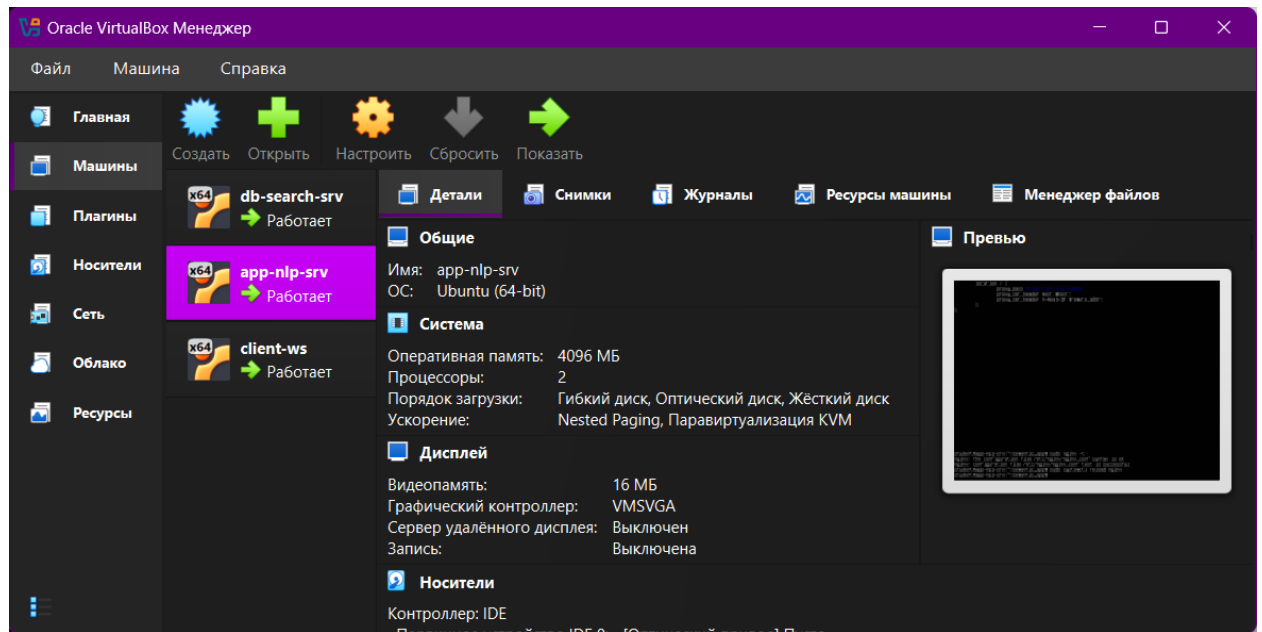


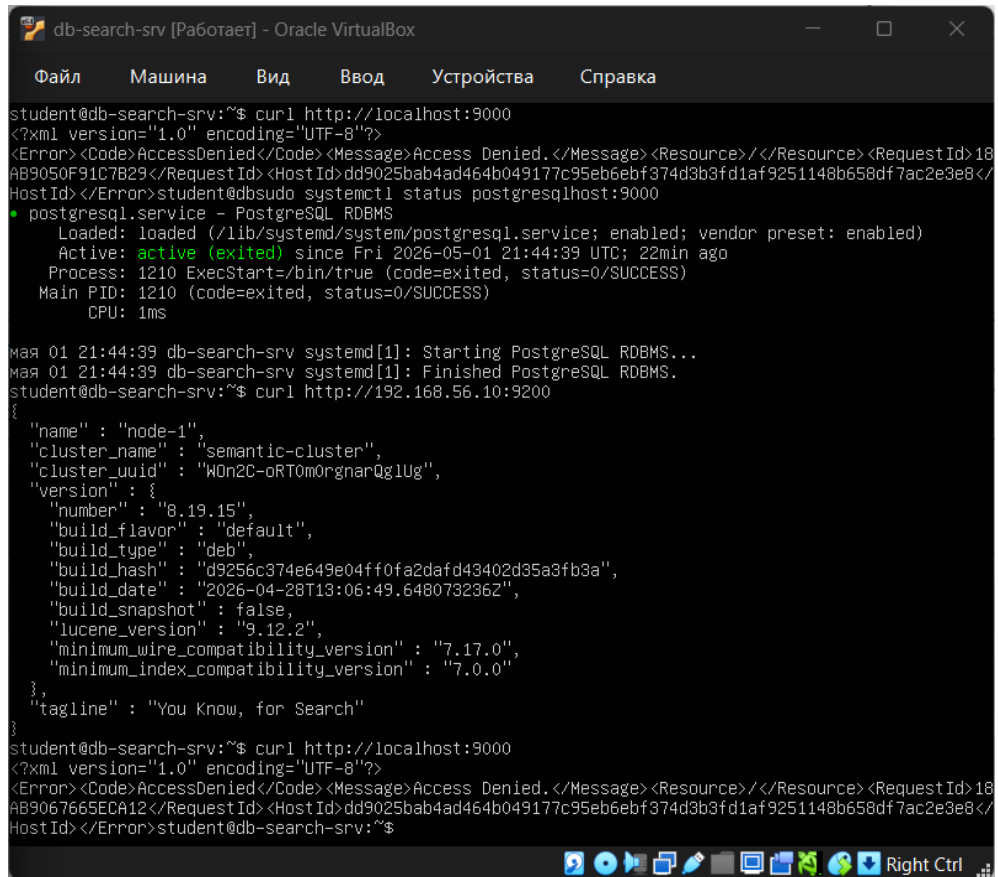
Рисунок 14 - VirtualBox с тремя запущенными VM

```
db-search-srv [Работает] - Oracle VirtualBox
Файл  Машина  Вид  Ввод  Устройства  Справка
character 13
2026-05-01 14:14:40.089 UTC [1022] postgres@semantic_picker: STATEMENT: GRANT USAGE IN SCHEMA public
  TO semantic_user;
2026-05-01 21:44:37.267 UTC [699] LOG:  received fast shutdown request
2026-05-01 21:44:37.271 UTC [699] LOG:  aborting any active transactions
2026-05-01 21:44:37.277 UTC [699] LOG:  background worker "logical replication launcher" (PID 737) e
xited with exit code 1
2026-05-01 21:44:37.279 UTC [732] LOG:  shutting down
2026-05-01 21:44:37.307 UTC [699] LOG:  database system is shut down
2026-05-01 21:44:37.484 UTC [1192] LOG:  starting PostgreSQL 14.22 (Ubuntu 14.22-0ubuntu0.22.04.1) o
n x86_64-pc-linux-gnu, compiled by gcc (Ubuntu 11.4.0-1ubuntu1~22.04.3) 11.4.0, 64-bit
2026-05-01 21:44:37.484 UTC [1192] LOG:  listening on IPv4 address "0.0.0.0", port 5432
2026-05-01 21:44:37.484 UTC [1192] LOG:  listening on IPv6 address ":::", port 5432
2026-05-01 21:44:37.489 UTC [1192] LOG:  listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432
"
2026-05-01 21:44:37.497 UTC [1193] LOG:  database system was shut down at 2026-05-01 21:44:37 UTC
2026-05-01 21:44:37.505 UTC [1192] LOG:  database system is ready to accept connections
student@db-search-srv:~$ sudo ss -tlnp | grep 5432
sudo: ss -tlnp: команда не найдена
student@db-search-srv:~$ sudo ss -tlnp | grep 5432
LISTEN 0      244          0.0.0.0:5432      0.0.0.0:*        users:(("postgres",pid=1192,fd=5)
)
LISTEN 0      244          [::]:5432        [::]:*          users:(("postgres",pid=1192,fd=6)
)
student@db-search-srv:~$ [ 2394.219118] e1000 0000:00:03:0 enp0s3: Reset adapter
sudo systemctl status postgresql
[sudo] пароль для student:
• postgresql.service - PostgreSQL RDBMS
  Loaded: loaded (/lib/systemd/system/postgresql.service; enabled; vendor preset: enabled)
  Active: active (exited) since Fri 2026-05-01 21:44:39 UTC; 18min ago
  Process: 1210 ExecStart=/bin/true (code=exited, status=0/SUCCESS)
  Main PID: 1210 (code=exited, status=0/SUCCESS)
  CPU: 1ms
мая 01 21:44:39 db-search-srv systemd[1]: Starting PostgreSQL RDBMS...
мая 01 21:44:39 db-search-srv systemd[1]: Finished PostgreSQL RDBMS.
student@db-search-srv:~$
```

Рисунок 15 - Работающий PostgreSQL

```
db-search-srv [Работает] - Oracle VirtualBox
Файл  Машина  Вид  Ввод  Устройства  Справка
sudo: ss -tlnp: команда не найдена
student@db-search-srv:~$ sudo ss -tlnp | grep 5432
LISTEN 0      244          0.0.0.0:5432      0.0.0.0:*        users:(("postgres",pid=1192,fd=5)
)
LISTEN 0      244          [::]:5432        [::]:*          users:(("postgres",pid=1192,fd=6)
)
student@db-search-srv:~$ [ 2394.219118] e1000 0000:00:03:0 enp0s3: Reset adapter
sudo systemctl status postgresql
[sudo] пароль для student:
• postgresql.service - PostgreSQL RDBMS
  Loaded: loaded (/lib/systemd/system/postgresql.service; enabled; vendor preset: enabled)
  Active: active (exited) since Fri 2026-05-01 21:44:39 UTC; 18min ago
  Process: 1210 ExecStart=/bin/true (code=exited, status=0/SUCCESS)
  Main PID: 1210 (code=exited, status=0/SUCCESS)
  CPU: 1ms
мая 01 21:44:39 db-search-srv systemd[1]: Starting PostgreSQL RDBMS...
мая 01 21:44:39 db-search-srv systemd[1]: Finished PostgreSQL RDBMS.
student@db-search-srv:~$ curl http://192.168.56.10:9200
{
  "name" : "node-1",
  "cluster_name" : "semantic-cluster",
  "cluster_uuid" : "W0n2C-oRT0m0rgnarQg1Ug",
  "version" : {
    "number" : "8.19.15",
    "build_flavor" : "default",
    "build_type" : "deb",
    "build_hash" : "d9256c374e649e04ff0fa2dafd43402d35a3fb3a",
    "build_date" : "2026-04-28T13:06:49.648073236Z",
    "build_snapshot" : false,
    "lucene_version" : "9.12.2",
    "minimum_wire_compatibility_version" : "7.17.0",
    "minimum_index_compatibility_version" : "7.0.0"
  },
  "tagline" : "You Know, for Search"
}
student@db-search-srv:~$
```

Рисунок 16 - Работающий Elasticsearch



```
student@db-search-srv:~$ curl http://localhost:9000
<?xml version="1.0" encoding="UTF-8"?>
<Error><Code>AccessDenied</Code><Message>Access Denied.</Message><Resource></Resource><RequestId>18
AB9050F91C7B29</RequestId><HostId>dd9025bab4ad464b049177c95eb6ebf374d3b3fd1af9251148b658df7ac2e3e8</
HostId></Error>student@dbsudo systemctl status postgresqlhost:9000
● postgresql.service - PostgreSQL RDBMS
   Loaded: loaded (/lib/systemd/system/postgresql.service; enabled; vendor preset: enabled)
   Active: active (exited) since Fri 2026-05-01 21:44:39 UTC; 22min ago
     Process: 1210 ExecStart=/bin/true (code=exited, status=0/SUCCESS)
    Main PID: 1210 (code=exited, status=0/SUCCESS)
       CPU: 1ms

мая 01 21:44:39 db-search-srv systemd[1]: Starting PostgreSQL RDBMS...
мая 01 21:44:39 db-search-srv systemd[1]: Finished PostgreSQL RDBMS.
student@db-search-srv:~$ curl http://192.168.56.10:9200
{
  "name" : "node-1",
  "cluster_name" : "semantic-cluster",
  "cluster_uuid" : "W0n2C-oRT0m0rgnarQg1Ug",
  "version" : {
    "number" : "8.19.15",
    "build_flavor" : "default",
    "build_type" : "deb",
    "build_hash" : "d9256c374e649e04ff0fa2dafd43402d35a3fb3a",
    "build_date" : "2026-04-28T13:06:49.648073236Z",
    "build_snapshot" : false,
    "lucene_version" : "9.12.2",
    "minimum_wire_compatibility_version" : "7.17.0",
    "minimum_index_compatibility_version" : "7.0.0"
  },
  "tagline" : "You Know, for Search"
}
student@db-search-srv:~$ curl http://localhost:9000
<?xml version="1.0" encoding="UTF-8"?>
<Error><Code>AccessDenied</Code><Message>Access Denied.</Message><Resource></Resource><RequestId>18
AB9067665ECA12</RequestId><HostId>dd9025bab4ad464b049177c95eb6ebf374d3b3fd1af9251148b658df7ac2e3e8</
HostId></Error>student@db-search-srv:~$
```

Рисунок 17 - Работающий MinIO

```
student@app-nlp-srv:~/semantic_app$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
student@app-nlp-srv:~/semantic_app$ sudo systemctl reload nginx
student@app-nlp-srv:~/semantic_app$ sudo systemctl status semantic
* semantic.service - Semantic Picker API
   Loaded: loaded (/etc/systemd/system/semantic.service; enabled; vendor preset: enabled)
   Active: active (running) since Fri 2026-05-01 21:20:38 UTC; 47min ago
     Main PID: 45198 (uvicorn)
        Tasks: 2 (limit: 4557)
       Memory: 50.3M
          CPU: 22.187s
       CGroup: /system.slice/semantic.service
               └─45198 /home/student/semantic_app/venv/bin/python3 /home/student/semantic_app/venv/bin/uvicorn main:app --host 0.0.0.0 --port 8000

May 01 21:28:41 app-nlp-srv uvicorn[45198]:   conn = _connect(dsn, connection_factory=connection_>
May 01 21:28:41 app-nlp-srv uvicorn[45198]: sqlalchemy.exc.OperationalError: (psycopg2.OperationalE>
May 01 21:28:41 app-nlp-srv uvicorn[45198]:   Is the server running on that host and acceptin>
May 01 21:28:41 app-nlp-srv uvicorn[45198]:   (Background on this error at: https://sqlalche.me/e/20/>
May 01 21:46:42 app-nlp-srv uvicorn[45198]: INFO:      127.0.0.1:44970 - "GET /api/content HTTP/1.1">
May 01 21:53:12 app-nlp-srv uvicorn[45198]: INFO:      127.0.0.1:55662 - "GET /docs HTTP/1.1" 200 OK
May 01 21:59:18 app-nlp-srv uvicorn[45198]: INFO:      127.0.0.1:49056 - "GET /docs HTTP/1.0" 200 OK
May 01 21:59:19 app-nlp-srv uvicorn[45198]: INFO:      127.0.0.1:49068 - "GET /openapi.json HTTP/1.0">
May 01 22:06:55 app-nlp-srv uvicorn[45198]: INFO:      127.0.0.1:57442 - "GET /docs HTTP/1.0" 200 OK
May 01 22:06:55 app-nlp-srv uvicorn[45198]: INFO:      127.0.0.1:57454 - "GET /openapi.json HTTP/1.0">
lines 1-20/20 (END)
```

Рисунок 18 - Работающий сервис API

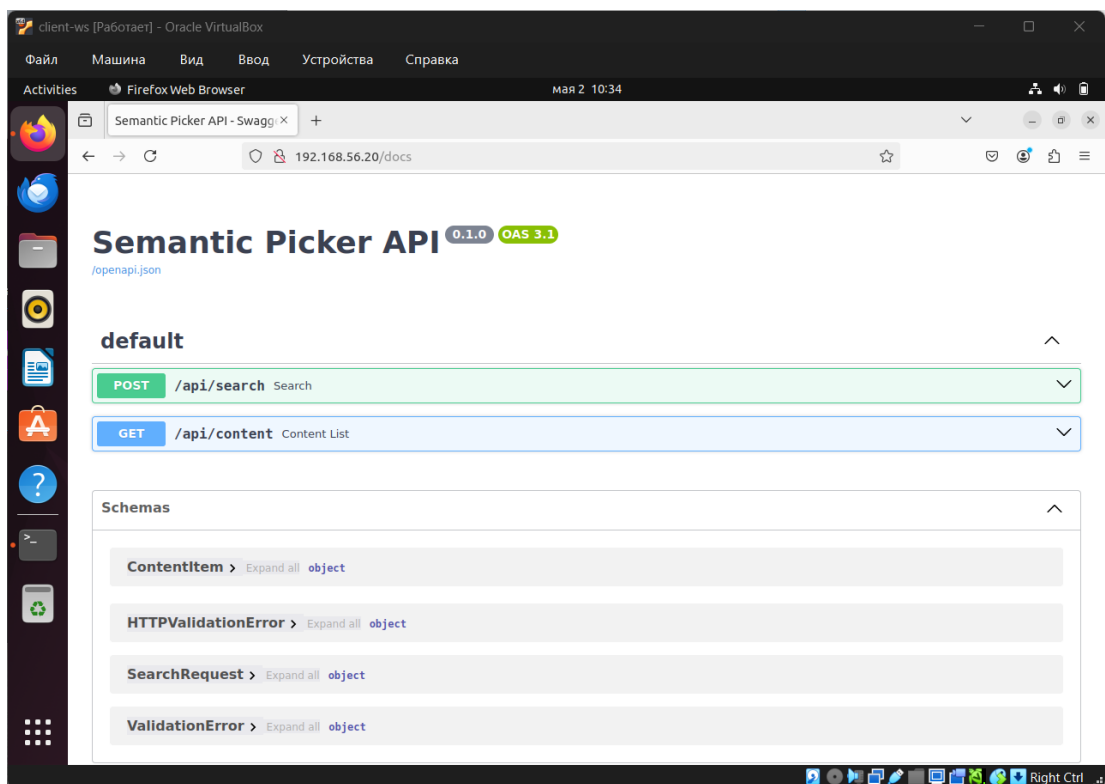


Рисунок 19 - Swagger-документация

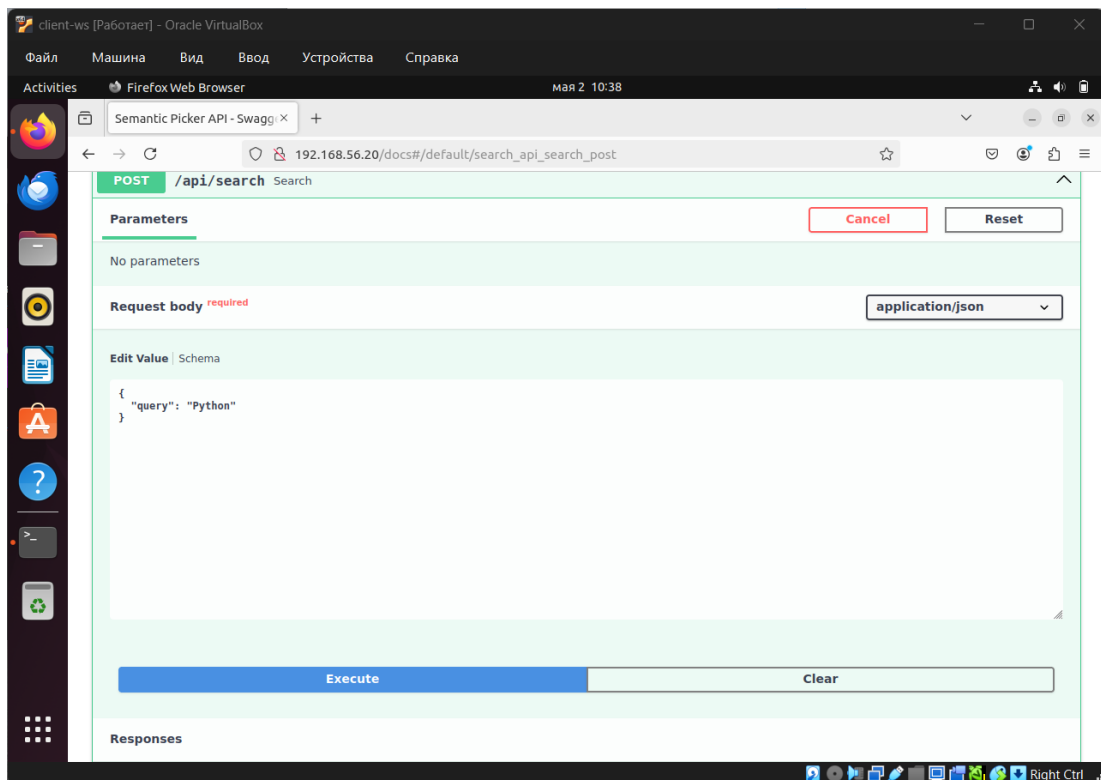


Рисунок 20 - Тестовый запрос

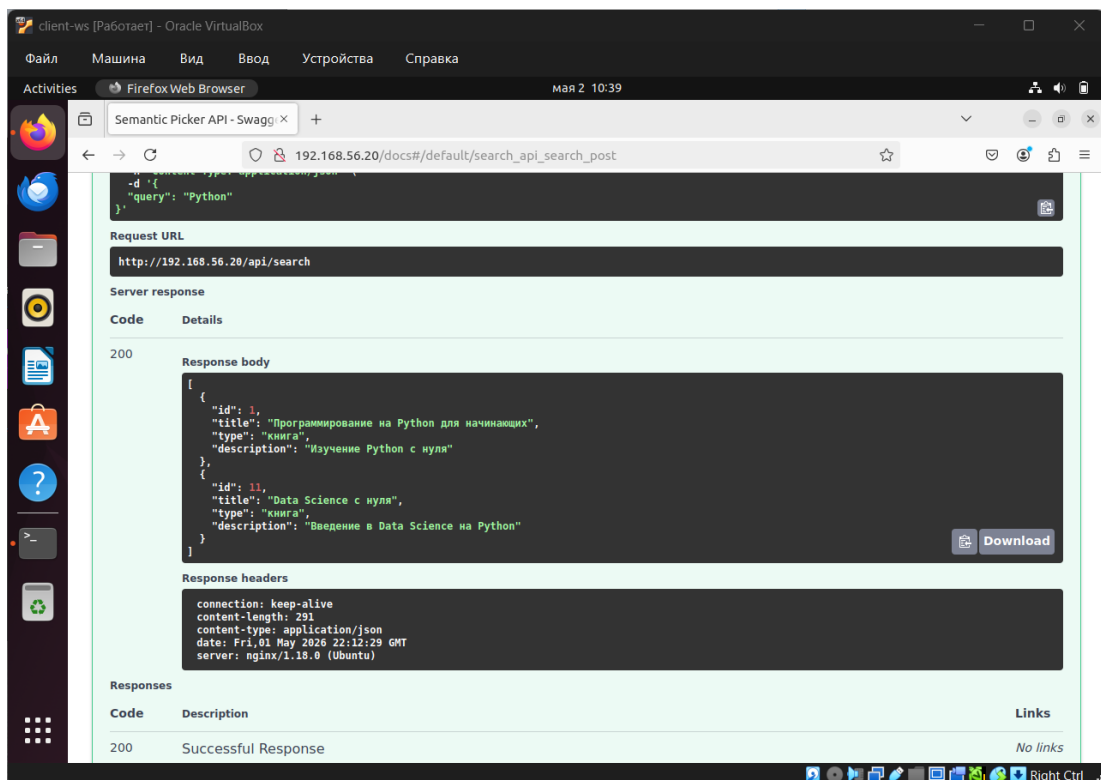


Рисунок 21 - Результат тестового поиска

Выводы

Выбран Oracle VM VirtualBox 7.0 (бесплатность, снапшоты, кроссплатформенность, Host-Only). Создан стенд из трёх ВМ: VM1 (PostgreSQL, Elasticsearch, MinIO), VM2 (FastAPI, RuBERT, Nginx), VM3 (браузер). На каждой ВМ 2 vCPU, 4 ГБ RAM. Скриншоты подтверждают работу PostgreSQL, Elasticsearch, MinIO, FastAPI, Swagger, выполнение тестового поиска. Стенд эмулирует ключевые функции АСУ.

РАЗДЕЛ 11. РАЗРАБОТКА ИНФОРМАЦИОННОГО WEB-ПОРТАЛА ДЛЯ ПОЛЬЗОВАТЕЛЕЙ

Введение

Для конечных пользователей необходим веб-портал с информацией о системе, демонстрацией поиска и визуализацией результатов. Раздел 11 основан на материалах практической работы №11.

Цель раздела 11 – разработать информационный web-портал по направлению деятельности организации, включающий не менее трёх разделов, с использованием CSS и веб-скриптов.

Задачи раздела:

1. Создать структуру портала (вкладки «О системе», «Поиск контента», «Результаты работы»).
2. Разместить описание АСУ, технологической основы, пользователей.
3. Реализовать эмуляцию семантического поиска с фильтрацией и пагинацией.
4. Реализовать отображение сводной статистики и журнала запросов.
5. Представить скриншоты и исходный код.

Описание разработанного портала

Web-портал реализован в виде статического веб-приложения, состоящего из трёх страниц-вкладок, объединённых в одном HTML-документе. Для обеспечения визуального оформления использован каскадный лист стилей (`style.css`), а интерактивность и динамическое обновление данных обеспечены сценарием на языке JavaScript (`script.js`). Портал не требует установки серверного ПО и открывается любым современным браузером.

Структура портала:

1. Вкладка «О системе» – содержит общую информацию о проектируемой АСУ, технологической основе и категориях

пользователей. Материалы полностью соответствуют описанию, приведённому в предыдущих работах (нейросетевое ядро, микросервисная архитектура, использование PostgreSQL, Elasticsearch, PyTorch и т.д.).

2. Вкладка «Поиск контента» – имитирует основной рабочий процесс АСУ: подбор мультимедиа контента по естественно-языковому запросу. Включает поле ввода, фильтр по типу контента и динамическую сетку карточек с результатами. Реализована постраничная навигация (пагинация) – по 6 элементов на страницу. При пустом запросе отображаются все доступные единицы контента.
3. Вкладка «Результаты работы» – демонстрирует сводные показатели эмуляции работы АСУ (общее число обработанных запросов, точность NLP-модели, объём базы контента, количество заявок на закупку), а также журнал последних запросов пользователей и динамику активности. Кнопка «Обновить сводку» случайным образом варьирует значения метрик, имитируя сбор актуальных данных.

Наполнение контентом выполнено в строгом соответствии с тематикой АСУ «Семантический подборщик»: карточки контента содержат книги, видеокурсы, аудиокниги и подкасты, относящиеся к ИТ- и образовательной сфере.

Использованные технологии и соответствие требованиям

1. HTML5 – семантическая разметка, обеспечивающая корректное отображение во всех браузерах.
2. CSS3 – адаптивная сетка на основе Grid, плавные анимации переходов, современное оформление карточек и пагинации.
3. JavaScript (ES6) – реализация логики переключения вкладок, фильтрации и поиска по ключевым словам, пагинации, а также эмуляции обновления статистики.

Результаты работы

Вкладка «О системе»

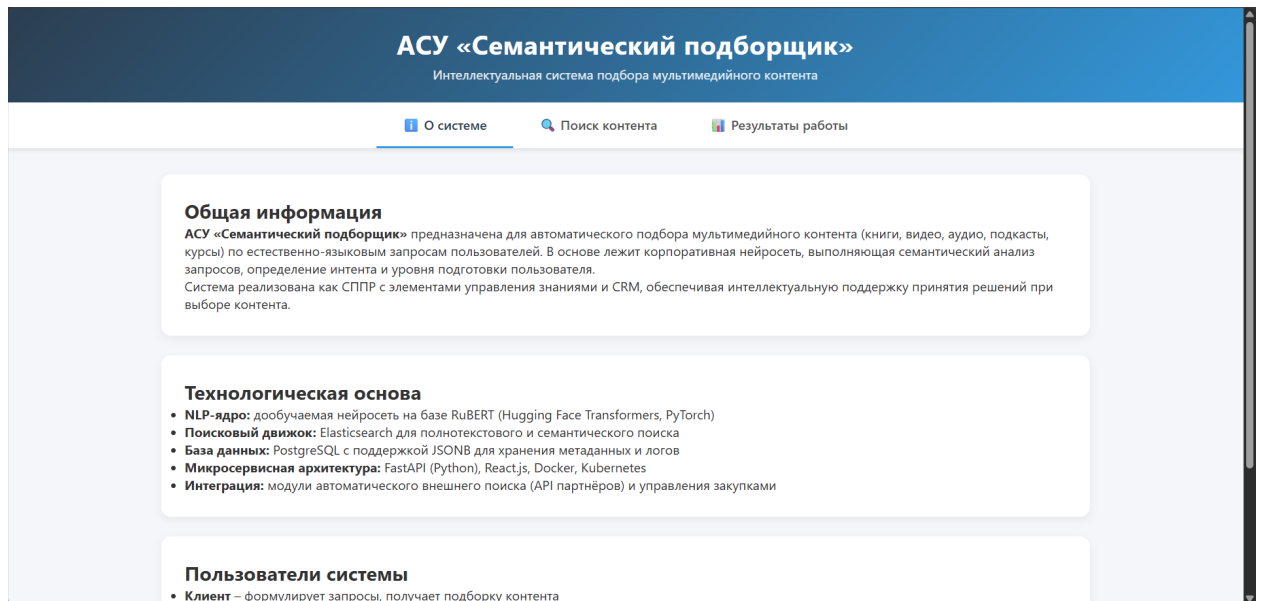


Рисунок 22 - Вкладка «О системе»

Вкладка «Поиск контента»

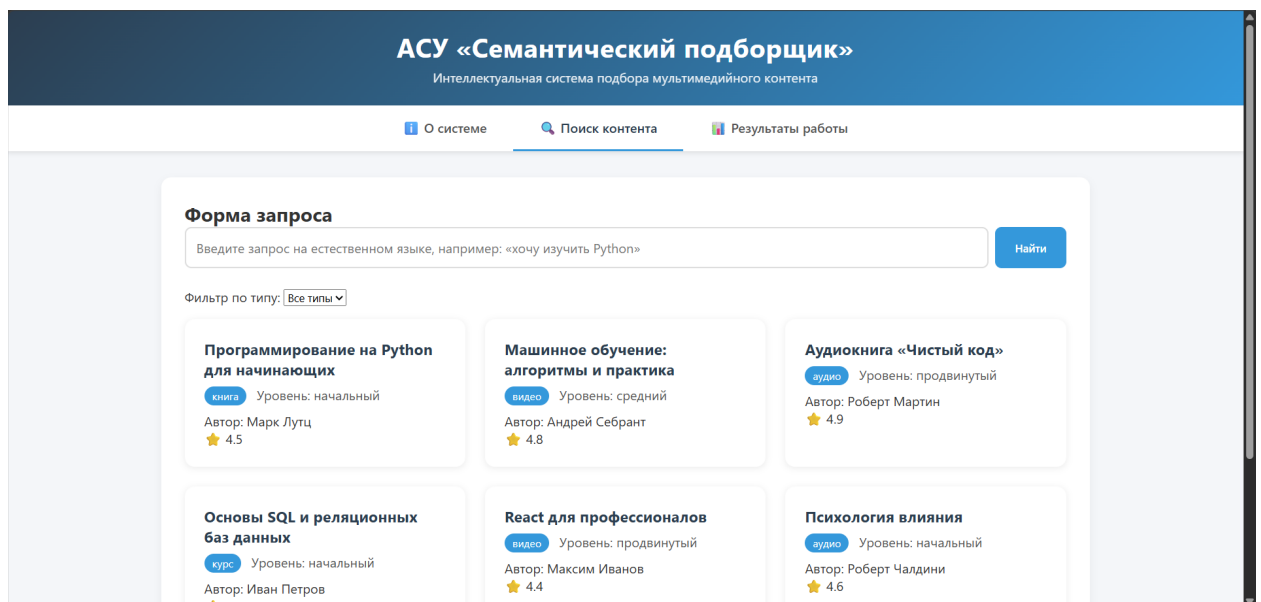


Рисунок 23 - Вкладка «Поиск контента»

Вкладка «Результаты работы»

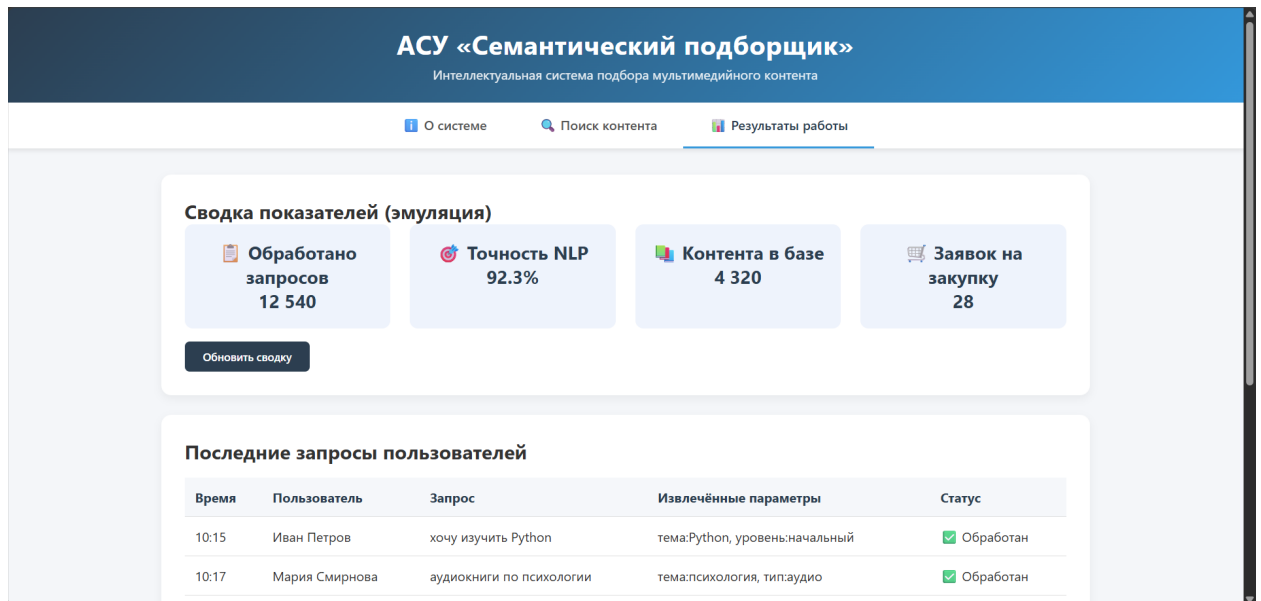


Рисунок 24 - Вкладка «Результаты работы»

Написанный код

Листинг 4: index.html

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Информационный портал АСУ «Семантический подборщик»</title>
  <link rel="stylesheet" href="./style.css">
</head>
<body>
  <header>
    <h1>АСУ «Семантический подборщик»</h1>
    <p>Интеллектуальная система подбора мультимедийного контента</p>
  </header>

  <nav>
    <button class="tab-link active" data-tab="tab1">О системе</button>
    <button class="tab-link" data-tab="tab2">Поиск контента</button>
    <button class="tab-link" data-tab="tab3">Результаты
работы</button>
  </nav>

  <main>
    <div id="tab1" class="tab-content active">
      <div class="card">
        <h2>Общая информация</h2>
        <p><strong>АСУ «Семантический подборщик»</strong>
предназначена для автоматического подбора мультимедийного контента (книги,
видео, аудио, подкасты, курсы) по естественно-языковым запросам
пользователей. В основе лежит корпоративная нейросеть, выполняющая
семантический анализ запросов, определение интента и уровня подготовки
пользователя.</p>

```

```

        <p>Система реализована как СППР с элементами управления
знаниями и CRM, обеспечивая интеллектуальную поддержку принятия решений при
выборе контента.</p>
    </div>
    <div class="card">
        <h2>Технологическая основа</h2>
        <ul>
            <li><strong>NLP-ядро:</strong> дообучаемая нейросеть на
базе RuBERT (Hugging Face Transformers, PyTorch)</li>
            <li><strong>Поисковый движок:</strong> Elasticsearch для
полнотекстового и семантического поиска</li>
            <li><strong>База данных:</strong> PostgreSQL с поддержкой
JSONB для хранения метаданных и логов</li>
            <li><strong>Микросервисная архитектура:</strong> FastAPI
(Python), React.js, Docker, Kubernetes</li>
            <li><strong>Интеграция:</strong> модули автоматического
внешнего поиска (API партнёров) и управления закупками</li>
        </ul>
    </div>
    <div class="card">
        <h2>Пользователи системы</h2>
        <ul>
            <li><strong>Клиент</strong> - формулирует запросы,
получает подборку контента</li>
            <li><strong>Контент-менеджер</strong> - управляет
каталогом, одобряет закупки</li>
            <li><strong>Администратор нейросети</strong> - дообучает
модель, контролирует качество</li>
            <li><strong>Руководитель</strong> - утверждает бюджет и
стратегию пополнения</li>
        </ul>
    </div>
</div>

<div id="tab2" class="tab-content">
    <div class="card">
        <h2>Форма запроса</h2>
        <div class="search-box">
            <input type="text" id="searchInput" placeholder="Введите
запрос на естественном языке, например: «хочу изучить Python»>
            <button onclick="executeSearch()">Найти</button>
        </div>
        <div style="margin-bottom:1rem;">
            <label>Фильтр по типу: </label>
            <select id="typeFilter" onchange="executeSearch()">
                <option value="all">Все типы</option>
                <option value="книга">Книга</option>
                <option value="видео">Видео</option>
                <option value="аудио">Аудио</option>
                <option value="курс">Курс</option>
                <option value="подкаст">Подкаст</option>
            </select>
        </div>
        <div id="searchResults" class="result-grid"></div>
        <div id="pagination" class="pagination"></div>
    </div>
</div>

<div id="tab3" class="tab-content">
    <div class="card">
        <h2>Сводка показателей (эмуляция)</h2>

```

```

        <div class="metrics">
            <div class="metric-box"> 📄 Обработано запросов<br><span
id="totalQueries">12 540</span></div>
            <div class="metric-box"> 🎯 Точность NLP<br><span
id="nlpAccuracy">92.3</span>%</div>
            <div class="metric-box"> 📖 Контента в базе<br><span
id="contentCount">4 320</span></div>
            <div class="metric-box"> 🛒 Заявок на закупку<br><span
id="purchaseOrders">28</span></div>
        </div>
        <button class="btn" onclick="refreshStats()">Обновить
сводку</button>
    </div>
    <div class="card">
        <h2>Последние запросы пользователей</h2>
        <table>
            <thead>

<tr><th>Время</th><th>Пользователь</th><th>Запрос</th><th>Извлечённые
параметры</th><th>Статус</th></tr>
            </thead>
            <tbody>
                <tr><td>10:15</td><td>Иван Петров</td><td>хочу
изучить Python</td><td>тема:Python, уровень:начальный</td><td>✅
Обработан</td></tr>
                <tr><td>10:17</td><td>Мария
Смирнова</td><td>аудиокниги по психологии</td><td>тема:психология,
тип:аудио</td><td>✅ Обработан</td></tr>
                <tr><td>10:22</td><td>Павел
Новиков</td><td>продвинутый курс по Go</td><td>тема:Go,
уровень:продвинутый</td><td>✅ Обработан</td></tr>
                <tr><td>10:25</td><td>Гость</td><td>скачать книгу
Чистый код</td><td>тема:Clean Code, действие:скачать</td><td>⚠ Нет
прав</td></tr>
            </tbody>
        </table>
    </div>
    <div class="card">
        <h2>Динамика активности (условные единицы)</h2>
        <p>Последние 7 дней: <strong>1240 → 1350 → 1420 → 1380 → 1500
→ 1480 → 1520</strong></p>
        <p>Среднее время ответа системы: <strong>1.8 сек</strong></p>
        <p>Доля запросов, потребовавших внешнего поиска:
<strong>12%</strong></p>
    </div>
</main>

<footer>
    © 2026 Информационный портал АСУ «Семантический подборщик». Учебный
проект Враженко Д.О.
</footer>

<script src="./script.js" defer></script>
</body>
</html>v

```

Листинг 5: style.css

```

* {
    margin: 0;

```

```

padding: 0;
box-sizing: border-box;
font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
}

body {
background: #f4f6f9;
color: #333;
display: flex;
flex-direction: column;
min-height: 100vh;
}

header {
background: linear-gradient(135deg, #2c3e50, #3498db);
color: white;
padding: 1.5rem 2rem;
text-align: center;
box-shadow: 0 4px 12px rgba(0, 0, 0, 0.2);
}

header h1 {
font-size: 2rem;
letter-spacing: 0.5px;
}

header p {
margin-top: 0.3rem;
opacity: 0.9;
}

nav {
background: white;
display: flex;
justify-content: center;
flex-wrap: wrap;
box-shadow: 0 2px 6px rgba(0, 0, 0, 0.1);
}

nav button {
background: none;
border: none;
padding: 1rem 2rem;
font-size: 1rem;
font-weight: 600;
color: #555;
cursor: pointer;
transition: 0.3s;
border-bottom: 3px solid transparent;
}

nav button:hover {
background: #f0f4f8;
color: #2c3e50;
}

nav button.active {
color: #2c3e50;
border-bottom-color: #3498db;
}

main {
flex: 1;
padding: 2rem;
max-width: 1200px;
margin: 0 auto;
}

```



```

        width: 100%;
    }
    .tab-content {
        display: none;
        animation: fadeIn 0.4s ease;
    }
    .tab-content.active {
        display: block;
    }
    @keyframes fadeIn {
        from { opacity: 0; transform: translateY(10px); }
        to { opacity: 1; transform: translateY(0); }
    }

    .card {
        background: white;
        border-radius: 12px;
        padding: 1.8rem;
        margin-bottom: 1.5rem;
        box-shadow: 0 4px 10px rgba(0, 0, 0, 0.05);
    }

    .metrics {
        display: flex;
        gap: 1.5rem;
        flex-wrap: wrap;
    }
    .metric-box {
        flex: 1 1 180px;
        background: #eef3fc;
        text-align: center;
        padding: 1.2rem;
        border-radius: 10px;
        font-size: 1.4rem;
        font-weight: bold;
        color: #2c3e50;
    }

    .search-box {
        display: flex;
        gap: 0.5rem;
        margin-bottom: 1.5rem;
    }
    .search-box input {
        flex: 1;
        padding: 0.8rem;
        border: 2px solid #ddd;
        border-radius: 8px;
        font-size: 1rem;
    }
    .search-box button {
        background: #3498db;
        color: white;
        border: none;
        padding: 0.8rem 1.5rem;
        border-radius: 8px;
        font-weight: 600;
        cursor: pointer;
        transition: 0.3s;
    }
    .search-box button:hover {

```

```

        background: #2980b9;
    }

    .result-grid {
        display: grid;
        grid-template-columns: repeat(auto-fill, minmax(280px, 1fr));
        gap: 1.5rem;
        margin-bottom: 1.5rem;
    }

    .content-card {
        background: white;
        border-radius: 12px;
        padding: 1.5rem;
        box-shadow: 0 2px 8px rgba(0, 0, 0, 0.08);
        transition: 0.3s;
    }

    .content-card:hover {
        box-shadow: 0 4px 16px rgba(0, 0, 0, 0.15);
    }

    .content-card h3 {
        color: #2c3e50;
        margin-bottom: 0.5rem;
    }

    .badge {
        display: inline-block;
        background: #3498db;
        color: white;
        padding: 0.2rem 0.6rem;
        border-radius: 20px;
        font-size: 0.8rem;
        margin-right: 0.5rem;
    }

    .pagination {
        display: flex;
        justify-content: center;
        align-items: center;
        gap: 1rem;
        margin-top: 1rem;
    }

    .pagination button {
        background: #2c3e50;
        color: white;
        border: none;
        padding: 0.5rem 1rem;
        border-radius: 6px;
        cursor: pointer;
        font-weight: 600;
        transition: 0.3s;
    }

    .pagination button:disabled {
        background: #ccc;
        cursor: not-allowed;
    }

    .pagination button:hover:not(:disabled) {
        background: #1e2b38;
    }

    .pagination span {
        font-weight: 600;
        color: #2c3e50;
    }

```

```

table {
  width: 100%;
  border-collapse: collapse;
  margin: 1rem 0;
}
th, td {
  text-align: left;
  padding: 0.8rem;
  border-bottom: 1px solid #e0e0e0;
}
th {
  background: #f5f7fa;
  color: #2c3e50;
}

.btn {
  background: #2c3e50;
  color: white;
  border: none;
  padding: 0.6rem 1.4rem;
  border-radius: 6px;
  cursor: pointer;
  font-weight: 600;
  margin-top: 1rem;
}
.btn:hover {
  background: #1e2b38;
}

footer {
  text-align: center;
  padding: 1rem;
  color: #777;
  font-size: 0.9rem;
  background: white;
  border-top: 1px solid #eee;
  margin-top: auto;
}

```

Листинг 6: script.js

```

const tabButtons = document.querySelectorAll('.tab-link');
const tabContents = document.querySelectorAll('.tab-content');

tabButtons.forEach(btn => {
  btn.addEventListener('click', () => {
    tabButtons.forEach(b => b.classList.remove('active'));
    tabContents.forEach(c => c.classList.remove('active'));
    btn.classList.add('active');
    document.getElementById(btn.dataset.tab).classList.add('active');
  });
});

const contentBase = [
  { title: "Программирование на Python для начинающих", type: "книга",
    author: "Марк Лутц", level: "начальный", rating: 4.5, keywords: ["python",
    "изучить", "начинающий"] },
  { title: "Машинное обучение: алгоритмы и практика", type: "видео",
    author: "Андрей Себрант", level: "средний", rating: 4.8, keywords: ["машинное

```

```

обучение", "ml", "алгоритмы"] },
  { title: "Аудиокнига «Чистый код»", type: "аудио", author: "Роберт
Мартин", level: "продвинутый", rating: 4.9, keywords: ["чистый код", "clean
code", "скачать"] },
  { title: "Основы SQL и реляционных баз данных", type: "курс", author:
"Иван Петров", level: "начальный", rating: 4.3, keywords: ["sql", "базы
данных", "изучить"] },
  { title: "React для профессионалов", type: "видео", author: "Максим
Иванов", level: "продвинутый", rating: 4.4, keywords: ["react", "фронтенд",
"профессионал"] },
  { title: "Психология влияния", type: "аудио", author: "Роберт Чалдини",
level: "начальный", rating: 4.6, keywords: ["психология", "влияние"] },
  { title: "Микросервисы: паттерны и практики", type: "книга", author:
"Крис Ричардсон", level: "продвинутый", rating: 4.9, keywords:
["микросервисы", "архитектура"] },
  { title: "Английский для IT", type: "курс", author: "Lingualeo", level:
"средний", rating: 4.2, keywords: ["английский", "it"] },
  { title: "Подкаст «Люди и код» выпуск 45", type: "подкаст", author:
"Алексей Котов", level: "начальный", rating: 4.0, keywords: ["подкаст", "it",
"стартапы"] },
  { title: "Глубокое обучение с PyTorch", type: "книга", author: "Eli
Stevens", level: "продвинутый", rating: 4.7, keywords: ["глубокое обучение",
"pytorch", "нейросети"] }
];

const ITEMS_PER_PAGE = 6;
let currentPage = 1;
let currentFilteredResults = [];

function getFilteredData() {
  const query =
document.getElementById('searchInput').value.toLowerCase().trim();
  const typeFilter = document.getElementById('typeFilter').value;

  let results = contentBase;

  if (query !== '') {
    results = results.filter(item =>
      item.keywords.some(kw => query.includes(kw)) ||
      item.title.toLowerCase().includes(query) ||
      item.author.toLowerCase().includes(query)
    );
  }

  if (typeFilter !== 'all') {
    results = results.filter(item => item.type === typeFilter);
  }
  return results;
}

function displayResults(results) {
  const container = document.getElementById('searchResults');
  const paginationDiv = document.getElementById('pagination');

  if (results.length === 0) {
    container.innerHTML = '<p>По вашему запросу ничего не найдено.
Возможно, потребуется внешний поиск.</p>';
    paginationDiv.innerHTML = '';
    return;
  }
}

```

```

const totalPages = Math.ceil(results.length / ITEMS_PER_PAGE);
if (currentPage > totalPages) currentPage = totalPages;
const startIndex = (currentPage - 1) * ITEMS_PER_PAGE;
const paginatedItems = results.slice(startIndex, startIndex +
ITEMS_PER_PAGE);

let html = '';
paginatedItems.forEach(item => {
  html += `
    <div class="content-card">
      <h3>${item.title}</h3>
      <span class="badge">${item.type}</span>
      <span style="color:#555;">Уровень: ${item.level}</span>
      <p style="margin-top:0.5rem;">Автор: ${item.author}</p>
      <p>★ ${item.rating}</p>
    </div>`;
});
container.innerHTML = html;

let paginationHtml = '';
if (totalPages > 1) {
  paginationHtml += `<button ${currentPage === 1 ? 'disabled' : ''}
onclick="goToPage(${currentPage - 1})">◀ Предыдущая</button>`;
  paginationHtml += `<span>Страница ${currentPage} из
${totalPages}</span>`;
  paginationHtml += `<button ${currentPage === totalPages ?
'disabled' : ''} onclick="goToPage(${currentPage + 1})">Следующая
▶</button>`;
}
paginationDiv.innerHTML = paginationHtml;
}

function goToPage(page) {
  currentPage = page;
  displayResults(currentFilteredResults);
}

function executeSearch() {
  currentFilteredResults = getFilteredData();
  currentPage = 1;
  displayResults(currentFilteredResults);
}

document.getElementById('searchInput').addEventListener('keypress',
function(e) {
  if (e.key === 'Enter') {
    e.preventDefault();
    executeSearch();
  }
});

function initialLoad() {
  currentFilteredResults = getFilteredData();
  currentPage = 1;
  displayResults(currentFilteredResults);
}

document.addEventListener('DOMContentLoaded', initialLoad);

function refreshStats() {
  const total = Math.floor(12540 + Math.random() * 200);

```

```

const accuracy = (92.3 + (Math.random() * 2 - 1)).toFixed(1);
const content = Math.floor(4320 + Math.random() * 50);
const purchases = Math.floor(28 + Math.random() * 5);

document.getElementById('totalQueries').textContent =
total.toLocaleString();
document.getElementById('nlpAccuracy').textContent = accuracy;
document.getElementById('contentCount').textContent =
content.toLocaleString();
document.getElementById('purchaseOrders').textContent = purchases;

const btn = document.querySelector('.btn');
btn.textContent = '✅ Обновлено';
setTimeout(() => { btn.textContent = 'Обновить сводку'; }, 1500);
}

```

Выводы

Разработан портал с тремя вкладками. Вкладка «О системе» содержит описание АСУ и технологий. Вкладка «Поиск контента» эмулирует поиск по ключевым словам с фильтром по типу и пагинацией. Вкладка «Результаты работы» показывает метрики (запросы, точность, объём базы, заявки) и журнал запросов. Реализовано на HTML5, CSS3 (адаптивная грид-сетка), JavaScript ES6. Портал является прототипом для интеграции с реальным бэкендом.

РАЗДЕЛ 12. РАЗРАБОТКА МОДУЛЯ МЕНЕДЖЕРА КОНТЕНТА ДЛЯ РАБОТЫ С БД

Введение

Для контент-менеджеров необходим модуль для управления каталогом мультимедиа контента (CRUD-операции). Раздел 12 основан на материалах практической работы №12.

Цель раздела 12 – разработать программный модуль с веб-интерфейсом, позволяющий работать с базой данных АСУ (выполнять запросы на внесение и получение данных).

Задачи раздела:

1. Выбрать технологический стек (Flask, psycpg2, Jinja2).
2. Реализовать просмотр, поиск, добавление, редактирование, удаление записей.
3. Обеспечить информационные сообщения.
4. Развернуть модуль на виртуальном стенде и подключить к БД.
5. Представить скриншоты и код.

Обоснование выбора модуля

Согласно архитектуре АСУ, разработанной в предыдущих работах, подсистема управления контентом отвечает за наполнение внутренней базы метаданных (книги, видео, аудио, курсы, подкасты). Для контент-менеджера критически важна возможность добавлять новые материалы, редактировать существующие описания, просматривать каталог и удалять устаревшие записи.

Поэтому в качестве программного модуля реализован «Менеджер контента» – веб-приложение на Flask (Python), подключающееся к PostgreSQL базе данных проектируемой АСУ и предоставляющее интуитивно понятный интерфейс для работы с таблицей контент.

Архитектура и технологии модуля

Модуль построен по классической клиент-серверной схеме:

- Серверная часть: Flask (Python), библиотека psycorg2 для взаимодействия с PostgreSQL.
- Клиентская часть: HTML-шаблоны с использованием Jinja2, стилизованные под корпоративный стиль.
- База данных: PostgreSQL (развёрнута на виртуальной машине VM1 стенда, IP 192.168.56.10, БД semantic_picker).

Модуль использует таблицу контент.

Функциональные возможности модуля

Модуль обеспечивает выполнение следующих операций:

- Просмотр всего списка контента – выборка всех записей из таблицы контент с отображением в табличном виде.
- Поиск – фильтрация по части названия или имени автора (оператор ILIKE).
- Добавление нового контента – форма с полями: название, тип, авторы, описание, уровень, язык; после отправки выполняется SQL-запрос INSERT.
- Редактирование – загрузка текущих значений в форму и выполнение UPDATE с установкой даты обновления.
- Удаление – запрос DELETE по идентификатору с подтверждением.
- Информирование пользователя – вывод флеш-сообщений об успехе или ошибке.

Все изменения немедленно отражаются на странице списка контента.

Структура базы данных и SQL-запросы

Модуль работает с таблицей контент, имеющей следующие поля, задействованные в интерфейсе:

- идентификатор (первичный ключ, автоинкремент);
- название (VARCHAR, обязательное);
- тип (книга, видео, аудио, курс, подкаст);
- авторы (текст);
- описание (текст);
- уровень (начальный, средний, продвинутый);
- язык (значение по умолчанию 'ru');
- служебные поля дата_создания, дата_обновления (заполняются автоматически).

Примеры SQL-запросов, формируемых приложением (все операции выполняются в рамках транзакций с откатом при ошибке):

```
-- Получение всех записей с возможностью поиска
SELECT * FROM контент WHERE название ILIKE '%python%' OR авторы ILIKE '%python%' ORDER BY идентификатор;

-- Вставка нового контента
INSERT INTO контент (название, тип, авторы, описание, уровень, язык)
VALUES ('Python для начинающих', 'книга', 'Марк Лутц', 'Изучение Python с нуля', 'начальный', 'ru');

-- Обновление записи
UPDATE контент SET название='Python. Продвинутый уровень',
уровень='продвинутый', дата_обновления = CURRENT_TIMESTAMP
WHERE идентификатор = 13;

-- Удаление записи
DELETE FROM контент WHERE идентификатор = 13;
```

Описание веб-интерфейса

Интерфейс состоит из трёх основных страниц:

1. Список контента (/content)

Отображает таблицу со всеми материалами, поисковую строку и кнопку «Добавить контент». Для каждой записи доступны кнопки «Ред.» (редактировать) и «Удалить». При удалении выводится диалог подтверждения JavaScript.

2. Форма добавления (/content/add)

Поля формы соответствуют колонкам таблицы, для типов и уровней используются выпадающие списки, обязательное поле – название.

3. Форма редактирования (/content/edit/<id>)

Форма, идентичная добавлению, но все поля предзаполнены текущими значениями. При сохранении автоматически обновляется дата изменения.

Все страницы имеют навигационную ссылку для возврата к списку. Для визуального оформления использован встроенный CSS, имитирующий корпоративный стиль (цветовая гамма в оттенках синего).

Результаты работы модуля

На рисунках 1–5 представлены снимки экрана, демонстрирующие работу модуля, развёрнутого на виртуальном стенде (VM2, IP 192.168.56.20, порт 5000). Подключение к базе данных производилось к серверу PostgreSQL на VM1 (192.168.56.10). Все операции выполнены успешно.

Мультимедиа контент

+ Добавить контент

Найти

ID	Название	Тип	Авторы	Уровень	Язык	Действия
1	Программирование на Python для начинающих	книга	Марк Лутц	начальный	ru	Ред. Удалить
2	Машинное обучение: алгоритмы и практика	видео	Андрей Себрант	средний	ru	Ред. Удалить
3	Аудиокнига "Чистый код"	аудио	Роберт Мартин	продвинутый	ru	Ред. Удалить
4	Основы SQL и реляционных баз данных	курс	Иван Петров	начальный	ru	Ред. Удалить
5	Deep Learning with PyTorch	книга	Eli Stevens et al.	продвинутый	en	Ред. Удалить
6	Психология влияния	аудио	Роберт Чалдини	начальный	ru	Ред. Удалить
7	React для профессионалов	видео	Максим Иванов	продвинутый	ru	Ред. Удалить
8	Английский для IT	курс	LinguaLeo	средний	ru	Ред. Удалить
9	Микросервисы: паттерны и практики	книга	Крис Ричардсон	продвинутый	ru	Ред. Удалить
10	Подкаст "Люди и код" выпуск 45	подкаст	Алексей Котов	начальный	ru	Ред. Удалить

Рисунок 25 - Главная страница модуля со списком контента

Добавление нового контента

Название:

Тип:

Авторы:

Описание:

Уровень:

Язык:

[Отмена](#)

Рисунок 26 - Форма добавления нового контента

Мультимедиа контент

Контент успешно добавлен!

ID	Название	Тип	Авторы	Уровень	Язык	Действия
1	Программирование на Python для начинающих	книга	Марк Лутц	начальный	ru	Ред. Удалить
2	Машинное обучение: алгоритмы и практика	видео	Андрей Себрант	средний	ru	Ред. Удалить
3	Аудиокнига "Чистый код"	аудио	Роберт Мартин	продвинутый	ru	Ред. Удалить
4	Основы SQL и реляционных баз данных	курс	Иван Петров	начальный	ru	Ред. Удалить
5	Deep Learning with PyTorch	книга	Eli Stevens et al.	продвинутый	en	Ред. Удалить
6	Психология влияния	аудио	Роберт Чалдини	начальный	ru	Ред. Удалить
7	React для профессионалов	видео	Максим Иванов	продвинутый	ru	Ред. Удалить
8	Английский для IT	курс	LinguaLeo	средний	ru	Ред. Удалить
9	Микросервисы: паттерны и практики	книга	Крис Ричардсон	продвинутый	ru	Ред. Удалить

Рисунок 27 - Сообщение об успешном добавлении и обновлённая таблица

Редактирование контента #14

Название:

Тип:

Авторы:

Описание:

Уровень:

Язык:

[Отмена](#)

Рисунок 28 - Форма редактирования контента (предзаполненные поля)

Поиск по названию или ID		Найти		Подтвердите действие на 192.168.56.20:5000		Удалить?		Уровень		Язык		Действия	
ID	Название												
1	Программирование на Python для начинающих							начальный		ru		Ред.	Удалить
2	Машинное обучение: алгоритмы и практика	видео	Андрей Себрант					средний		ru		Ред.	Удалить
3	Аудиокнига "Чистый код"	аудио	Роберт Мартин					продвинутый		ru		Ред.	Удалить
4	Основы SQL и реляционных баз данных	курс	Иван Петров					начальный		ru		Ред.	Удалить
5	Deep Learning with PyTorch	книга	Eli Stevens et al.					продвинутый		en		Ред.	Удалить
6	Психология влияния	аудио	Роберт Чалдини					начальный		ru		Ред.	Удалить
7	React для профессионалов	видео	Максим Иванов					продвинутый		ru		Ред.	Удалить
8	Английский для IT	курс	LinguaLeo					средний		ru		Ред.	Удалить
9	Микросервисы: паттерны и практики	книга	Крис Ричардсон					продвинутый		ru		Ред.	Удалить
10	Подкаст "Люди и код" выпуск 45	подкаст	Алексей Котов					начальный		ru		Ред.	Удалить
11	Data Science с нуля	книга	Джозл Грас					начальный		ru		Ред.	Удалить
12	Архитектура компьютера	видео	Эндрю Таненбаум					средний		ru		Ред.	Удалить
14	Тестовый контент (ред.)	видео	Тестовый автор2 (изм.)					средний		ru, en		Ред.	Удалить

Рисунок 29 - Результат удаления элемента

Выводы

Разработан модуль «Менеджер контента» на Flask с psycopg2 и Jinja2. Реализованы: просмотр с поиском (LIKE), добавление, редактирование (с обновлением даты), удаление с подтверждением. Используются параметризованные запросы, транзакции, флеш-сообщения. Модуль развёрнут на VM2 (192.168.56.20:5000), подключён к PostgreSQL на VM1. Скриншоты подтверждают успешное выполнение операций.

РАЗДЕЛ 13. СХЕМА СЕТИ И АНАЛИЗ УГРОЗ ИНФОРМАЦИОННОЙ БЕЗОПАСНОСТИ

Введение

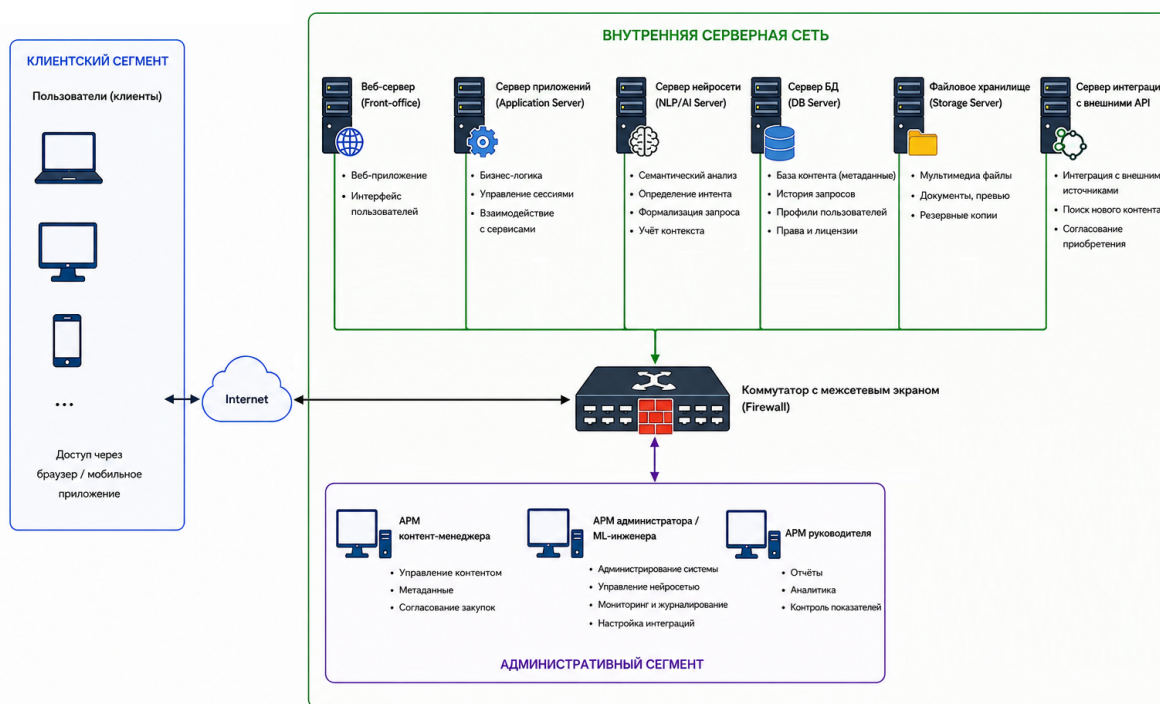
Для обеспечения информационной безопасности необходимо спроектировать схему сети и выявить актуальные угрозы. Раздел 13 основан на материалах практической работы №13.

Цель раздела 13 – построить схему компьютерной сети АСУ «Семантический подборщик» и определить угрозы безопасности хранения/обработки информации на рабочих станциях и угрозы передачи информации по сети.

Задачи раздела:

1. Разработать схему сети с сегментацией (DMZ, приложения, данные, управление).
2. Выделить угрозы для рабочих станций.
3. Выделить угрозы для каналов передачи данных.

Схема компьютерной сети проектируемой АСУ «Семантический подборщик»



Определение проблем информационной безопасности для построенной сети

Для проектируемой АСУ «Семантический подборщик» выделены следующие категории угроз безопасности, классифицированные в соответствии с требованиями задания: угрозы хранения и обработки информации на рабочих станциях и угрозы передачи информации по компьютерной сети.

Угрозы безопасности хранения и обработки информации на рабочих станциях

К рабочим станциям относятся АРМ пользователей (клиентов), контент-менеджеров, администратора нейросети и руководителя. На этих устройствах могут храниться временные данные (кэш браузера, загружаемые файлы, пароли, cookie-файлы), а также выполняться обработка запросов на стороне клиента (JavaScript-код портала).

Угроза	Описание	Возможные последствия
Несанкционированный доступ (НСД) к устройству	Физический доступ постороннего лица к разблокированному компьютеру/ноутбуку	Компрометация учётных данных, истории запросов, избранного; доступ к загруженному контенту
Вредоносное программное обеспечение (вирусы, шпионское ПО, кейлоггеры)	Заражение рабочей станции через фишинг, загрузку нелегального контента	Перехват вводимых паролей и текстов запросов; кража cookie и токенов сессии
Уязвимости операционной системы и прикладного ПО	Использование необновлённой ОС, браузера или плагинов с известными уязвимостями	Эксплуатация уязвимостей для выполнения произвольного кода, получения доступа к файловой системе и данным браузера
Фишинг и социальная инженерия	Пользователь может быть обманут и перейти по поддельной ссылке, имитирующей портал АСУ	Компрометация логина и пароля, передача их злоумышленнику
Хранение конфиденциальных данных в открытом виде	Сохранение паролей в браузере без мастер-пароля, кэширование загруженных файлов в незашифрованном виде	При компрометации устройства — утечка персональных данных и лицензионного контента
Недостаточная	Использование слабых паролей,	Облегчённый подбор или

Угроза	Описание	Возможные последствия
аутентификация пользователей	отсутствие двухфакторной аутентификации на уровне доступа к устройству	угадывание учётных данных, ведущий к несанкционированному доступу к системе
Нарушение конфиденциальности при локальной обработке	Клиентский JavaScript-код портала может быть модифицирован через атаку типа «человек посередине» или XSS, что приведёт к утечке данных из памяти браузера	Перехват текстов запросов и параметров, кража токенов, подмена результатов поиска

Угрозы безопасности передачи информации по компьютерной сети

Данные передаются между клиентами и серверами, а также внутри сегментов ЦОД. Угрозы классифицированы по каналам передачи.

Угроза	Описание	Возможные последствия
Перехват трафика (сниффинг)	Пассивный перехват незашифрованного или слабо зашифрованного сетевого трафика в публичных сетях и локальных сегментах	Чтение текстов запросов, параметров поиска, метаданных контента, учётных данных (если не используется HTTPS)
Атака «человек посередине» (Man-in-the-Middle, MitM)	Злоумышленник вклинивается в канал связи между клиентом и сервером, подменяя или модифицируя трафик	Перехват сессионных cookie, подмена ответов поиска, перенаправление на фишинговые страницы
Подмена сервера (spoofing)	Перенаправление клиента на подставной сервер вследствие компрометации DNS или подмены ARP-таблиц	Полный контроль над сессией пользователя, кража учётных данных
Отказ в обслуживании (DoS/DDoS)	Массированная отправка запросов на балансировщик или веб-серверы с целью истощения ресурсов	Недоступность системы для легитимных пользователей, убытки от простоя
Уязвимости сетевых протоколов (TCP/IP, DNS, HTTP/2)	Эксплуатация известных уязвимостей в протоколах передачи данных (например, HTTP-request smuggling, DNS cache poisoning)	Обход межсетевых экранов, подмена трафика, утечка информации
Несанкционированное проникновение во внутреннюю сеть	Использование уязвимостей в веб-приложениях (SQL-инъекции, XSS, удалённое выполнение кода) для продвижения вглубь сети	Компрометация серверов БД, NLP-моделей, файлового хранилища; хищение или уничтожение данных
Недостаточная сегментация сети	Отсутствие строгих правил фильтрации между DMZ, сегментом приложений и данными позволяет злоумышленнику, скомпрометировавшему веб-сервер, свободно перемещаться	Доступ к базам данных, логам, хранилищу контента напрямую

Угроза	Описание	Возможные последствия
	по внутренней сети	
Утечка данных при взаимодействии с внешними источниками	Передача поисковых запросов или метаданных контента в API партнёров без должного шифрования или контроля	Раскрытие тем запросов пользователей, коммерческой тайны о закупках
Неавторизованный доступ через беспроводные сети	Подключение клиентов через незащищённые точки Wi-Fi (общественные сети) без обязательного использования VPN	Перехват сессионных данных, MitM-атаки на уровне точки доступа
Уязвимости в VPN-концентраторе или средствах удалённого доступа	Ошибки конфигурации VPN, использование устаревших протоколов (PPTP) или слабых ключей шифрования	Компрометация административных учётных записей и получение полного контроля над инфраструктурой

Выводы

Построена схема сети с сегментами: DMZ, приложения, данные, управление, клиентские АРМ (внутренние и удалённые через VPN). Выявлены 7 угроз для рабочих станций (НСД, ВПО, фишинг, уязвимости, хранение в открытом виде, слабая аутентификация, XSS). Выявлены 10 угроз для сетевой передачи (сниффинг, MitM, spoofing, DoS, уязвимости протоколов, проникновение в сеть, недостаточная сегментация, утечка при внешних API, неавторизованный Wi-Fi, уязвимости VPN). Угрозы учтены при выборе средств защиты в разделах 14–15.

РАЗДЕЛ 14. ВЫБОР СРЕДСТВ ЗАЩИТЫ ИНФОРМАЦИИ НА РАБОЧИХ СТАНЦИЯХ

Введение

На основе выявленных угроз для рабочих станций (раздел 13) необходимо выбрать программные средства защиты информации (СЗИ). Раздел 14 основан на материалах практической работы №14.

Цель раздела 14 – выбрать программные средства защиты информации, хранимой и обрабатываемой на рабочих станциях, в соответствии с используемыми аппаратными и программными средствами и с учётом выделенных угроз.

Задачи раздела:

1. Сформулировать требования к защите рабочих станций.
2. Провести сравнительный анализ трёх отечественных СЗИ.
3. Обосновать выбор и описать возможности выбранного решения.

Анализ угроз безопасности для рабочих станций проектируемой АСУ

Проектируемая АСУ «Семантический подборщик» предназначена для автоматического подбора и продажи мультимедийного контента по естественно-языковым запросам. Система включает рабочие станции трёх категорий пользователей: администратора, контент-менеджера и руководителя отдела. Рабочие станции представляют собой персональные компьютеры или ноутбуки под управлением ОС Windows 10/11 Pro, используемые для доступа к веб-интерфейсу АСУ через браузер, а также к служебным ресурсам. Через эти рабочие станции обрабатывается конфиденциальная информация, включая персональные данные пользователей (история запросов, адреса электронной почты, профили) и сведения о закупках контента.

На основе анализа архитектуры и бизнес-процессов системы выделены следующие актуальные угрозы безопасности для рабочих станций:

1. Заражение вредоносным программным обеспечением (ВПО) – возможно через фишинговые письма, загрузку файлов из Интернета, подключаемые съёмные носители. Последствиями могут стать хищение учётных данных, шифрование данных, удалённое управление рабочей станцией.
2. Несанкционированный доступ (НСД) к локальным данным – при физическом доступе постороннего к оставленной без присмотра рабочей станции или в случае кражи/утери ноутбука.
3. Утечка конфиденциальных данных – несанкционированное копирование ПДн пользователей, метаданных контента, отчётов о закупках на внешние носители, передача через облачные сервисы или по электронной почте.
4. Сетевые атаки на рабочую станцию – сканирование портов, эксплуатация уязвимостей ОС/ПО, атаки типа «человек посередине» при работе в незащищённой сети.
5. Компрометация учётных записей – подбор слабых паролей, использование одних и тех же учётных данных на внешних ресурсах, отсутствие двухфакторной аутентификации.

Указанные угрозы способны привести к нарушению конфиденциальности персональных данных (ФЗ-152), целостности служебной информации и доступности самой АСУ.

Требования к защите рабочих станций

С учётом выделенных угроз, а также нефункциональных требований, и нормативных документов (ФЗ-152, методика оценки угроз ФСТЭК), программные средства защиты рабочих станций должны обеспечивать:

- защиту от вредоносного ПО в режиме реального времени;
- персональный межсетевой экран с контролем входящего/исходящего

трафика;

- контроль и блокировку несанкционированного использования съёмных носителей;
- шифрование жёсткого диска для предотвращения утечки данных при утере устройства;
- разграничение доступа к локальным ресурсам и обязательную блокировку сеанса при бездействии;
- централизованное управление политиками безопасности, мониторинг событий и обновление антивирусных баз с единой консоли;
- совместимость с ОС Windows 10/11 Pro, низкое потребление системных ресурсов (в рамках характеристик рабочих станций).

Сравнительный анализ программных средств защиты

Для выбора оптимального решения были рассмотрены три отечественных продукта класса Endpoint Protection, имеющие сертификацию ФСТЭК и широко применяемые в корпоративных средах: Kaspersky Endpoint Security для бизнеса, Dr.Web Enterprise Security Suite и ViPNet Endpoint Protection.

Критерий	Kaspersky Endpoint Security	Dr.Web Enterprise Security Suite	ViPNet Endpoint Protection
Антивирусная защита (сигнатурный + облачный + эвристический анализ)	Да, высокий уровень обнаружения	Да, высокий уровень обнаружения	Да, средний уровень (фокус на сетевой защите)
Персональный межсетевой экран	Встроенный, настраиваемые правила	Встроенный, базовые возможности	Встроенный, продвинутые функции с контролем приложений
Контроль подключаемых устройств (USB, съёмные диски)	Да, гибкие политики доступа	Да, контроль по типам и серийным номерам	Да, интеграция с политиками НСД
Шифрование диска (FDE)	Встроенный модуль (на базе BitLocker или собственного)	Отсутствует (требуется стороннее ПО)	Поддержка шифрования совместно с ViPNet SafeDisk
Централизованное управление	Kaspersky Security Center, единая консоль, облачная/локальная версия	Dr.Web Security Control Center, локальная консоль	ViPNet Administrator, локальная консоль
Защита от шифровальщиков и поведенческий анализ	Да, System Watcher	Да, Preventive Protection	Частично (на уровне сетевого экрана)
Сертификация ФСТЭК	Сертифицирован (НДВ, СОВ, АУЗ)	Сертифицирован (НДВ, СОВ)	Сертифицирован для ГИС и значимых объектов КИИ
Совместимость с Windows 10/11	Полная	Полная	Полная
Потребление ресурсов	Среднее	Низкое	Среднее

Каждый из продуктов обладает достаточным базовым функционалом для защиты рабочих станций. Dr.Web отличается низкой ресурсоёмкостью, что важно для маломощных машин, но не имеет встроенного шифрования диска. ViPNet ориентирован на комплексную защиту сетей и объектов КИИ, однако его антивирусные возможности уступают конкурентам. Kaspersky Endpoint Security выделяется наиболее сбалансированным набором функций, включая шифрование, поведенческий анализ и удобное централизованное управление.

Выбор и описание программных средств защиты для рабочих станций

В качестве программного средства защиты информации, хранимой и обрабатываемой на рабочих станциях проектируемой АСУ, выбирается Kaspersky Endpoint Security для бизнеса – Стандартный с консолью управления Kaspersky Security Center. Выбор обоснован следующими аргументами:

- Полное покрытие всех выделенных угроз: антивирусная защита + брандмауэр + контроль устройств + шифрование диска в одном агенте.
- Встроенный модуль шифрования на основе технологий BitLocker (с возможностью хранения ключей в KSC) позволяет выполнить требования по защите данных при утере/краже ноутбука без приобретения дополнительного ПО.
- Компонент «System Watcher» обеспечивает поведенческий анализ и защиту от шифровальщиков, что критически важно при возможном проникновении через фишинг.
- Централизованное управление через Kaspersky Security Center даёт возможность администратору нейросети единообразно настраивать политики для всех рабочих станций, отслеживать инциденты и автоматически обновлять базы и версии ПО.
- Продукт сертифицирован ФСТЭК России, что необходимо для легитимного использования в системе, обрабатывающей персональные данные.

Описание возможностей выбранного решения (в контексте АСУ «Семантический подборщик»):

- Защита от вредоносных программ. Многоуровневая система обнаружения угроз защищает рабочие станции от вирусов, троянов,

шпионского ПО. Это предотвращает кражу учётных данных контент-менеджеров и администратора.

- Межсетевой экран. Персональный брандмауэр ограничивает сетевую активность приложений, запрещая неавторизованные исходящие соединения, что снижает риск утечек данных и препятствует управлению заражённой рабочей станцией извне.
- Контроль устройств. Гибкие политики позволяют разрешить только корпоративные USB-накопители, запретить подключение личных смартфонов и внешних дисков. Это исключает несанкционированное копирование выборок из базы контента, логов запросов и ПДн клиентов.
- Шифрование данных. Полное шифрование системного диска гарантирует невозможность чтения хранимой на рабочей станции информации (включая кэшированные браузером данные, временные файлы с отчётами) в случае кражи ноутбука.
- Защита от шифровальщиков. Поведенческий компонент System Watcher отслеживает подозрительную активность процессов (массовое изменение файлов) и при обнаружении атаки автоматически создаёт резервные копии изменяемых файлов, а также блокирует вредоносный процесс.
- Централизованный мониторинг и аудит. Все события безопасности (срабатывание антивируса, попытки подключения запрещённых устройств, ошибки аутентификации) консолидируются в Kaspersky Security Center, что позволяет администратору АСУ своевременно реагировать на инциденты.

Совместно с данным программным средством на рабочих станциях применяются встроенные механизмы ОС Windows: BitLocker (для шифрования, управляемый KES), Windows Hello или двухфакторная аутентификация (аппаратные токены, смарт-карты), а также обязательная

блокировка экрана после 5 минут простоя. Эти меры дополняют защиту от НСД.

Выводы

Сформулированы требования: защита от ВПО, персональный брандмауэр, контроль USB, шифрование диска, централизованное управление, совместимость с Windows, низкое потребление ресурсов, сертификация ФСТЭК. Проведён сравнительный анализ Kaspersky, Dr.Web, VipNet. Выбран Kaspersky Endpoint Security (Стандартный) с консолью KSC. Обоснование: полное покрытие угроз, встроенное шифрование, поведенческий анализ, контроль устройств, централизованное управление, сертификация. Описаны возможности. Дополнительно используются BitLocker, двухфакторная аутентификация, блокировка экрана.

РАЗДЕЛ 15. ВЫБОР СРЕДСТВ ЗАЩИТЫ ПЕРЕДАЧИ ИНФОРМАЦИИ ПО СЕТИ

Введение

Для защиты каналов передачи данных, выявленных в разделе 13, необходимо выбрать комплекс средств. Раздел 15 основан на материалах практической работы №15.

Цель раздела 15 – выбрать средства обеспечения безопасности передачи информации по компьютерной сети для АСУ «Семантический подборщик» с учётом выделенных угроз.

Задачи раздела:

1. Проанализировать угрозы передачи данных и сформулировать требования.
2. Провести сравнительный анализ основных средств защиты (TLS, WireGuard, IPsec, Istio mTLS, NGFW).
3. Обосновать выбор комплекса средств и описать их возможности.

Анализ угроз безопасности передачи информации в сети проектируемой АСУ

Согласно архитектуре АСУ информационное взаимодействие осуществляется по следующим каналам:

- взаимодействие внешних пользователей с веб-интерфейсом через сеть Интернет;
- обмен данными между микросервисами (FastAPI, Elasticsearch, PostgreSQL, NLP-модуль) внутри центра обработки данных;
- обращение модуля внешнего поиска к API партнёрских платформ по сети Интернет;
- удалённое администрирование и мониторинг компонентов системы администратором нейросети и техническим персоналом.

На основе модели угроз и нефункциональных требований выделены следующие актуальные угрозы безопасности передаваемой информации:

- перехват трафика (сниффинг) – пассивный перехват данных при передаче по незащищённым каналам на уровне провайдера или локальной сети. Актуально для пользовательских сессий и межсервисного обмена;
- атака «Человек посередине» – активный перехват с возможностью модификации трафика. Например, подмена сертификата веб-сервера, перехват сессионных cookie;
- несанкционированный доступ к сетевым ресурсам – попытки прямого подключения к портам СУБД, поискового движка или файлового хранилища в обход прикладной логики;
- утечка данных при взаимодействии с внешними API – передача конфиденциальных параметров запроса или API-ключей в открытом виде к внешним источникам контента;
- компрометация канала управления – перехват учётных данных администратора или управляющих команд при удалённом администрировании (SSH, Kubernetes API);
- внутренние атаки типа «Восток-Запад» – распространение вредоносного трафика между микросервисами внутри ЦОД после компрометации одного из контейнеров.

Требования к средствам защиты передачи данных

С учётом выделенных угроз, нормативных документов (ФЗ №152 «О персональных данных», ГОСТ 34.10-2018, приказы ФСТЭК) и нефункциональных требований к системе, средства защиты должны обеспечивать:

- шифрование и аутентификацию всех внешних клиент-серверных соединений;

- организацию защищённых туннелей для удалённого администрирования и связи между распределёнными сегментами;
- взаимную аутентификацию и шифрование трафика между микросервисами внутри ЦОД;
- фильтрацию и контроль трафика на периметре сети с функциями предотвращения вторжений;
- защищённое хранение и передачу ключевой информации и паролей;
- соответствие требованиям по локализации персональных данных и сертификации средств криптозащиты при использовании на территории РФ.

Сравнительный анализ существующих средств обеспечения безопасности передачи информации

В таблице 3 представлены основные классы средств защиты передачи данных и их представители, релевантные для проектируемой АСУ.

Таблица 3 - Сравнение средств защиты передачи данных

Критерий / Средство	TLS 1.3 / HTTPS	WireGuard VPN	IPsec (StrongSwan)	Istio Service Mesh (mTLS)	NGFW (UserGate, ViPNet)
Уровень модели OSI	Транспортный / Прикладной	Сетевой (L3)	Сетевой (L3)	Прикладной (L7)	L3-L7
Основное назначение	Защита веб-трафика клиент-сервер	Простое высокопроизводительное туннелирование	Универсальное туннелирование и site-to-site	Взаимная аутентификация и шифрование сервисов в K8s	Защита периметра, межсетевое экранирование, IPS/IDS, VPN
Криптостойкость	AES-256-GCM, ChaCha20-Poly1305, ECDHE	ChaCha20-Poly1305, Curve25519	AES-256-GCM, IKEv2	Автоматическая ротация сертификатов, AES-GCM	Зависит от реализации VPN (обычно IPsec, OpenVPN)
Производительность	Небольшие накладные расходы на CPU	Очень высокая, минимальные задержки	Средняя, может требовать аппаратного ускорения	Доп. задержка из-за sidecar-прокси (Envoy)	Высокая (специализированное железо/ПО)
Удобство управления	Простота (настройка)	Простота (конфигурация)	Сложность (настройка)	Высокое (декларативная)	Наличие единой

	веб-сервера)	я в несколько строк)	политик IKE, маршрутизации)	я конфигурация, интеграция с K8s)	консоли, централизованный аудит
Сертификация ФСТЭК	СКЗИ класса КС1/КС2 в составе ПО	Не сертифицирован как самостоятельное СКЗИ	Сертифицирован в составе СКЗИ	Не применимо (инфраструктурный слой)	Полная сертификация всего устройства

Выбор и обоснование комплекса средств защиты

На основе архитектуры проектируемой АСУ и сравнительного анализа, для обеспечения безопасности передачи информации предлагается эшелонированный подход, включающий следующие средства:

1. TLS 1.3 / HTTPS – основной протокол защиты взаимодействия пользователей с веб-интерфейсом системы через Интернет. Обеспечивает конфиденциальность и целостность сессии, предотвращает перехват и MitM-атаки. SSL-терминация выполняется на Nginx-балансировщике, согласно выбранной архитектуре.
2. Межсетевой экран нового поколения с функциями VPN – выбран в качестве аппаратно-программной платформы для защиты периметра ЦОД. Обеспечивает:
 - фильтрацию входящего и исходящего трафика (Firewall);
 - обнаружение и предотвращение сетевых атак (IPS/IDS);
 - организацию защищённого удалённого доступа (VPN) для администратора нейросети и контент-менеджеров по протоколу IPsec или ViPNet-туннелю, что сертифицировано ФСТЭК и удовлетворяет требованиям по защите ПДн.
3. Istio Service Mesh (mTLS) – для защиты межсервисного трафика внутри кластера Kubernetes. Принудительное использование mutual TLS гарантирует, что все вызовы между модулями FastAPI, NLP-сервисом и Elasticsearch шифруются, а сервисы взаимно аутентифицируются, что полностью нейтрализует угрозу типа

«Восток-Запад» и предотвращает несанкционированный доступ к базам данных в обход API.

4. WireGuard – легковесный VPN для быстрого и безопасного соединения с сервером резервного копирования или внешним хранилищем данных (S3), если они территориально удалены.
5. Протокол SSHv2 – для защищённого консольного администрирования серверных операционных систем, используемый только из изолированного сегмента управления.

Описание возможностей выбранных средств с учётом угроз

TLS 1.3 / HTTPS

Нейтрализуемая угроза: пассивный перехват, MitM.

Возможности: шифрование всего трафика между браузером пользователя и точкой входа; использование стойких шифронаборов, эфемерных ключей Диффи-Хеллмана для обеспечения прямой секретности; применение сертификатов, выпущенных доверенным центром, предотвращает подмену сервера.

NGFW (ViPNet Coordinator HW)

Нейтрализуемая угроза: сетевые атаки, сканирование портов, несанкционированный доступ извне, утечка данных при администрировании.

Возможности: осуществляет глубокий анализ пакетов, блокирует попытки эксплуатации уязвимостей на периметре; создаёт криптографически защищённый туннель между рабочей станцией администратора и внутренней сетью ЦОД, обеспечивая конфиденциальность и аутентификацию управляющих команд; ведёт централизованный лог событий безопасности, служит точкой аудита.

Istio Service Mesh (mTLS)

Нейтрализуемая угроза: несанкционированный доступ к внутренним сервисам, подмена сервиса, прослушивание внутреннего трафика.

Возможности: автоматически внедряет sidecar-прокси к каждому поду, который прозрачно шифрует весь TCP-трафик между микросервисами; использует собственный центр сертификации (Citadel) для выдачи и ротации короткоживущих сертификатов X.509 каждому сервису; политики авторизации позволяют ограничить, например, прямой доступ к PostgreSQL только от сервиса авторизации, блокируя любые другие соединения, даже изнутри кластера.

WireGuard

Нейтрализуемая угроза: перехват резервных копий или трафика синхронизации между ЦОД.

Возможности: обеспечивает высокопроизводительное шифрованное соединение точка-точка с аутентификацией на основе открытых ключей; простота конфигурации и минимальная поверхность атаки благодаря малому объёму кода.

Выводы

Предложен эшелонированный подход: TLS 1.3/HTTPS для веб-трафика; NGFW (ViPNet Coordinator) для периметра и удалённого доступа; Istio Service Mesh с mTLS для шифрования и аутентификации микросервисов внутри Kubernetes; WireGuard для соединения с удалённым хранилищем; SSHv2 для администрирования. Выбранные средства нейтрализуют все угрозы, выявленные в разделе 13 (сниффинг, MitM, spoofing, DoS, внутренние атаки, недостаточную сегментацию и др.), и соответствуют требованиям ФСТЭК.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы спроектирована автоматизированная система управления подбором и продажей мультимедиа контента – АСУ «Семантический подборщик». Выполнены все поставленные задачи, результаты которых кратко обобщены ниже.

1. Функциональное проектирование. Определён участок автоматизации – обработка естественно-языковых запросов пользователей и интеллектуальный подбор контента с учётом контекста. Разработана функциональная структура, включающая 6 подсистем. Сформулированы 20 функциональных и 20 нефункциональных требований, охватывающих производительность (время отклика ≤ 2 с, доступность $\geq 99.5\%$), безопасность (TLS 1.3, разграничение доступа, локализация ПДн в РФ), удобство и архитектуру (микросервисы, контейнеризация).
2. UML-моделирование. Выделены три модуля («Интерфейс взаимодействия», «Управление контентом», «Аналитика и администрирование»), определены пять пользователей. Построены диаграммы вариантов использования (с отношениями include/extend), диаграммы последовательности для трёх процессов (поиск контента, закупка, дообучение нейросети) и диаграмма классов из 15 сущностей.
3. Программно-аппаратная платформа. Выбраны: аппаратное обеспечение (кластеры серверов, GPU-серверы NVIDIA, объектное хранилище, сетевое оборудование 25/100 Гбит/с) и программное (Linux, Docker, Kubernetes, PostgreSQL, Elasticsearch, PyTorch, RuBERT, FastAPI, React, Keycloak, ELK, Prometheus). Реализован модуль автоматического дообучения нейросети на Python с использованием Hugging Face Transformers, MLflow и PyTorch.

4. База данных. На основе ER-диаграммы создана физическая БД PostgreSQL из 17 таблиц. Разработаны SQL-скрипты создания, наполнения (≥ 10 записей в каждой таблице) и аналитического запроса. База данных интегрирована с веб-модулями.
5. Виртуальный стенд и интерфейсы. Развёрнут стенд из трёх виртуальных машин (VirtualBox) с установленными PostgreSQL, Elasticsearch, MinIO, FastAPI, RuBERT. Разработаны: информационный web-портал для клиентов (HTML, CSS, JS, эмуляция поиска и статистики) и модуль «Менеджер контента» на Flask для CRUD-операций с таблицей контента.
6. Информационная безопасность. Построена схема сети с сегментацией. Выявлены угрозы (НСД, ВПО, сниффинг, MitM, DoS и др.). Для рабочих станций выбрано Kaspersky Endpoint Security (сертифицирован ФСТЭК, включает антивирус, брандмауэр, контроль USB, шифрование диска). Для защиты каналов предложен эшелонированный подход: TLS 1.3, NGFW (ViPNet Coordinator), Istio Service Mesh с mTLS, WireGuard, SSHv2.

Все разработанные компоненты успешно протестированы на виртуальном стенде и могут быть развёрнуты в промышленной среде с использованием контейнеризации и оркестрации. Спроектированная АСУ соответствует современным стандартам, законодательству РФ (ФЗ №152, ФЗ №242, ГК РФ) и позволяет полностью автоматизировать подбор мультимедиа контента, сократить время реакции и повысить удовлетворённость пользователей.

Таким образом, цель курсовой работы достигнута, все задачи решены. Полученные результаты могут служить основой для реального внедрения АСУ «Семантический подборщик» в организации, предоставляющей образовательные и развлекательные мультимедийные услуги.

СПИСОК ИСТОЧНИКОВ

1. ГОСТ 34.601-90. Информационная технология. Комплекс стандартов на автоматизированные системы. Стадии создания. – М.: Стандартинформ, 2009.
2. ГОСТ Р ИСО/МЭК 12207-2010. Информационная технология. Процессы жизненного цикла программных средств. – М.: Стандартинформ, 2011.
3. ГОСТ Р ИСО 9241-11-2010. Эргономика взаимодействия человек-система. Часть 11. Пригодность. – М.: Стандартинформ, 2011.
4. Федеральный закон №152-ФЗ от 27.07.2006 «О персональных данных» (ред. от 14.07.2022).
5. Федеральный закон №242-ФЗ от 21.07.2014 «О внесении изменений в отдельные законодательные акты Российской Федерации в части уточнения порядка обработки персональных данных в информационно-телекоммуникационных сетях».
6. Гражданский кодекс РФ (часть четвертая) от 18.12.2006 № 230-ФЗ – раздел VII «Права на результаты интеллектуальной деятельности и средства индивидуализации».
7. Приказ ФСТЭК России от 11.02.2013 № 17 «Об утверждении Требований о защите информации, не составляющей государственную тайну, содержащейся в государственных информационных системах».
8. Буч Г., Рамбо Д., Якобсон А. Язык UML. Руководство пользователя. – 2-е изд. – М.: ДМК Пресс, 2021. – 496 с.
9. Фаулер М. UML. Основы. – 3-е изд. – СПб.: Символ-Плюс, 2018. – 192 с.
10. Ричардсон К. Микросервисы. Паттерны разработки и рефакторинга. – СПб.: Питер, 2020. – 544 с.

11. Документация PostgreSQL [Электронный ресурс]. – Режим доступа: <https://www.postgresql.org/docs/> (дата обращения: 21.05.2026).
12. Документация Elasticsearch [Электронный ресурс]. – Режим доступа: <https://www.elastic.co/guide/> (дата обращения: 21.05.2026).
13. Документация PyTorch [Электронный ресурс]. – Режим доступа: <https://pytorch.org/docs/> (дата обращения: 21.05.2026).
14. Документация Hugging Face Transformers [Электронный ресурс]. – Режим доступа: <https://huggingface.co/docs/transformers/> (дата обращения: 21.05.2026).
15. Документация Docker [Электронный ресурс]. – Режим доступа: <https://docs.docker.com/> (дата обращения: 21.05.2026).
16. Документация Kubernetes [Электронный ресурс]. – Режим доступа: <https://kubernetes.io/docs/> (дата обращения: 21.05.2026).
17. Котеров Д.В., Симдянов И.В. РНР 7. Основы. – СПб.: БХВ-Петербург, 2018. – 688 с. (использовано для общих принципов веб-разработки).
18. Петин В.А. Проектирование автоматизированных систем управления. – Томск: Изд-во ТПУ, 2017. – 180 с.
19. Макаров А.В., Михайлов С.А. Защита информации в автоматизированных системах. – М.: Горячая линия – Телеком, 2022. – 312 с.